CELT Reference Manual

0.3.2

Generated by Doxygen 1.5.3

Fri May 16 12:02:47 2008

Contents

Chapter 1

CELT Module Index

1.1 CELT Modules

Here is a list of all modules:

Encoding and decoding . . . . . **??**

Chapter 2

CELT Data Structure Index

2.1 CELT Data Structures

Here are the data structures with brief descriptions:

CELTDecoder (Decoder state) . . . . . ??
CELTEncoder (Encoder state) . . . . . ??
CELTHeader (Header data) . . . . . ??
CELTMode (Mode definition) . . . . . ??
ec_byte_buffer . . . . . ??
ec_dec . . . . . ??
ec_enc . . . . . ??
kiss_fft_cpx . . . . . ??
kiss_fft_state . . . . . ??
kiss_fftr_state . . . . . ??
kiss_twiddle_cpx . . . . . ??
mdct_lookup . . . . . ??
PsyDecay . . . . . ??

Chapter 3

CELT File Index

3.1 CELT File List

Here is a list of all files with brief descriptions:

libcelt/_kiss_fft_guts.h . . . ??
libcelt/arch.h (Various architecture definitions for CELT) . . . ??
libcelt/bands.c . . . ??
libcelt/bands.h . . . ??
libcelt/celt.c . . . ??
libcelt/celt.h (Contains all the functions for encoding and decoding audio streams) . . . ??
libcelt/celt_header.h . . . ??
libcelt/celt_types.h (CELT types) . . . ??
libcelt/compute_entropy_matrix.m . . . ??
libcelt/cwrs.c . . . ??
libcelt/cwrs.h . . . ??
libcelt/dump_hex.m . . . ??
libcelt/dump_modes.c . . . ??
libcelt/ecintrin.h . . . ??
libcelt/entcode.c . . . ??
libcelt/entcode.h . . . ??
libcelt/entdec.c . . . ??
libcelt/entdec.h . . . ??
libcelt/entenc.c . . . ??
libcelt/entenc.h . . . ??
libcelt/entropy.m . . . ??
libcelt/entropy2.m . . . ??
libcelt/fixed_c5x.h (Fixed-point operations for the TI C5x DSP family) . . . ??
libcelt/fixed_c6x.h (Fixed-point operations for the TI C6x DSP family) . . . ??
libcelt/fixed_debug.h (Fixed-point operations with debugging) . . . ??
libcelt/fixed_generic.h (Generic fixed-point operations) . . . ??
libcelt/header.c . . . ??
libcelt/kfft_double.h . . . ??
libcelt/kfft_single.c . . . ??
libcelt/kfft_single.h . . . ??
libcelt/kiss_fft.c . . . ??
libcelt/kiss_fft.h . . . ??
libcelt/kiss_fftr.c . . . ??

libcelt/kiss_fftr.h . . . ??
libcelt/laplace.c . . . ??
libcelt/laplace.h . . . ??
libcelt/mathops.h (Various math functions) . . . ??
libcelt/mdct.c . . . ??
libcelt/mdct.h . . . ??
libcelt/mfrngcod.h . . . ??
libcelt/modes.c . . . ??
libcelt/modes.h . . . ??
libcelt/os_support.h . . . ??
libcelt/pgain_table.h . . . ??
libcelt/pitch.c (Pitch analysis) . . . ??
libcelt/pitch.h (Pitch analysis) . . . ??
libcelt/plc.c . . . ??
libcelt/plot_alloc.m . . . ??
libcelt/plot_BER.m . . . ??
libcelt/plot_distribution.m . . . ??
libcelt/plot_entropy.m . . . ??
libcelt/plot_latency.m . . . ??
libcelt/plot_loss.m . . . ??
libcelt/plot_loss2.m . . . ??
libcelt/plot_PLC.m . . . ??
libcelt/psy.c . . . ??
libcelt/psy.h . . . ??
libcelt/quant_bands.c . . . ??
libcelt/quant_bands.h . . . ??
libcelt/quant_pitch.c . . . ??
libcelt/quant_pitch.h . . . ??
libcelt/rangedec.c . . . ??
libcelt/rangeenc.c . . . ??
libcelt/rate.c . . . ??
libcelt/rate.h . . . ??
libcelt/read_audio.m . . . ??
libcelt/stack_alloc.h (Temporary memory allocation on stack) . . . ??
libcelt/static_modes.c . . . ??
libcelt/static_modes.h . . . ??
libcelt/static_modes_rev.h . . . ??
libcelt/testcelt.c . . . ??
libcelt/useful.m . . . ??
libcelt/ventropy.m . . . ??
libcelt/vq.c . . . ??
libcelt/vq.h (Vector quantisation of the residual) . . . ??
libcelt/write_audio.m . . . ??

Chapter 4

CELT Module Documentation

4.1 Encoding and decoding

Functions

- CELTMode * celt_mode_create (celt_int32_t Fs, int channels, int frame_size, int lookahead, int *error)
- void celt_mode_destroy (CELTMode *mode)
- int celt_mode_info (const CELTMode *mode, int request, celt_int32_t *value)
- CELTEncoder * celt_encoder_create (const CELTMode *mode)
- void celt_encoder_destroy (CELTEncoder *st)
- int celt_encode (CELTEncoder *st, celt_int16_t *pcm, unsigned char *compressed, int nbCompressedBytes)
- CELTDecoder * celt_decoder_create (const CELTMode *mode)
- void celt_decoder_destroy (CELTDecoder *st)
- int celt_decode (CELTDecoder *st, unsigned char *data, int len, celt_int16_t *pcm)

4.1.1 Detailed Description

4.1.2 Function Documentation

4.1.2.1 int celt_decode (CELTDecoder * *st*, unsigned char * *data*, int *len*, celt_int16_t * *pcm*)

Decodes a frame of audio.

Parameters:

st Decoder state

data Compressed data produced by an encoder

len Number of bytes to read from "data". This MUST be exactly the number of bytes returned by the encoder. Using a larger value WILL NOT WORK.

pcm One frame (frame_size samples per channel) of decoded PCM will be returned here.

Returns:

Error code.

4.1.2.2 CELTDecoder* celt_decoder_create (const CELTMode * *mode*)

Creates a new decoder state. Each stream needs its own decoder state (can't be shared across simultaneous streams).

Parameters:

mode Contains all the information about the characteristics of the stream (must be the same characteristics as used for the encoder)

Returns:

Newly created decoder state.

Definition at line 437 of file celt.c.

References CELTDecoder::block_size, CELT_OK, CHANNELS, check_mode(), CELTDecoder::frame_size, CELTDecoder::last_pitch_index, MAX_PERIOD, CELTMode::mdctSize, CELTDecoder::mode, CELTMode::nbEBands, CELTDecoder::oldBandE, CELTDecoder::out_mem, CELTMode::overlap, CELTDecoder::overlap, and CELTDecoder::preemph_memD.

4.1.2.3 void celt_decoder_destroy (CELTDecoder * *st*)

Destroys a a decoder state.

Parameters:

st Decoder state to be destroyed

Definition at line 464 of file celt.c.

References CELT_OK, check_mode(), CELTDecoder::mode, CELTDecoder::oldBandE, CELTDecoder::out_mem, and CELTDecoder::preemph_memD.

4.1.2.4 int celt_encode (CELTEncoder * *st*, celt_int16_t * *pcm*, unsigned char * *compressed*, int *nbCompressedBytes*)

Encodes a frame of audio.

Parameters:

st Encoder state

pcm PCM audio in signed 16-bit format (native endian). There must be exactly frame_size samples per channel. The input data is overwritten by a copy of what the remote decoder would decode.

compressed The compressed data is written here

nbCompressedBytes Number of bytes to use for compressing the frame (can change from one frame to another)

Returns:

Number of bytes written to "compressed". Should be the same as "nbCompressedBytes" unless the stream is VBR. If negative, an error has occured (see error codes). It is IMPORTANT that the length returned be somehow transmitted to the decoder. Otherwise, no decoding is possible.

4.1.2.5 CELTEncoder* celt_encoder_create (const CELTMode * *mode*)

Creates a new encoder state. Each stream needs its own encoder state (can't be shared across simultaneous streams).

Parameters:

mode Contains all the information about the characteristics of the stream (must be the same characteristics as used for the decoder)

Returns:

Newly created encoder state.

Definition at line 78 of file celt.c.

References CELTEncoder::block_size, CELTEncoder::buf, CELT_OK, check_mode(), ec_byte_writeinit(), ec_enc_init(), CELTEncoder::enc, CELTEncoder::frame_size, CELTEncoder::in_mem, MAX_PERIOD, CELTMode::mdctSize, CELTEncoder::mode, N, CELTMode::nbChannels, CELTMode::nbEBands, CELTEncoder::oldBandE, CELTEncoder::out_mem, CELTMode::overlap, CELTEncoder::overlap, CELTEncoder::preemph_memD, and CELTEncoder::preemph_memE.

4.1.2.6 void celt_encoder_destroy (CELTEncoder * *st*)

Destroys a an encoder state.

Parameters:

st Encoder state to be destroyed

Definition at line 109 of file celt.c.

References CELTEncoder::buf, CELT_OK, check_mode(), ec_byte_writeclear(), CELTEncoder::in_mem, CELTEncoder::mode, CELTEncoder::oldBandE, CELTEncoder::out_mem, CELTEncoder::preemph_memD, and CELTEncoder::preemph_memE.

4.1.2.7 CELTMode* celt_mode_create (celt_int32_t *Fs*, int *channels*, int *frame_size*, int *lookahead*, int * *error*)

Creates a new mode struct. This will be passed to an encoder or decoder. The mode MUST NOT BE DESTROYED until the encoders and decoders that use it are destroyed as well.

Parameters:

Fs Sampling rate (32000 to 64000 Hz)

channels Number of channels

frame_size Number of samples (per channel) to encode in each packet (64 - 256)

lookahead Extra latency (in samples per channel) in addition to the frame size (between 32 and frame_size). The larger that value, the better the quality (at the expense of latency)

error Returned error code (if NULL, no error will be returned)

Returns:

A newly created mode

Definition at line 270 of file modes.c.

Referenced by main().

4.1.2.8 void celt_mode_destroy (CELTMode * *mode*)

Destroys a mode struct. Only call this after all encoders and decoders using this mode are destroyed as well.

Parameters:

mode Mode to be destroyed

Definition at line 387 of file modes.c.

Referenced by main().

4.1.2.9 int celt_mode_info (const CELTMode * *mode*, int *request*, celt_int32_t * *value*)

Query information from a mode

Definition at line 55 of file modes.c.

Referenced by celt_header_init(), and main().

Chapter 5

CELT Data Structure Documentation

5.1 CELTDecoder Struct Reference

Decoder state.

Collaboration diagram for CELTDecoder:

Data Fields

- const CELTMode * mode
- int frame_size
- int block_size
- int overlap
- ec_byte_buffer buf
- ec_enc enc
- celt_sig_t *restrict preemph_memD
- celt_sig_t * out_mem
- celt_word16_t * oldBandE
- int last_pitch_index

5.1.1 Detailed Description

Decoder state.

Decoder state

Definition at line 419 of file celt.c.

5.1.2 Field Documentation

5.1.2.1 const CELTMode* CELTDecoder::mode

Definition at line 420 of file celt.c.

Referenced by celt_decoder_create(), and celt_decoder_destroy().

5.1.2.2 int CELTDecoder::frame_size

Definition at line 421 of file celt.c.

Referenced by celt_decoder_create().

5.1.2.3 int CELTDecoder::block_size

Definition at line 422 of file celt.c.

Referenced by celt_decoder_create().

5.1.2.4 int CELTDecoder::overlap

Definition at line 423 of file celt.c.

Referenced by celt_decoder_create().

5.1.2.5 ec_byte_buffer CELTDecoder::buf

Definition at line 425 of file celt.c.

5.1.2.6 ec_enc CELTDecoder::enc

Definition at line 426 of file celt.c.

5.1.2.7 celt_sig_t* restrict CELTDecoder::preemph_memD

Definition at line 428 of file celt.c.

Referenced by celt_decoder_create(), and celt_decoder_destroy().

5.1.2.8 celt_sig_t* CELTDecoder::out_mem

Definition at line 430 of file celt.c.

Referenced by celt_decoder_create(), and celt_decoder_destroy().

5.1.2.9 celt_word16_t* CELTDecoder::oldBandE

Definition at line 432 of file celt.c.

Referenced by celt_decoder_create(), and celt_decoder_destroy().

5.1.2.10 int CELTDecoder::last_pitch_index

Definition at line 434 of file celt.c.

Referenced by celt_decoder_create().

The documentation for this struct was generated from the following file:

- libcelt/celt.c

5.2 CELTEncoder Struct Reference

Encoder state.

Collaboration diagram for CELTEncoder:

Data Fields

- const CELTMode * mode
- int frame_size
- int block_size
- int overlap
- int channels
- ec_byte_buffer buf
- ec_enc enc
- celt_word16_t *restrict preemph_memE
- celt_sig_t *restrict preemph_memD
- celt_sig_t * in_mem
- celt_sig_t * out_mem
- celt_word16_t * oldBandE

5.2.1 Detailed Description

Encoder state.

Encoder state

Definition at line 59 of file celt.c.

5.2.2 Field Documentation

5.2.2.1 const CELTMode* CELTEncoder::mode

Mode used by the encoder

Definition at line 60 of file celt.c.

Referenced by celt_encoder_create(), and celt_encoder_destroy().

5.2.2.2 int CELTEncoder::frame_size

Definition at line 61 of file celt.c.

Referenced by celt_encoder_create().

5.2.2.3 int CELTEncoder::block_size

Definition at line 62 of file celt.c.

Referenced by celt_encoder_create().

5.2.2.4 int CELTEncoder::overlap

Definition at line 63 of file celt.c.

Referenced by celt_encoder_create().

5.2.2.5 int CELTEncoder::channels

Definition at line 64 of file celt.c.

5.2.2.6 ec_byte_buffer CELTEncoder::buf

Definition at line 66 of file celt.c.

Referenced by celt_encoder_create(), and celt_encoder_destroy().

5.2.2.7 ec_enc CELTEncoder::enc

Definition at line 67 of file celt.c.

Referenced by celt_encoder_create().

5.2.2.8 celt_word16_t* restrict CELTEncoder::preemph_memE

Definition at line 69 of file celt.c.

Referenced by celt_encoder_create(), and celt_encoder_destroy().

5.2.2.9 celt_sig_t* restrict CELTEncoder::preemph_memD

Definition at line 70 of file celt.c.

Referenced by celt_encoder_create(), and celt_encoder_destroy().

5.2.2.10 celt_sig_t* CELTEncoder::in_mem

Definition at line 72 of file celt.c.

Referenced by celt_encoder_create(), and celt_encoder_destroy().

5.2.2.11 celt_sig_t* CELTEncoder::out_mem

Definition at line 73 of file celt.c.

Referenced by celt_encoder_create(), and celt_encoder_destroy().

5.2.2.12 celt_word16_t* CELTEncoder::oldBandE

Definition at line 75 of file celt.c.

Referenced by celt_encoder_create(), and celt_encoder_destroy().

The documentation for this struct was generated from the following file:

- libcelt/celt.c

5.3 CELTHeader Struct Reference

Header data.

`#include <celt_header.h>`

Data Fields

- char codec_id [8]
- char codec_version [20]
- celt_int32_t version_id
- celt_int32_t header_size
- celt_int32_t sample_rate
- celt_int32_t nb_channels
- celt_int32_t frame_size
- celt_int32_t overlap
- celt_int32_t bytes_per_packet
- celt_int32_t extra_headers

5.3.1 Detailed Description

Header data.

Header data to be used for Ogg files (or possibly other encapsulation)

Definition at line 45 of file celt_header.h.

5.3.2 Field Documentation

5.3.2.1 char CELTHeader::codec_id[8]

MUST be "CELT " (four spaces)

Definition at line 46 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.2 char CELTHeader::codec_version[20]

Version used (as string)

Definition at line 47 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.3 celt_int32_t CELTHeader::version_id

Version id (negative for until stream is frozen)

Definition at line 48 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.4 celt_int32_t CELTHeader::header_size

Size of this header

Definition at line 49 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.5 celt_int32_t CELTHeader::sample_rate

Sampling rate of the original audio

Definition at line 50 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.6 celt_int32_t CELTHeader::nb_channels

Number of channels

Definition at line 51 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.7 celt_int32_t CELTHeader::frame_size

Samples per frame (per channel)

Definition at line 52 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.8 celt_int32_t CELTHeader::overlap

Overlapping samples (per channel)

Definition at line 53 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.9 celt_int32_t CELTHeader::bytes_per_packet

Number of bytes per compressed packet (0 if unknown)

Definition at line 54 of file celt_header.h.

Referenced by celt_header_init().

5.3.2.10 celt_int32_t CELTHeader::extra_headers

Number of additional headers that follow this header

Definition at line 55 of file celt_header.h.

Referenced by celt_header_init().

The documentation for this struct was generated from the following file:

- libcelt/celt_header.h

5.4 CELTMode Struct Reference

Mode definition.

`#include <modes.h>`

Collaboration diagram for CELTMode:

Data Fields

- celt_uint32_t marker_start
- celt_int32_t Fs
- int overlap
- int mdctSize
- int nbChannels
- int nbEBands
- int nbPBands
- int pitchEnd
- const celt_int16_t * eBands
- const celt_int16_t * pBands
- celt_word16_t ePredCoef
- int nbAllocVectors
- const celt_int16_t * allocVectors
- const celt_int16_t *const * bits
- mdct_lookup mdct
- kiss_fftr_cfg fft
- const celt_word16_t * window
- struct PsyDecay psy
- int * prob
- const celt_int16_t * energy_alloc
- celt_uint32_t marker_end

5.4.1 Detailed Description

Mode definition.

Mode definition (opaque)

Definition at line 71 of file modes.h.

5.4.2 Field Documentation

5.4.2.1 celt_uint32_t CELTMode::marker_start

Definition at line 72 of file modes.h.

Referenced by celt_mode_create(), celt_mode_destroy(), and check_mode().

5.4.2.2 celt_int32_t CELTMode::Fs

Definition at line 73 of file modes.h.

Referenced by celt_header_init(), celt_mode_create(), and dump_modes().

5.4.2.3 int CELTMode::overlap

Definition at line 74 of file modes.h.

Referenced by celt_decoder_create(), celt_encoder_create(), celt_header_init(), celt_mode_create(), celt_mode_info(), do_plc(), dump_header(), and dump_modes().

5.4.2.4 int CELTMode::mdctSize

Definition at line 75 of file modes.h.

Referenced by celt_decoder_create(), celt_encoder_create(), celt_header_init(), celt_mode_create(), celt_mode_info(), dump_header(), and dump_modes().

5.4.2.5 int CELTMode::nbChannels

Definition at line 76 of file modes.h.

Referenced by celt_encoder_create(), celt_header_init(), celt_mode_create(), celt_mode_info(), compute_alloc_cache(), dump_header(), dump_modes(), quant_energy(), renormalise_bands(), and unquant_energy().

5.4.2.6 int CELTMode::nbEBands

Definition at line 78 of file modes.h.

Referenced by celt_decoder_create(), celt_encoder_create(), celt_mode_create(), celt_mode_destroy(), compute_alloc_cache(), compute_allocation(), compute_band_energies(), denormalise_bands(), dump_modes(), normalise_bands(), quant_bands(), quant_energy(), quant_prob_alloc(), renormalise_bands(), stereo_mix(), unquant_bands(), and unquant_energy().

5.4.2.7 int CELTMode::nbPBands

Definition at line 79 of file modes.h.

Referenced by compute_pitch_gain(), dump_modes(), and pitch_quant_bands().

5.4.2.8 int CELTMode::pitchEnd

Definition at line 80 of file modes.h.

Referenced by compute_alloc_cache(), dump_modes(), quant_bands(), and unquant_bands().

5.4.2.9 const celt_int16_t* CELTMode::eBands

Definition for each "pseudo-critical band"

Definition at line 82 of file modes.h.

Referenced by celt_mode_create(), celt_mode_destroy(), compute_alloc_cache(), compute_band_energies(), denormalise_bands(), dump_modes(), normalise_bands(), quant_bands(), stereo_mix(), and unquant_bands().

5.4.2.10 const celt_int16_t* CELTMode::pBands

Definition of the bands used for the pitch

Definition at line 83 of file modes.h.

Referenced by celt_mode_destroy(), compute_pitch_gain(), dump_modes(), and pitch_quant_bands().

5.4.2.11 celt_word16_t CELTMode::ePredCoef

Prediction coefficient for the energy encoding

Definition at line 85 of file modes.h.

Referenced by celt_mode_create(), and dump_modes().

5.4.2.12 int CELTMode::nbAllocVectors

Number of lines in the matrix below

Definition at line 87 of file modes.h.

Referenced by compute_allocation(), and dump_modes().

5.4.2.13 const celt_int16_t* CELTMode::allocVectors

Number of bits in each band for several rates

Definition at line 88 of file modes.h.

Referenced by celt_mode_destroy(), compute_allocation(), and dump_modes().

5.4.2.14 const celt_int16_t* const* CELTMode::bits

Cache for pulses->bits mapping in each band

Definition at line 90 of file modes.h.

Referenced by celt_mode_destroy(), compute_alloc_cache(), and dump_modes().

5.4.2.15 mdct_lookup CELTMode::mdct

Definition at line 93 of file modes.h.

Referenced by celt_mode_create(), and celt_mode_destroy().

5.4.2.16 kiss_fftr_cfg CELTMode::fft

Definition at line 94 of file modes.h.

Referenced by celt_mode_create(), and celt_mode_destroy().

5.4.2.17 const celt_word16_t* CELTMode::window

Definition at line 96 of file modes.h.

Referenced by celt_mode_create(), celt_mode_destroy(), do_plc(), and dump_modes().

5.4.2.18 struct PsyDecay CELTMode::psy [read]

Definition at line 98 of file modes.h.

Referenced by celt_mode_create(), celt_mode_destroy(), and dump_modes().

5.4.2.19 int* CELTMode::prob

Definition at line 100 of file modes.h.

Referenced by celt_mode_create(), and celt_mode_destroy().

5.4.2.20 const celt_int16_t* CELTMode::energy_alloc

Definition at line 101 of file modes.h.

Referenced by celt_mode_destroy().

5.4.2.21 celt_uint32_t CELTMode::marker_end

Definition at line 103 of file modes.h.

Referenced by celt_mode_create(), celt_mode_destroy(), and check_mode().

The documentation for this struct was generated from the following file:

- libcelt/modes.h

5.5 ec_byte_buffer Struct Reference

`#include <entcode.h>`

Data Fields

- unsigned char * buf
- unsigned char * ptr
- long storage

5.5.1 Detailed Description

Definition at line 25 of file entcode.h.

5.5.2 Field Documentation

5.5.2.1 unsigned char* ec_byte_buffer::buf

Definition at line 26 of file entcode.h.

Referenced by ec_byte_look1(), ec_byte_look4(), ec_byte_read1(), ec_byte_read4(), ec_byte_readinit(), ec_byte_write1(), ec_byte_write4(), ec_byte_writeclear(), ec_byte_writecopy(), ec_byte_writeinit(), and ec_byte_writetrunc().

5.5.2.2 unsigned char* ec_byte_buffer::ptr

Definition at line 27 of file entcode.h.

Referenced by ec_byte_adv1(), ec_byte_adv4(), ec_byte_look1(), ec_byte_look4(), ec_byte_read1(), ec_byte_read4(), ec_byte_readinit(), ec_byte_write1(), ec_byte_write4(), ec_byte_writecopy(), ec_byte_writeinit(), and ec_byte_writetrunc().

5.5.2.3 long ec_byte_buffer::storage

Definition at line 28 of file entcode.h.

Referenced by ec_byte_look1(), ec_byte_look4(), ec_byte_read1(), ec_byte_read4(), ec_byte_readinit(), ec_byte_write1(), ec_byte_write4(), ec_byte_writecopy(), and ec_byte_writeinit().

The documentation for this struct was generated from the following file:

- libcelt/entcode.h

5.6 ec_dec Struct Reference

`#include <entdec.h>`

Collaboration diagram for ec_dec:

Data Fields

- ec_byte_buffer * buf
- int rem
- ec_uint32 rng
- ec_uint32 dif
- ec_uint32 nrm

5.6.1 Detailed Description

Definition at line 12 of file entdec.h.

5.6.2 Field Documentation

5.6.2.1 ec_byte_buffer* ec_dec::buf

Definition at line 14 of file entdec.h.

Referenced by ec_dec_init(), and ec_dec_tell().

5.6.2.2 int ec_dec::rem

Definition at line 16 of file entdec.h.

Referenced by ec_dec_init().

5.6.2.3 ec_uint32 ec_dec::rng

Definition at line 18 of file entdec.h.

Referenced by ec_dec_init(), ec_dec_tell(), ec_dec_update(), ec_decode(), and ec_decode_bin().

5.6.2.4 ec_uint32 ec_dec::dif

Definition at line 21 of file entdec.h.

Referenced by ec_dec_init(), ec_dec_update(), ec_decode(), and ec_decode_bin().

5.6.2.5 ec_uint32 ec_dec::nrm

Definition at line 23 of file entdec.h.

Referenced by ec_dec_update(), ec_decode(), and ec_decode_bin().

The documentation for this struct was generated from the following file:

- libcelt/entdec.h

5.7 ec_enc Struct Reference

`#include <entenc.h>`

Collaboration diagram for ec_enc:

Data Fields

- ec_byte_buffer * buf
- int rem
- size_t ext
- ec_uint32 rng
- ec_uint32 low

5.7.1 Detailed Description

Definition at line 13 of file entenc.h.

5.7.2 Field Documentation

5.7.2.1 ec_byte_buffer* ec_enc::buf

Definition at line 15 of file entenc.h.

Referenced by ec_enc_init(), and ec_enc_tell().

5.7.2.2 int ec_enc::rem

Definition at line 17 of file entenc.h.

Referenced by ec_enc_done(), ec_enc_init(), and ec_enc_tell().

5.7.2.3 size_t ec_enc::ext

Definition at line 19 of file entenc.h.

Referenced by ec_enc_done(), ec_enc_init(), and ec_enc_tell().

5.7.2.4 ec_uint32 ec_enc::rng

Definition at line 21 of file entenc.h.

Referenced by ec_enc_done(), ec_enc_init(), ec_enc_tell(), ec_encode(), and ec_encode_bin().

5.7.2.5 ec_uint32 ec_enc::low

Definition at line 23 of file entenc.h.

Referenced by ec_enc_done(), ec_enc_init(), ec_encode(), and ec_encode_bin().

The documentation for this struct was generated from the following file:

- libcelt/entenc.h

5.8 kiss_fft_cpx Struct Reference

`#include <kiss_fft.h>`

Data Fields

- kiss_fft_scalar r
- kiss_fft_scalar i

5.8.1 Detailed Description

Definition at line 69 of file kiss_fft.h.

5.8.2 Field Documentation

5.8.2.1 kiss_fft_scalar kiss_fft_cpx::r

Definition at line 70 of file kiss_fft.h.

Referenced by kiss_fft_stride(), kiss_fftr_twiddles(), and kiss_fftri().

5.8.2.2 kiss_fft_scalar kiss_fft_cpx::i

Definition at line 71 of file kiss_fft.h.

Referenced by kiss_fft_stride(), kiss_fftr_twiddles(), and kiss_fftri().

The documentation for this struct was generated from the following file:

- libcelt/kiss_fft.h

5.9 kiss_fft_state Struct Reference

`#include <_kiss_fft_guts.h>`

Collaboration diagram for kiss_fft_state:

Data Fields

- int nfft
- kiss_fft_scalar scale
- int factors [2 ∗MAXFACTORS]
- int ∗ bitrev
- kiss_twiddle_cpx twiddles [1]

5.9.1 Detailed Description

Definition at line 33 of file _kiss_fft_guts.h.

5.9.2 Field Documentation

5.9.2.1 int kiss_fft_state::nfft

Definition at line 34 of file _kiss_fft_guts.h.

Referenced by kiss_fft_alloc(), kiss_fft_stride(), kiss_fftr_twiddles(), kiss_fftri(), and kiss_ifft_stride().

5.9.2.2 kiss_fft_scalar kiss_fft_state::scale

Definition at line 36 of file _kiss_fft_guts.h.

Referenced by kiss_fft_alloc(), kiss_fft_stride(), and kiss_fftr_alloc().

5.9.2.3 int kiss_fft_state::factors[2 ∗MAXFACTORS]

Definition at line 38 of file _kiss_fft_guts.h.

Referenced by kiss_fft_alloc(), kiss_fft_stride(), kiss_fftr_inplace(), kiss_fftri(), and kiss_ifft_stride().

5.9.2.4 int∗ kiss_fft_state::bitrev

Definition at line 39 of file _kiss_fft_guts.h.

Referenced by kiss_fft_alloc(), kiss_fft_stride(), kiss_fftri(), and kiss_ifft_stride().

5.9.2.5 kiss_twiddle_cpx kiss_fft_state::twiddles[1]

Definition at line 40 of file _kiss_fft_guts.h.

Referenced by kiss_fft_alloc().

The documentation for this struct was generated from the following file:

- libcelt/_kiss_fft_guts.h

5.10 kiss_fftr_state Struct Reference

`#include <kiss_fftr.h>`

Collaboration diagram for kiss_fftr_state:

Data Fields

- kiss_fft_cfg substate
- kiss_twiddle_cpx * super_twiddles

5.10.1 Detailed Description

Definition at line 24 of file kiss_fftr.h.

5.10.2 Field Documentation

5.10.2.1 kiss_fft_cfg kiss_fftr_state::substate

Definition at line 25 of file kiss_fftr.h.

Referenced by kiss_fftr(), kiss_fftr_alloc(), kiss_fftr_inplace(), kiss_fftr_twiddles(), and kiss_fftri().

5.10.2.2 kiss_twiddle_cpx* kiss_fftr_state::super_twiddles

Definition at line 26 of file kiss_fftr.h.

Referenced by kiss_fftr_alloc(), kiss_fftr_twiddles(), and kiss_fftri().

The documentation for this struct was generated from the following file:

- libcelt/kiss_fftr.h

5.11 kiss_twiddle_cpx Struct Reference

`#include <kiss_fft.h>`

Data Fields

- kiss_twiddle_scalar r
- kiss_twiddle_scalar i

5.11.1 Detailed Description

Definition at line 74 of file kiss_fft.h.

5.11.2 Field Documentation

5.11.2.1 kiss_twiddle_scalar kiss_twiddle_cpx::r

Definition at line 75 of file kiss_fft.h.

5.11.2.2 kiss_twiddle_scalar kiss_twiddle_cpx::i

Definition at line 76 of file kiss_fft.h.

The documentation for this struct was generated from the following file:

- libcelt/kiss_fft.h

5.12 mdct_lookup Struct Reference

`#include <mdct.h>`

Collaboration diagram for mdct_lookup:

Data Fields

- int n
- kiss_fft_cfg kfft
- kiss_twiddle_scalar *restrict trig

5.12.1 Detailed Description

Definition at line 51 of file mdct.h.

5.12.2 Field Documentation

5.12.2.1 int mdct_lookup::n

Definition at line 52 of file mdct.h.

Referenced by mdct_backward(), mdct_forward(), and mdct_init().

5.12.2.2 kiss_fft_cfg mdct_lookup::kfft

Definition at line 53 of file mdct.h.

Referenced by mdct_backward(), mdct_clear(), mdct_forward(), and mdct_init().

5.12.2.3 kiss_twiddle_scalar* restrict mdct_lookup::trig

Definition at line 54 of file mdct.h.

Referenced by mdct_backward(), mdct_clear(), mdct_forward(), and mdct_init().

The documentation for this struct was generated from the following file:

- libcelt/mdct.h

5.13 PsyDecay Struct Reference

`#include <psy.h>`

Data Fields

- const celt_word16_t *restrict decayR

5.13.1 Detailed Description

Definition at line 36 of file psy.h.

5.13.2 Field Documentation

5.13.2.1 const celt_word16_t* restrict PsyDecay::decayR

Definition at line 38 of file psy.h.

Referenced by dump_modes(), psydecay_clear(), and psydecay_init().

The documentation for this struct was generated from the following file:

- libcelt/psy.h

Chapter 6

CELT File Documentation

6.1 libcelt/_kiss_fft_guts.h File Reference

```
#include "kiss_fft.h"
```

Data Structures

- struct kiss_fft_state

Defines

- #define MIN(a, b) ((a)<(b) ? (a):(b))
- #define MAX(a, b) ((a)>(b) ? (a):(b))
- #define MAXFACTORS 32
- #define EXT32(a) (a)
- #define S_MUL(a, b) ((a)*(b))
- #define C_MUL(m, a, b)
- #define C_MULC(m, a, b)
- #define C_MUL4(m, a, b) C_MUL(m,a,b)
- #define C_FIXDIV(c, div)
- #define C_MULBYSCALAR(c, s)
- #define CHECK_OVERFLOW_OP(a, op, b)
- #define C_ADD(res, a, b)
- #define C_SUB(res, a, b)
- #define C_ADDTO(res, a)
- #define C_SUBFROM(res, a)
- #define KISS_FFT_COS(phase) (kiss_fft_scalar) cos(phase)
- #define KISS_FFT_SIN(phase) (kiss_fft_scalar) sin(phase)
- #define HALF_OF(x) ((x)*.5)
- #define kf_cexp(x, phase)
- #define kf_cexp2(x, phase)

6.1.1 Define Documentation

6.1.1.1 #define C_ADD(res, a, b)

Value:

```
do { \
            CHECK_OVERFLOW_OP((a).r,+,(b).r)\
            CHECK_OVERFLOW_OP((a).i,+,(b).i)\
            (res).r=(a).r+(b).r;  (res).i=(a).i+(b).i; \
    }while(0)
```

Definition at line 172 of file _kiss_fft_guts.h.

Referenced by kiss_fftri().

6.1.1.2 #define C_ADDTO(res, a)

Value:

```
do { \
            CHECK_OVERFLOW_OP((res).r,+,(a).r)\
            CHECK_OVERFLOW_OP((res).i,+,(a).i)\
            (res).r += (a).r;  (res).i += (a).i;\
    }while(0)
```

Definition at line 184 of file _kiss_fft_guts.h.

6.1.1.3 #define C_FIXDIV(c, div)

Definition at line 162 of file _kiss_fft_guts.h.

Referenced by kiss_fftr_twiddles().

6.1.1.4 #define C_MUL(m, a, b)

Value:

```
do{ (m).r = (a).r*(b).r - (a).i*(b).i;\
        (m).i = (a).r*(b).i + (a).i*(b).r; }while(0)
```

Definition at line 153 of file _kiss_fft_guts.h.

Referenced by kiss_fftri().

6.1.1.5 #define C_MUL4(m, a, b) C_MUL(m,a,b)

Definition at line 160 of file _kiss_fft_guts.h.

6.1.1.6 #define C_MULBYSCALAR(c, s)

Value:

```
do{ (c).r *= (s);\
        (c).i *= (s); }while(0)
```

Definition at line 163 of file _kiss_fft_guts.h.

6.1.1.7 #define C_MULC(m, a, b)

Value:

```
do{ (m).r = (a).r*(b).r + (a).i*(b).i;\
        (m).i = (a).i*(b).r - (a).r*(b).i; }while(0)
```

Definition at line 156 of file _kiss_fft_guts.h.

Referenced by kiss_fftr_twiddles().

6.1.1.8 #define C_SUB(res, a, b)

Value:

```
do { \
            CHECK_OVERFLOW_OP((a).r,-,(b).r)\
            CHECK_OVERFLOW_OP((a).i,-,(b).i)\
            (res).r=(a).r-(b).r;  (res).i=(a).i-(b).i; \
    }while(0)
```

Definition at line 178 of file _kiss_fft_guts.h.

Referenced by kiss_fftri().

6.1.1.9 #define C_SUBFROM(res, a)

Value:

```
do {\
            CHECK_OVERFLOW_OP((res).r,-,(a).r)\
            CHECK_OVERFLOW_OP((res).i,-,(a).i)\
            (res).r -= (a).r;  (res).i -= (a).i; \
    }while(0)
```

Definition at line 191 of file _kiss_fft_guts.h.

6.1.1.10 #define CHECK_OVERFLOW_OP(a, op, b)

Definition at line 169 of file _kiss_fft_guts.h.

Referenced by kiss_fftr_twiddles().

6.1.1.11 #define EXT32(a) (a)

Definition at line 150 of file _kiss_fft_guts.h.

Referenced by kiss_fftr_twiddles().

6.1.1.12 #define HALF_OF(x) ((x)*.5)

Definition at line 212 of file _kiss_fft_guts.h.

Referenced by kiss_fftr_twiddles().

6.1.1.13 #define kf_cexp(x, phase)

Value:

```
do{ \
                (x)->r = KISS_FFT_COS(phase);\
                (x)->i = KISS_FFT_SIN(phase);\
        }while(0)
```

Definition at line 215 of file _kiss_fft_guts.h.

Referenced by kiss_fft_alloc(), and kiss_fftr_alloc().

6.1.1.14 #define kf_cexp2(x, phase)

Value:

```
do{ \
      (x)->r = TRIG_UPSCALE*celt_cos_norm((phase));\
      (x)->i = TRIG_UPSCALE*celt_cos_norm((phase)-32768);\
}while(0)
```

Definition at line 221 of file _kiss_fft_guts.h.

Referenced by kiss_fft_alloc(), and kiss_fftr_alloc().

6.1.1.15 #define KISS_FFT_COS(phase) (kiss_fft_scalar) cos(phase)

Definition at line 210 of file _kiss_fft_guts.h.

6.1.1.16 #define KISS_FFT_SIN(phase) (kiss_fft_scalar) sin(phase)

Definition at line 211 of file _kiss_fft_guts.h.

6.1.1.17 #define MAX(a, b) ((a)>(b) ? (a):(b))

Definition at line 19 of file _kiss_fft_guts.h.

6.1.1.18 #define MAXFACTORS 32

Definition at line 27 of file _kiss_fft_guts.h.

6.1.1.19 #define MIN(a, b) ((a)<(b) ? (a):(b))

Definition at line 18 of file _kiss_fft_guts.h.

6.1.1.20 #define S_MUL(a, b) ((a)∗(b))

Definition at line 152 of file _kiss_fft_guts.h.

Referenced by mdct_backward(), and mdct_forward().

6.2 libcelt/arch.h File Reference

Various architecture definitions for CELT.

```
#include "celt_types.h"
```

Defines

- #define celt_fatal(str) _celt_fatal(str, __FILE__, __LINE__);
- #define celt_assert(cond)
- #define celt_assert2(cond, message)
- #define IMUL32(a, b) ((a)*(b))
- #define ABS(x) ((x) < 0 ? (-(x)) : (x))
- #define ABS16(x) ((x) < 0 ? (-(x)) : (x))
- #define MIN16(a, b) ((a) < (b) ? (a) : (b))
- #define MAX16(a, b) ((a) > (b) ? (a) : (b))
- #define ABS32(x) ((x) < 0 ? (-(x)) : (x))
- #define MIN32(a, b) ((a) < (b) ? (a) : (b))
- #define MAX32(a, b) ((a) > (b) ? (a) : (b))
- #define Q15ONE 1.0f
- #define Q30ONE 1.0f
- #define SIG_SCALING 1.f
- #define SIG_SCALING_1 1.f
- #define NORM_SCALING 1.f
- #define NORM_SCALING_1 1.f
- #define ENER_SCALING 1.f
- #define ENER_SCALING_1 1.f
- #define PGAIN_SCALING 1.f
- #define PGAIN_SCALING_1 1.f
- #define DB_SCALING 1.f
- #define DB_SCALING_1 1.f
- #define EPSILON 1e-15f
- #define VERY_SMALL 1e-15f
- #define VERY_LARGE32 1e15f
- #define VERY_LARGE16 1e15f
- #define Q15_ONE ((celt_word16_t)1.f)
- #define Q15_ONE_1 ((celt_word16_t)1.f)
- #define QCONST16(x, bits) (x)
- #define QCONST32(x, bits) (x)
- #define NEG16(x) (-(x))
- #define NEG32(x) (-(x))
- #define EXTRACT16(x) (x)
- #define EXTEND32(x) (x)
- #define SHR16(a, shift) (a)
- #define SHL16(a, shift) (a)
- #define SHR32(a, shift) (a)
- #define SHL32(a, shift) (a)
- #define PSHR16(a, shift) (a)
- #define PSHR32(a, shift) (a)
- #define VSHR32(a, shift) (a)

- #define SATURATE16(x, a) (x)
- #define SATURATE32(x, a) (x)
- #define PSHR(a, shift) (a)
- #define SHR(a, shift) (a)
- #define SHL(a, shift) (a)
- #define SATURATE(x, a) (x)
- #define ROUND16(a, shift) (a)
- #define HALF32(x) (.5f*(x))
- #define ADD16(a, b) ((a)+(b))
- #define SUB16(a, b) ((a)-(b))
- #define ADD32(a, b) ((a)+(b))
- #define SUB32(a, b) ((a)-(b))
- #define MULT16_16_16(a, b) ((a)*(b))
- #define MULT16_16(a, b) ((celt_word32_t)(a)*(celt_word32_t)(b))
- #define MAC16_16(c, a, b) ((c)+(celt_word32_t)(a)*(celt_word32_t)(b))
- #define MULT16_32_Q11(a, b) ((a)*(b))
- #define MULT16_32_Q13(a, b) ((a)*(b))
- #define MULT16_32_Q14(a, b) ((a)*(b))
- #define MULT16_32_Q15(a, b) ((a)*(b))
- #define MULT16_32_Q16(a, b) ((a)*(b))
- #define MULT16_32_P15(a, b) ((a)*(b))
- #define MULT32_32_Q31(a, b) ((a)*(b))
- #define MAC16_32_Q11(c, a, b) ((c)+(a)*(b))
- #define MAC16_32_Q15(c, a, b) ((c)+(a)*(b))
- #define MAC16_16_Q11(c, a, b) ((c)+(a)*(b))
- #define MAC16_16_Q13(c, a, b) ((c)+(a)*(b))
- #define MAC16_16_P13(c, a, b) ((c)+(a)*(b))
- #define MULT16_16_Q11_32(a, b) ((a)*(b))
- #define MULT16_16_Q13(a, b) ((a)*(b))
- #define MULT16_16_Q14(a, b) ((a)*(b))
- #define MULT16_16_Q15(a, b) ((a)*(b))
- #define MULT16_16_P15(a, b) ((a)*(b))
- #define MULT16_16_P13(a, b) ((a)*(b))
- #define MULT16_16_P14(a, b) ((a)*(b))
- #define DIV32_16(a, b) (((celt_word32_t)(a))/(celt_word16_t)(b))
- #define PDIV32_16(a, b) (((celt_word32_t)(a))/(celt_word16_t)(b))
- #define DIV32(a, b) (((celt_word32_t)(a))/(celt_word32_t)(b))
- #define PDIV32(a, b) (((celt_word32_t)(a))/(celt_word32_t)(b))
- #define BYTES_PER_CHAR 1
- #define BITS_PER_CHAR 8
- #define LOG2_BITS_PER_CHAR 3

Typedefs

- typedef float celt_word16_t
- typedef float celt_word32_t
- typedef float celt_sig_t
- typedef float celt_norm_t
- typedef float celt_ener_t
- typedef float celt_pgain_t
- typedef float celt_mask_t

6.2.1 Detailed Description

Various architecture definitions for CELT.

Definition in file arch.h.

6.2.2 Define Documentation

6.2.2.1 #define ABS(x) ((x) < 0 ? (-(x)) : (x))

Absolute integer value.

Definition at line 52 of file arch.h.

6.2.2.2 #define ABS16(x) ((x) < 0 ? (-(x)) : (x))

Absolute 16-bit value.

Definition at line 53 of file arch.h.

6.2.2.3 #define ABS32(x) ((x) < 0 ? (-(x)) : (x))

Absolute 32-bit value.

Definition at line 56 of file arch.h.

6.2.2.4 #define ADD16(a, b) ((a)+(b))

Definition at line 180 of file arch.h.

Referenced by _celt_lpc(), alg_quant(), and find_spectral_pitch().

6.2.2.5 #define ADD32(a, b) ((a)+(b))

Definition at line 182 of file arch.h.

Referenced by alg_quant(), celt_decode(), celt_encode(), compute_masking(), compute_pitch_gain(), fir_mem16(), iir_mem16(), kiss_fftr_twiddles(), mdct_init(), and quant_pitch().

6.2.2.6 #define BITS_PER_CHAR 8

Definition at line 230 of file arch.h.

6.2.2.7 #define BYTES_PER_CHAR 1

Definition at line 229 of file arch.h.

6.2.2.8 #define celt_assert(cond)

Definition at line 46 of file arch.h.

Referenced by ncwrs_unext32(), and ncwrs_unext64().

6.2.2.9 #define celt_assert2(cond, message)

Definition at line 47 of file arch.h.

Referenced by alg_quant().

6.2.2.10 #define celt_fatal(str) _celt_fatal(str, __FILE__, __LINE__);

Definition at line 41 of file arch.h.

Referenced by denormalise_bands(), kf_work(), ki_work(), kiss_fft_stride(), kiss_ifft_stride(), and quant_energy().

6.2.2.11 #define DB_SCALING 1.f

Definition at line 145 of file arch.h.

6.2.2.12 #define DB_SCALING_1 1.f

Definition at line 146 of file arch.h.

6.2.2.13 #define DIV32(a, b) (((celt_word32_t)(a))/(celt_word32_t)(b))

Definition at line 213 of file arch.h.

Referenced by kiss_fft_alloc(), kiss_fftr_alloc(), and mdct_init().

6.2.2.14 #define DIV32_16(a, b) (((celt_word32_t)(a))/(celt_word16_t)(b))

Definition at line 211 of file arch.h.

Referenced by _celt_lpc(), and stereo_mix().

6.2.2.15 #define ENER_SCALING 1.f

Definition at line 140 of file arch.h.

Referenced by quant_energy().

6.2.2.16 #define ENER_SCALING_1 1.f

Definition at line 141 of file arch.h.

Referenced by quant_energy().

6.2.2.17 #define EPSILON 1e-15f

Definition at line 148 of file arch.h.

Referenced by compute_pitch_gain(), find_spectral_pitch(), intra_fold(), intra_prediction(), intra_unquant(), and stereo_mix().

6.2.2.18 #define EXTEND32(x) (x)

Definition at line 161 of file arch.h.

Referenced by _celt_lpc(), celt_encode(), fir_mem16(), iir_mem16(), mdct_init(), quant_pitch(), and stereo_mix().

6.2.2.19 #define EXTRACT16(x) (x)

Definition at line 160 of file arch.h.

Referenced by alg_quant(), compute_pitch_gain(), find_spectral_pitch(), fir_mem16(), and iir_mem16().

6.2.2.20 #define HALF32(x) (.5f∗(x))

Definition at line 178 of file arch.h.

6.2.2.21 #define IMUL32(a, b) ((a)∗(b))

Definition at line 50 of file arch.h.

Referenced by ec_dec_update(), ec_encode(), and ec_encode_bin().

6.2.2.22 #define LOG2_BITS_PER_CHAR 3

Definition at line 231 of file arch.h.

6.2.2.23 #define MAC16_16(c, a, b) ((c)+(celt_word32_t)(a)∗(celt_word32_t)(b))

Definition at line 186 of file arch.h.

Referenced by alg_quant(), compute_pitch_gain(), fir_mem16(), iir_mem16(), intra_fold(), intra_prediction(), intra_unquant(), and vq_index().

6.2.2.24 #define MAC16_16_P13(c, a, b) ((c)+(a)∗(b))

Definition at line 202 of file arch.h.

Referenced by _celt_lpc().

6.2.2.25 #define MAC16_16_Q11(c, a, b) ((c)+(a)∗(b))

Definition at line 200 of file arch.h.

6.2.2.26 #define MAC16_16_Q13(c, a, b) ((c)+(a)∗(b))

Definition at line 201 of file arch.h.

6.2.2.27 #define MAC16_32_Q11(c, a, b) ((c)+(a)∗(b))

Definition at line 197 of file arch.h.

6.2.2.28 #define MAC16_32_Q15(c, a, b) ((c)+(a)∗(b))

Definition at line 198 of file arch.h.

6.2.2.29 #define MAX16(a, b) ((a) > (b) ? (a) : (b))

Maximum 16-bit value.

Definition at line 55 of file arch.h.

6.2.2.30 #define MAX32(a, b) ((a) > (b) ? (a) : (b))

Maximum 32-bit value.

Definition at line 58 of file arch.h.

Referenced by stereo_mix().

6.2.2.31 #define MIN16(a, b) ((a) < (b) ? (a) : (b))

Maximum 16-bit value.

Definition at line 54 of file arch.h.

6.2.2.32 #define MIN32(a, b) ((a) < (b) ? (a) : (b))

Maximum 32-bit value.

Definition at line 57 of file arch.h.

Referenced by celt_mode_create().

6.2.2.33 #define MULT16_16(a, b) ((celt_word32_t)(a)∗(celt_word32_t)(b))

Definition at line 185 of file arch.h.

Referenced by _celt_lpc(), alg_quant(), celt_encode(), compute_masking(), find_spectral_pitch(), fir_mem16(), iir_mem16(), intra_fold(), intra_prediction(), intra_unquant(), quant_pitch(), and stereo_mix().

6.2.2.34 #define MULT16_16_16(a, b) ((a)∗(b))

Definition at line 184 of file arch.h.

Referenced by find_spectral_pitch().

6.2.2.35 #define MULT16_16_P13(a, b) ((a)∗(b))

Definition at line 208 of file arch.h.

6.2.2.36 #define MULT16_16_P14(a, b) ((a)∗(b))

Definition at line 209 of file arch.h.

6.2.2.37 #define MULT16_16_P15(a, b) ((a)∗(b))

Definition at line 207 of file arch.h.

6.2.2.38 #define MULT16_16_Q11_32(a, b) ((a)∗(b))

Definition at line 203 of file arch.h.

6.2.2.39 #define MULT16_16_Q13(a, b) ((a)∗(b))

Definition at line 204 of file arch.h.

Referenced by _celt_lpc().

6.2.2.40 #define MULT16_16_Q14(a, b) ((a)∗(b))

Definition at line 205 of file arch.h.

Referenced by stereo_mix().

6.2.2.41 #define MULT16_16_Q15(a, b) ((a)∗(b))

Definition at line 206 of file arch.h.

Referenced by alg_quant(), find_spectral_pitch(), intra_prediction(), intra_unquant(), pitch_quant_bands(), quant_bands(), and unquant_bands().

6.2.2.42 #define MULT16_32_P15(a, b) ((a)∗(b))

Definition at line 193 of file arch.h.

6.2.2.43 #define MULT16_32_Q11(a, b) ((a)∗(b))

Definition at line 188 of file arch.h.

6.2.2.44 #define MULT16_32_Q13(a, b) ((a)∗(b))

Definition at line 189 of file arch.h.

Referenced by denormalise_bands().

6.2.2.45 #define MULT16_32_Q14(a, b) ((a)∗(b))

Definition at line 190 of file arch.h.

6.2.2.46 #define MULT16_32_Q15(a, b) ((a)∗(b))

Definition at line 191 of file arch.h.

Referenced by celt_decode(), celt_encode(), compute_pitch_gain(), denormalise_bands(), mdct_backward(), and mdct_forward().

6.2.2.47 #define MULT16_32_Q16(a, b) ((a)∗(b))

Definition at line 192 of file arch.h.

6.2.2.48 #define MULT32_32_Q31(a, b) ((a)∗(b))

Definition at line 195 of file arch.h.

6.2.2.49 #define NEG16(x) (-(x))

Definition at line 158 of file arch.h.

Referenced by iir_mem16().

6.2.2.50 #define NEG32(x) (-(x))

Definition at line 159 of file arch.h.

Referenced by _celt_lpc().

6.2.2.51 #define NORM_SCALING 1.f

Definition at line 138 of file arch.h.

Referenced by alg_quant().

6.2.2.52 #define NORM_SCALING_1 1.f

Definition at line 139 of file arch.h.

6.2.2.53 #define PDIV32(a, b) (((celt_word32_t)(a))/(celt_word32_t)(b))

Definition at line 214 of file arch.h.

6.2.2.54 #define PDIV32_16(a, b) (((celt_word32_t)(a))/(celt_word16_t)(b))

Definition at line 212 of file arch.h.

6.2.2.55 #define PGAIN_SCALING 1.f

Definition at line 142 of file arch.h.

6.2.2.56 #define PGAIN_SCALING_1 1.f

Definition at line 143 of file arch.h.

6.2.2.57 #define PSHR(a, shift) (a)

Definition at line 172 of file arch.h.

6.2.2.58 #define PSHR16(a, shift) (a)

Definition at line 166 of file arch.h.

6.2.2.59 #define PSHR32(a, shift) (a)

Definition at line 167 of file arch.h.

Referenced by _celt_lpc(), fir_mem16(), iir_mem16(), intra_fold(), intra_prediction(), intra_unquant(), and kiss_fftr_twiddles().

6.2.2.60 #define Q15_ONE ((celt_word16_t)1.f)

Definition at line 152 of file arch.h.

Referenced by intra_prediction(), and intra_unquant().

6.2.2.61 #define Q15_ONE_1 ((celt_word16_t)1.f)

Definition at line 153 of file arch.h.

6.2.2.62 #define Q15ONE 1.0f

Definition at line 133 of file arch.h.

Referenced by celt_mode_create(), psydecay_init(), and quant_pitch().

6.2.2.63 #define Q30ONE 1.0f

Definition at line 134 of file arch.h.

6.2.2.64 #define QCONST16(x, bits) (x)

Definition at line 155 of file arch.h.

Referenced by alg_quant(), celt_encode(), celt_mode_create(), and compute_pitch_gain().

6.2.2.65 #define QCONST32(x, bits) (x)

Definition at line 156 of file arch.h.

Referenced by celt_encode(), and quant_pitch().

6.2.2.66 #define ROUND16(a, shift) (a)

Definition at line 177 of file arch.h.

Referenced by alg_quant().

6.2.2.67 #define SATURATE(x, a) (x)

Definition at line 175 of file arch.h.

Referenced by fir_mem16(), and iir_mem16().

6.2.2.68 #define SATURATE16(x, a) (x)

Definition at line 169 of file arch.h.

6.2.2.69 #define SATURATE32(x, a) (x)

Definition at line 170 of file arch.h.

6.2.2.70 #define SHL(a, shift) (a)

Definition at line 174 of file arch.h.

6.2.2.71 #define SHL16(a, shift) (a)

Definition at line 163 of file arch.h.

Referenced by alg_quant(), quant_bands(), and unquant_bands().

6.2.2.72 #define SHL32(a, shift) (a)

Definition at line 165 of file arch.h.

Referenced by _celt_lpc(), celt_encode(), intra_fold(), intra_prediction(), intra_unquant(), kiss_fft_alloc(), kiss_fftr_alloc(), mdct_init(), and stereo_mix().

6.2.2.73 #define SHR(a, shift) (a)

Definition at line 173 of file arch.h.

6.2.2.74 #define SHR16(a, shift) (a)

Definition at line 162 of file arch.h.

Referenced by quant_pitch().

6.2.2.75 #define SHR32(a, shift) (a)

Definition at line 164 of file arch.h.

Referenced by alg_quant(), celt_encode(), compute_pitch_gain(), find_spectral_pitch(), and kiss_fftr_twiddles().

6.2.2.76 #define SIG_SCALING 1.f

Definition at line 136 of file arch.h.

6.2.2.77 #define SIG_SCALING_1 1.f

Definition at line 137 of file arch.h.

6.2.2.78 #define SUB16(a, b) ((a)-(b))

Definition at line 181 of file arch.h.

Referenced by _celt_lpc(), alg_quant(), find_spectral_pitch(), and vq_index().

6.2.2.79 #define SUB32(a, b) ((a)-(b))

Definition at line 183 of file arch.h.

Referenced by _celt_lpc(), celt_encode(), and kiss_fftr_twiddles().

6.2.2.80 #define VERY_LARGE16 1e15f

Definition at line 151 of file arch.h.

Referenced by alg_quant().

6.2.2.81 #define VERY_LARGE32 1e15f

Definition at line 150 of file arch.h.

Referenced by vq_index().

6.2.2.82 #define VERY_SMALL 1e-15f

Definition at line 149 of file arch.h.

6.2.2.83 #define VSHR32(a, shift) (a)

Definition at line 168 of file arch.h.

Referenced by stereo_mix().

6.2.3 Typedef Documentation

6.2.3.1 typedef float celt_ener_t

Definition at line 129 of file arch.h.

6.2.3.2 typedef float celt_mask_t

Definition at line 131 of file arch.h.

6.2.3.3 typedef float celt_norm_t

Definition at line 128 of file arch.h.

6.2.3.4 typedef float celt_pgain_t

Definition at line 130 of file arch.h.

6.2.3.5 typedef float celt_sig_t

Definition at line 127 of file arch.h.

6.2.3.6 typedef float celt_word16_t

Definition at line 124 of file arch.h.

6.2.3.7 typedef float celt_word32_t

Definition at line 125 of file arch.h.

6.3 libcelt/bands.c File Reference

```
#include <math.h>
#include "bands.h"
#include "modes.h"
#include "vq.h"
#include "cwrs.h"
#include "stack_alloc.h"
#include "os_support.h"
#include "mathops.h"
```

Functions

- void compute_band_energies (const CELTMode *m, const celt_sig_t *X, celt_ener_t *bank)
- void normalise_bands (const CELTMode *m, const celt_sig_t *restrict freq, celt_norm_t *restrict X, const celt_ener_t *bank)
- void renormalise_bands (const CELTMode *m, celt_norm_t *restrict X)
- void denormalise_bands (const CELTMode *m, const celt_norm_t *restrict X, celt_sig_t *restrict freq, const celt_ener_t *bank)
- void compute_pitch_gain (const CELTMode *m, const celt_norm_t *X, const celt_norm_t *P, celt_pgain_t *gains)
- void pitch_quant_bands (const CELTMode *m, celt_norm_t *restrict P, const celt_pgain_t *restrict gains)
- void quant_bands (const CELTMode *m, celt_norm_t *restrict X, celt_norm_t *P, celt_mask_t *W, int total_bits, ec_enc *enc)
- void unquant_bands (const CELTMode *m, celt_norm_t *restrict X, celt_norm_t *P, int total_bits, ec_dec *dec)
- void stereo_mix (const CELTMode *m, celt_norm_t *X, const celt_ener_t *bank, int dir)

Variables

- const celt_word16_t sqrtC_1 [2] = {QCONST16(1.f, 14), QCONST16(1.414214f, 14)}

6.3.1 Function Documentation

6.3.1.1 void compute_band_energies (const CELTMode * *m*, const celt_sig_t * *X*, celt_ener_t * *bands*)

Compute the amplitude (sqrt energy) in each of the bands

Parameters:

m Mode data

X Spectrum

bands Square root of the energy for each band (returned)

Definition at line 162 of file bands.c.

Referenced by celt_decode(), celt_encode(), and renormalise_bands().

6.3.1.2 void compute_pitch_gain (const CELTMode * *m*, const celt_norm_t * *X*, const celt_norm_t * *P*, celt_pgain_t * *gains*)

Compute the pitch predictor gain for each pitch band

Parameters:

m Mode data

X Spectrum to predict

P Pitch vector (normalised)

gains Gain computed for each pitch band (returned)

bank Square root of the energy for each band

Definition at line 238 of file bands.c.

Referenced by celt_encode().

6.3.1.3 void denormalise_bands (const CELTMode * *m*, const celt_norm_t *restrict *X*, celt_sig_t *restrict *freq*, const celt_ener_t * *bands*)

Denormalise each band of X to restore full amplitude

Parameters:

m Mode data

X Spectrum (returned de-normalised)

bands Square root of the energy for each band

Definition at line 214 of file bands.c.

Referenced by celt_decode(), and celt_encode().

6.3.1.4 void normalise_bands (const CELTMode * *m*, const celt_sig_t *restrict *freq*, celt_norm_t *restrict *X*, const celt_ener_t * *bands*)

Normalise each band of X such that the energy in each band is equal to 1

Parameters:

m Mode data

X Spectrum (returned normalised)

bands Square root of the energy for each band

Definition at line 183 of file bands.c.

Referenced by celt_decode(), celt_encode(), and renormalise_bands().

6.3.1.5 void pitch_quant_bands (const CELTMode * *m*, celt_norm_t *restrict *P*, const celt_pgain_t *restrict *gains*)

Definition at line 276 of file bands.c.

Referenced by celt_decode(), and celt_encode().

6.3.1.6 void quant_bands (const CELTMode * *m*, celt_norm_t *restrict *X*, celt_norm_t * *P*, celt_mask_t * *W*, int *total_bits*, ec_enc * *enc*)

Quantisation/encoding of the residual spectrum

Parameters:

m Mode data

X Residual (normalised)

P Pitch vector (normalised)

W Perceptual weighting

total_bits Total number of bits that can be used for the frame (including the ones already spent)

enc Entropy encoder

Definition at line 294 of file bands.c.

Referenced by celt_encode().

6.3.1.7 void renormalise_bands (const CELTMode * *m*, celt_norm_t *restrict *X*)

Definition at line 201 of file bands.c.

Referenced by celt_decode(), and celt_encode().

6.3.1.8 void stereo_mix (const CELTMode * *m*, celt_norm_t * *X*, const celt_ener_t * *bank*, int *dir*)

Definition at line 412 of file bands.c.

Referenced by celt_decode(), and celt_encode().

6.3.1.9 void unquant_bands (const CELTMode * *m*, celt_norm_t *restrict *X*, celt_norm_t * *P*, int *total_bits*, ec_dec * *dec*)

Decoding of the residual spectrum

Parameters:

m Mode data

X Residual (normalised)

P Pitch vector (normalised)

total_bits Total number of bits that can be used for the frame (including the ones already spent)

dec Entropy decoder

Definition at line 357 of file bands.c.

Referenced by celt_decode().

6.3.2 Variable Documentation

6.3.2.1 const celt_word16_t sqrtC_1[2] = {QCONST16(1.f, 14), QCONST16(1.414214f, 14)}

Definition at line 81 of file bands.c.

6.4 libcelt/bands.h File Reference

```
#include "arch.h"
#include "modes.h"
#include "entenc.h"
#include "entdec.h"
#include "rate.h"
```

Functions

- void exp_rotation (celt_norm_t *X, int len, int dir, int stride, int iter)
- void compute_band_energies (const CELTMode *m, const celt_sig_t *X, celt_ener_t *bands)
- void normalise_bands (const CELTMode *m, const celt_sig_t *restrict freq, celt_norm_t *restrict X, const celt_ener_t *bands)
- void renormalise_bands (const CELTMode *m, celt_norm_t *restrict X)
- void denormalise_bands (const CELTMode *m, const celt_norm_t *restrict X, celt_sig_t *restrict freq, const celt_ener_t *bands)
- void compute_pitch_gain (const CELTMode *m, const celt_norm_t *X, const celt_norm_t *P, celt_pgain_t *gains)
- void pitch_quant_bands (const CELTMode *m, celt_norm_t *restrict P, const celt_pgain_t *restrict gains)
- void quant_bands (const CELTMode *m, celt_norm_t *restrict X, celt_norm_t *P, celt_mask_t *W, int total_bits, ec_enc *enc)
- void unquant_bands (const CELTMode *m, celt_norm_t *restrict X, celt_norm_t *P, int total_bits, ec_dec *dec)
- void stereo_mix (const CELTMode *m, celt_norm_t *X, const celt_ener_t *bank, int dir)

6.4.1 Function Documentation

6.4.1.1 void compute_band_energies (const CELTMode * *m*, const celt_sig_t * *X*, celt_ener_t * *bands*)

Compute the amplitude (sqrt energy) in each of the bands

Parameters:

m Mode data

X Spectrum

bands Square root of the energy for each band (returned)

Definition at line 162 of file bands.c.

References CHANNELS, CELTMode::eBands, and CELTMode::nbEBands.

6.4.1.2 void compute_pitch_gain (const CELTMode * *m*, const celt_norm_t * *X*, const celt_norm_t * *P*, celt_pgain_t * *gains*)

Compute the pitch predictor gain for each pitch band

Parameters:

m Mode data

X Spectrum to predict

P Pitch vector (normalised)

gains Gain computed for each pitch band (returned)

bank Square root of the energy for each band

Definition at line 238 of file bands.c.

References ADD32, celt_div, CHANNELS, EPSILON, EXTRACT16, MAC16_16, MULT16_32_Q15, CELTMode::nbPBands, CELTMode::pBands, QCONST16, and SHR32.

6.4.1.3 void denormalise_bands (const CELTMode * *m*, const celt_norm_t *restrict *X*, celt_sig_t *restrict *freq*, const celt_ener_t * *bands*)

Denormalise each band of X to restore full amplitude

Parameters:

m Mode data

X Spectrum (returned de-normalised)

bands Square root of the energy for each band

Definition at line 214 of file bands.c.

References celt_fatal, CHANNELS, CELTMode::eBands, MULT16_32_Q13, MULT16_32_Q15, and CELTMode::nbEBands.

6.4.1.4 void exp_rotation (celt_norm_t * *X*, int *len*, int *dir*, int *stride*, int *iter*)

Applies a series of rotations so that pulses are spread like a two-sided exponential. The effect of this is to reduce the tonal noise created by the sparse spectrum resulting from the pulse codebook

6.4.1.5 void normalise_bands (const CELTMode * *m*, const celt_sig_t *restrict *freq*, celt_norm_t *restrict *X*, const celt_ener_t * *bands*)

Normalise each band of X such that the energy in each band is equal to 1

Parameters:

m Mode data

X Spectrum (returned normalised)

bands Square root of the energy for each band

Definition at line 183 of file bands.c.

References CHANNELS, CELTMode::eBands, and CELTMode::nbEBands.

6.4.1.6 void pitch_quant_bands (const CELTMode * *m*, celt_norm_t *restrict *P*, const celt_pgain_t *restrict *gains*)

Definition at line 276 of file bands.c.

References CHANNELS, MULT16_16_Q15, CELTMode::nbPBands, and CELTMode::pBands.

6.4.1.7 void quant_bands (const CELTMode * *m*, celt_norm_t *restrict *X*, celt_norm_t * *P*, celt_mask_t * *W*, int *total_bits*, ec_enc * *enc*)

Quantisation/encoding of the residual spectrum

Parameters:

m Mode data

X Residual (normalised)

P Pitch vector (normalised)

W Perceptual weighting

total_bits Total number of bits that can be used for the frame (including the ones already spent)

enc Entropy encoder

Definition at line 294 of file bands.c.

References alg_quant(), ALLOC, celt_sqrt, CHANNELS, compute_allocation(), CELTMode::eBands, ec_enc_tell(), intra_fold(), intra_prediction(), MULT16_16_Q15, CELTMode::nbEBands, CELTMode::pitchEnd, RESTORE_STACK, SAVE_STACK, SHL16, and VARDECL.

6.4.1.8 void renormalise_bands (const CELTMode * *m*, celt_norm_t *restrict *X*)

Definition at line 201 of file bands.c.

References ALLOC, compute_band_energies(), CELTMode::nbChannels, CELTMode::nbEBands, normalise_bands(), RESTORE_STACK, SAVE_STACK, and VARDECL.

6.4.1.9 void stereo_mix (const CELTMode * *m*, celt_norm_t * *X*, const celt_ener_t * *bank*, int *dir*)

Definition at line 412 of file bands.c.

References celt_sqrt, CHANNELS, DIV32_16, CELTMode::eBands, EPSILON, EXTEND32, MAX32, MULT16_16, MULT16_16_Q14, CELTMode::nbEBands, r, SHL32, and VSHR32.

6.4.1.10 void unquant_bands (const CELTMode * *m*, celt_norm_t *restrict *X*, celt_norm_t * *P*, int *total_bits*, ec_dec * *dec*)

Decoding of the residual spectrum

Parameters:

m Mode data

X Residual (normalised)

P Pitch vector (normalised)

total_bits Total number of bits that can be used for the frame (including the ones already spent)

dec Entropy decoder

Definition at line 357 of file bands.c.

References alg_unquant(), ALLOC, celt_sqrt, CHANNELS, compute_allocation(), CELTMode::eBands, ec_dec_tell(), intra_fold(), intra_unquant(), MULT16_16_Q15, CELTMode::nbEBands, CELTMode::pitchEnd, RESTORE_STACK, SAVE_STACK, SHL16, and VARDECL.

6.5 libcelt/celt.c File Reference

```
#include "os_support.h"
#include "mdct.h"
#include <math.h>
#include "celt.h"
#include "pitch.h"
#include "kiss_fftr.h"
#include "bands.h"
#include "modes.h"
#include "entcode.h"
#include "quant_pitch.h"
#include "quant_bands.h"
#include "psy.h"
#include "rate.h"
#include "stack_alloc.h"
```

Data Structures

- struct CELTEncoder

  Encoder state.

- struct CELTDecoder

  Decoder state.

Defines

- #define CELT_C

Functions

- CELTEncoder EXPORT * celt_encoder_create (const CELTMode *mode)
- void EXPORT celt_encoder_destroy (CELTEncoder *st)
- int EXPORT celt_encode (CELTEncoder *restrict st, celt_int16_t *restrict pcm, unsigned char *compressed, int nbCompressedBytes)
- CELTDecoder EXPORT * celt_decoder_create (const CELTMode *mode)
- void EXPORT celt_decoder_destroy (CELTDecoder *st)
- int EXPORT celt_decode (CELTDecoder *restrict st, unsigned char *data, int len, celt_int16_t *restrict pcm)

6.5.1 Define Documentation

6.5.1.1 #define CELT_C

Definition at line 36 of file celt.c.

6.5.2 Function Documentation

6.5.2.1 int EXPORT celt_decode (CELTDecoder *restrict *st*, unsigned char * *data*, int *len*, celt_int16_t *restrict *pcm*)

< Interleaved signal MDCTs

< Interleaved normalised MDCTs

< Interleaved normalised pitch MDCTs

Definition at line 536 of file celt.c.

References ADD32, ALLOC, CELT_CORRUPTED_DATA, CELT_INVALID_MODE, CELT_MOVE, CELT_OK, CHANNELS, check_mode(), compute_band_energies(), denormalise_bands(), ec_byte_readinit(), ec_dec_init(), ec_dec_tell(), ec_dec_uint(), MAX_PERIOD, MULT16_32_Q15, normalise_bands(), pitch_quant_bands(), renormalise_bands(), RESTORE_STACK, SAVE_STACK, stereo_mix(), unquant_bands(), unquant_energy(), unquant_pitch(), and VARDECL.

Referenced by main().

6.5.2.2 int EXPORT celt_encode (CELTEncoder *restrict *st*, celt_int16_t *restrict *pcm*, unsigned char * *compressed*, int *nbCompressedBytes*)

< Interleaved signal MDCTs

< Interleaved normalised MDCTs

< Interleaved normalised pitch MDCTs

Definition at line 214 of file celt.c.

References ADD32, ALLOC, B, CELT_COPY, CELT_INTERNAL_ERROR, CELT_INVALID_MODE, CELT_MOVE, CELT_OK, CHANNELS, check_mode(), compute_band_energies(), compute_mdct_masking(), compute_pitch_gain(), denormalise_bands(), ec_enc_bits(), ec_enc_done(), ec_enc_init(), ec_enc_tell(), ec_enc_uint(), EXTEND32, find_spectral_pitch(), MAX_PERIOD, MULT16_16, MULT16_32_Q15, normalise_bands(), pitch_quant_bands(), QCONST16, QCONST32, quant_bands(), quant_energy(), quant_pitch(), renormalise_bands(), RESTORE_STACK, SAVE_STACK, SHL32, SHR32, stereo_mix(), SUB32, and VARDECL.

Referenced by main().

6.6 libcelt/celt.h File Reference

Contains all the functions for encoding and decoding audio streams.

```
#include "celt_types.h"
```

Defines

- #define CELT_OK 0
- #define CELT_BAD_ARG -1
- #define CELT_INVALID_MODE -2
- #define CELT_INTERNAL_ERROR -3
- #define CELT_CORRUPTED_DATA -4
- #define CELT_GET_FRAME_SIZE 1000
- #define CELT_GET_LOOKAHEAD 1001
- #define CELT_GET_NB_CHANNELS 1002
- #define CELT_GET_BITSTREAM_VERSION 2000

Typedefs

- typedef struct CELTEncoder CELTEncoder

  Encoder state.

- typedef struct CELTDecoder CELTDecoder
- typedef struct CELTMode CELTMode

Functions

- CELTMode * celt_mode_create (celt_int32_t Fs, int channels, int frame_size, int lookahead, int *error)
- void celt_mode_destroy (CELTMode *mode)
- int celt_mode_info (const CELTMode *mode, int request, celt_int32_t *value)
- CELTEncoder * celt_encoder_create (const CELTMode *mode)
- void celt_encoder_destroy (CELTEncoder *st)
- int celt_encode (CELTEncoder *st, celt_int16_t *pcm, unsigned char *compressed, int nbCompressedBytes)
- CELTDecoder * celt_decoder_create (const CELTMode *mode)
- void celt_decoder_destroy (CELTDecoder *st)
- int celt_decode (CELTDecoder *st, unsigned char *data, int len, celt_int16_t *pcm)

6.6.1 Detailed Description

Contains all the functions for encoding and decoding audio streams.

Definition in file celt.h.

6.6.2 Define Documentation

6.6.2.1 #define CELT_BAD_ARG -1

An (or more) invalid argument (e.g. out of range)

Definition at line 50 of file celt.h.

Referenced by celt_header_to_packet(), celt_mode_create(), and celt_mode_info().

6.6.2.2 #define CELT_CORRUPTED_DATA -4

The data passed (e.g. compressed data to decoder) is corrupted

Definition at line 56 of file celt.h.

Referenced by celt_decode().

6.6.2.3 #define CELT_GET_BITSTREAM_VERSION 2000

GET the bit-stream version for compatibility check

Definition at line 67 of file celt.h.

Referenced by celt_header_init(), and celt_mode_info().

6.6.2.4 #define CELT_GET_FRAME_SIZE 1000

GET the frame size used in the current mode

Definition at line 60 of file celt.h.

Referenced by celt_mode_info(), and main().

6.6.2.5 #define CELT_GET_LOOKAHEAD 1001

GET the lookahead used in the current mode

Definition at line 62 of file celt.h.

Referenced by celt_mode_info().

6.6.2.6 #define CELT_GET_NB_CHANNELS 1002

GET the number of channels used in the current mode

Definition at line 64 of file celt.h.

Referenced by celt_mode_info(), and main().

6.6.2.7 #define CELT_INTERNAL_ERROR -3

An internal error was detected

Definition at line 54 of file celt.h.

Referenced by celt_encode().

6.6.2.8 #define CELT_INVALID_MODE -2

The mode struct passed is invalid

Definition at line 52 of file celt.h.

Referenced by celt_decode(), celt_encode(), and check_mode().

6.6.2.9 #define CELT_OK 0

No error

Definition at line 48 of file celt.h.

Referenced by celt_decode(), celt_decoder_create(), celt_decoder_destroy(), celt_encode(), celt_encoder_create(), celt_encoder_destroy(), celt_mode_create(), celt_mode_destroy(), celt_mode_info(), and check_mode().

6.6.3 Typedef Documentation

6.6.3.1 typedef struct CELTDecoder CELTDecoder

State of the decoder. One decoder state is needed for each stream. It is initialised once at the beginning of the stream. Do ∗not∗ re-initialise the state for every frame

Definition at line 80 of file celt.h.

6.6.3.2 typedef struct CELTEncoder CELTEncoder

Encoder state.

Contains the state of an encoder. One encoder state is needed for each stream. It is initialised once at the beginning of the stream. Do ∗not∗ re-initialise the state for every frame.

Definition at line 75 of file celt.h.

6.6.3.3 typedef struct CELTMode CELTMode

The mode contains all the information necessary to create an encoder. Both the encoder and decoder need to be initialised with exactly the same mode, otherwise the quality will be very bad

Definition at line 85 of file celt.h.

6.7 libcelt/celt_header.h File Reference

```
#include "celt.h"
#include "celt_types.h"
```

Data Structures

- struct CELTHeader

  Header data.

Functions

- void celt_header_init (CELTHeader *header, const CELTMode *m)
- int celt_header_to_packet (const CELTHeader *header, unsigned char *packet, celt_uint32_t size)
- int celt_header_from_packet (const unsigned char *packet, celt_uint32_t size, CELTHeader *header)

6.7.1 Function Documentation

6.7.1.1 int celt_header_from_packet (const unsigned char * *packet*, celt_uint32_t *size*, CELTHeader * *header*)

Definition at line 106 of file header.c.

6.7.1.2 void celt_header_init (CELTHeader * *header*, const CELTMode * *m*)

Creates a basic header struct

Definition at line 65 of file header.c.

6.7.1.3 int celt_header_to_packet (const CELTHeader * *header*, unsigned char * *packet*, celt_uint32_t *size*)

Definition at line 80 of file header.c.

6.8 libcelt/celt_types.h File Reference

CELT types.

Typedefs

- typedef short celt_int16_t
- typedef unsigned short celt_uint16_t
- typedef int celt_int32_t
- typedef unsigned int celt_uint32_t
- typedef long long celt_int64_t
- typedef unsigned long long celt_uint64_t

6.8.1 Detailed Description

CELT types.

Definition in file celt_types.h.

6.8.2 Typedef Documentation

6.8.2.1 typedef short celt_int16_t

Definition at line 159 of file celt_types.h.

6.8.2.2 typedef int celt_int32_t

Definition at line 161 of file celt_types.h.

6.8.2.3 typedef long long celt_int64_t

Definition at line 163 of file celt_types.h.

6.8.2.4 typedef unsigned short celt_uint16_t

Definition at line 160 of file celt_types.h.

6.8.2.5 typedef unsigned int celt_uint32_t

Definition at line 162 of file celt_types.h.

6.8.2.6 typedef unsigned long long celt_uint64_t

Definition at line 164 of file celt_types.h.

6.9 libcelt/compute_entropy_matrix.m File Reference

Variables

- function M
- for b

6.9.1 Variable Documentation

6.9.1.1 for b

Initial value:

```
0:R
     beta = b/R
```

Definition at line 5 of file compute_entropy_matrix.m.

Referenced by ec_dec_tell(), and ec_enc_tell().

6.9.1.2 function M

Initial value:

```
 compute_entropy_matrix(E, R, res)

for a = 0:R
   alpha = a/R
```

Definition at line 1 of file compute_entropy_matrix.m.

6.10 libcelt/cwrs.c File Reference

```
#include "os_support.h"
#include <stdlib.h>
#include <string.h>
#include "cwrs.h"
#include "mathops.h"
```

Functions

- celt_uint32_t ncwrs_unext32 (int _n, celt_uint32_t *_ui)
- celt_uint64_t ncwrs_unext64 (int _n, celt_uint64_t *_ui)
- celt_uint32_t ncwrs_u32 (int _n, int _m, celt_uint32_t *_u)
- celt_uint64_t ncwrs_u64 (int _n, int _m, celt_uint64_t *_u)
- void cwrsi32 (int _n, int _m, celt_uint32_t _i, int *_x, int *_s, celt_uint32_t *_u)
- void cwrsi64 (int _n, int _m, celt_uint64_t _i, int *_x, int *_s, celt_uint64_t *_u)
- celt_uint32_t icwrs32 (int _n, int _m, const int *_x, const int *_s, celt_uint32_t *_u)
- celt_uint64_t icwrs64 (int _n, int _m, const int *_x, const int *_s, celt_uint64_t *_u)
- void comb2pulse (int _n, int _m, int *restrict _y, const int *_x, const int *_s)
- void pulse2comb (int _n, int _m, int *_x, int *_s, const int *_y)
- void encode_pulses (int *_y, int N, int K, ec_enc *enc)
- void decode_pulses (int *_y, int N, int K, ec_dec *dec)

6.10.1 Function Documentation

6.10.1.1 void comb2pulse (int *_n*, int *_m*, int *restrict _y*, const int * *_x*, const int * *_s*)

Definition at line 271 of file cwrs.c.

Referenced by decode_pulses().

6.10.1.2 void cwrsi32 (int *_n*, int *_m*, celt_uint32_t *_i*, int * *_x*, int * *_s*, celt_uint32_t * *_u*)

Definition at line 170 of file cwrs.c.

6.10.1.3 void cwrsi64 (int *_n*, int *_m*, celt_uint64_t *_i*, int * *_x*, int * *_s*, celt_uint64_t * *_u*)

Definition at line 194 of file cwrs.c.

6.10.1.4 void decode_pulses (int * *_y*, int *N*, int *K*, ec_dec * *dec*)

Definition at line 365 of file cwrs.c.

Referenced by alg_unquant().

6.10.1.5 void encode_pulses (int * _y, int N, int K, ec_enc * enc)

Definition at line 329 of file cwrs.c.

Referenced by alg_quant().

6.10.1.6 celt_uint32_t icwrs32 (int _n, int _m, const int * _x, const int * _s, celt_uint32_t * _u)

Definition at line 224 of file cwrs.c.

6.10.1.7 celt_uint64_t icwrs64 (int _n, int _m, const int * _x, const int * _s, celt_uint64_t * _u)

Definition at line 245 of file cwrs.c.

6.10.1.8 celt_uint32_t ncwrs_u32 (int _n, int _m, celt_uint32_t * _u)

Definition at line 146 of file cwrs.c.

6.10.1.9 celt_uint64_t ncwrs_u64 (int _n, int _m, celt_uint64_t * _u)

Definition at line 155 of file cwrs.c.

Referenced by compute_alloc_cache().

6.10.1.10 celt_uint32_t ncwrs_unext32 (int _n, celt_uint32_t * _ui)

Definition at line 109 of file cwrs.c.

References celt_assert.

Referenced by ncwrs_u32().

6.10.1.11 celt_uint64_t ncwrs_unext64 (int _n, celt_uint64_t * _ui)

Definition at line 126 of file cwrs.c.

Referenced by compute_alloc_cache(), and ncwrs_u64().

6.10.1.12 void pulse2comb (int _n, int _m, int * _x, int * _s, const int * _y)

Definition at line 285 of file cwrs.c.

Referenced by encode_pulses().

6.11 libcelt/cwrs.h File Reference

```
#include "arch.h"
#include "stack_alloc.h"
#include "entenc.h"
#include "entdec.h"
```

Functions

- celt_uint32_t ncwrs_u32 (int _n, int _m, celt_uint32_t *_u)
- void cwrsi32 (int _n, int _m, celt_uint32_t _i, int *_x, int *_s, celt_uint32_t *_u)
- celt_uint32_t icwrs32 (int _n, int _m, const int *_x, const int *_s, celt_uint32_t *_u)
- celt_uint64_t ncwrs_u64 (int _n, int _m, celt_uint64_t *_u)
- celt_uint64_t ncwrs_unext64 (int _n, celt_uint64_t *_u)
- void cwrsi64 (int _n, int _m, celt_uint64_t _i, int *_x, int *_s, celt_uint64_t *_u)
- celt_uint64_t icwrs64 (int _n, int _m, const int *_x, const int *_s, celt_uint64_t *_u)
- void comb2pulse (int _n, int _m, int *restrict _y, const int *_x, const int *_s)
- void pulse2comb (int _n, int _m, int *_x, int *_s, const int *_y)
- void encode_pulses (int *_y, int N, int K, ec_enc *enc)
- void decode_pulses (int *_y, int N, int K, ec_dec *dec)

6.11.1 Function Documentation

6.11.1.1 void comb2pulse (int *_n*, int *_m*, int *restrict _y*, const int * *_x*, const int * *_s*)

Definition at line 271 of file cwrs.c.

References CELT_MEMSET, and k.

6.11.1.2 void cwrsi32 (int *_n*, int *_m*, celt_uint32_t *_i*, int * *_x*, int * *_s*, celt_uint32_t * *_u*)

Definition at line 170 of file cwrs.c.

References k, and p.

6.11.1.3 void cwrsi64 (int *_n*, int *_m*, celt_uint64_t *_i*, int * *_x*, int * *_s*, celt_uint64_t * *_u*)

Definition at line 194 of file cwrs.c.

References k, and p.

6.11.1.4 void decode_pulses (int * *_y*, int *N*, int *K*, ec_dec * *dec*)

Definition at line 365 of file cwrs.c.

References ALLOC, celt_ilog2, comb2pulse(), RESTORE_STACK, SAVE_STACK, and VARDECL.

6.11.1.5 void encode_pulses (int * _y, int N, int K, ec_enc * enc)

Definition at line 329 of file cwrs.c.

References ALLOC, celt_ilog2, pulse2comb(), RESTORE_STACK, SAVE_STACK, and VARDECL.

6.11.1.6 celt_uint32_t icwrs32 (int _n, int _m, const int * _x, const int * _s, celt_uint32_t * _u)

Definition at line 224 of file cwrs.c.

References k, and p.

6.11.1.7 celt_uint64_t icwrs64 (int _n, int _m, const int * _x, const int * _s, celt_uint64_t * _u)

Definition at line 245 of file cwrs.c.

References k, and p.

6.11.1.8 celt_uint32_t ncwrs_u32 (int _n, int _m, celt_uint32_t * _u)

Definition at line 146 of file cwrs.c.

References CELT_MEMSET, k, and ncwrs_unext32().

6.11.1.9 celt_uint64_t ncwrs_u64 (int _n, int _m, celt_uint64_t * _u)

Definition at line 155 of file cwrs.c.

References CELT_MEMSET, k, and ncwrs_unext64().

6.11.1.10 celt_uint64_t ncwrs_unext64 (int _n, celt_uint64_t * _u)

Definition at line 126 of file cwrs.c.

References celt_assert.

6.11.1.11 void pulse2comb (int _n, int _m, int * _x, int * _s, const int * _y)

Definition at line 285 of file cwrs.c.

References k.

6.12 libcelt/dump_hex.m File Reference

Functions

- function dump_hex (data, file) F
- fprintf (F,"\n")
- end fclose (F)

Variables

- for k

6.12.1 Function Documentation

6.12.1.1 function dump_hex (data, file)

6.12.1.2 end fclose (F)

Referenced by main().

6.12.1.3 fprintf (F, "\n")

Referenced by dump_header(), dump_modes(), and main().

6.12.2 Variable Documentation

6.12.2.1 for k

Initial value:

```
1:size(data,1)
   fprintf(F, "0x%04x, ", data(k,:))
```

Definition at line 3 of file dump_hex.m.

Referenced by comb2pulse(), cwrsi32(), cwrsi64(), dump_modes(), icwrs32(), icwrs64(), kiss_fftr_twiddles(), kiss_fftri(), ncwrs_u32(), ncwrs_u64(), and pulse2comb().

6.13 libcelt/dump_modes.c File Reference

```
#include <stdio.h>
#include "modes.h"
#include "celt.h"
#include "rate.h"
```

Defines

- #define INT16 "%d"
- #define INT32 "%d"
- #define FLOAT "%f"
- #define WORD16 FLOAT
- #define WORD32 FLOAT

Functions

- void dump_modes (FILE *file, CELTMode **modes, int nb_modes)
- void dump_header (FILE *file, CELTMode **modes, int nb_modes)
- int main (int argc, char **argv)

6.13.1 Define Documentation

6.13.1.1 #define FLOAT "%f"

Definition at line 43 of file dump_modes.c.

6.13.1.2 #define INT16 "%d"

Definition at line 41 of file dump_modes.c.

6.13.1.3 #define INT32 "%d"

Definition at line 42 of file dump_modes.c.

Referenced by dump_modes().

6.13.1.4 #define WORD16 FLOAT

Definition at line 49 of file dump_modes.c.

Referenced by dump_modes().

6.13.1.5 #define WORD32 FLOAT

Definition at line 50 of file dump_modes.c.

6.13.2 Function Documentation

6.13.2.1 void dump_header (FILE * *file*, CELTMode ** *modes*, int *nb_modes*)

Definition at line 180 of file dump_modes.c.

References fprintf(), CELTMode::mdctSize, CELTMode::nbChannels, and CELTMode::overlap.

Referenced by main().

6.13.2.2 void dump_modes (FILE * *file*, CELTMode ** *modes*, int *nb_modes*)

Definition at line 54 of file dump_modes.c.

References CELTMode::allocVectors, CELTMode::bits, PsyDecay::decayR, CELTMode::eBands, CELTMode::ePredCoef, fprintf(), CELTMode::Fs, INT32, k, MAX_PERIOD, MAX_PULSES, CELTMode::mdctSize, CELTMode::nbAllocVectors, CELTMode::nbChannels, CELTMode::nbEBands, CELTMode::nbPBands, CELTMode::overlap, CELTMode::pBands, CELTMode::pitchEnd, CELTMode::psy, CELTMode::window, and WORD16.

Referenced by main().

6.13.2.3 int main (int *argc*, char ** *argv*)

Definition at line 219 of file dump_modes.c.

References celt_mode_create(), dump_header(), dump_modes(), fclose(), and fprintf().

6.14 libcelt/ecintrin.h File Reference

```
#include <math.h>
#include <limits.h>
```

Defines

- #define _ecintrin_H (1)
- #define EC_MAXI(_a, _b) ((_a)-((_a)-(_b)&-((_b)>(_a))))
- #define EC_MINI(_a, _b) ((_a)+((_b)-(_a)&-((_b)<(_a))))
- #define EC_SIGNI(_a) (((_a)>0)-((_a)<0))
- #define EC_SIGNMASK(_a) (-((_a)<0))
- #define EC_CLAMPI(_a, _b, _c) (EC_MAXI(_a,EC_MINI(_b,_c)))
- #define EC_ILOG(_x) (ec_ilog(_x))
- #define EC_ILOG64(_x) (ec_ilog64(_x))

6.14.1 Define Documentation

6.14.1.1 #define _ecintrin_H (1)

Definition at line 5 of file ecintrin.h.

6.14.1.2 #define EC_CLAMPI(_a, _b, _c) (EC_MAXI(_a,EC_MINI(_b,_c)))

Definition at line 49 of file ecintrin.h.

6.14.1.3 #define EC_ILOG(_x) (ec_ilog(_x))

Definition at line 77 of file ecintrin.h.

Referenced by ec_dec_tell(), ec_dec_uint(), ec_enc_tell(), and ec_enc_uint().

6.14.1.4 #define EC_ILOG64(_x) (ec_ilog64(_x))

Definition at line 102 of file ecintrin.h.

Referenced by ec_dec_uint64(), and ec_enc_uint64().

6.14.1.5 #define EC_MAXI(_a, _b) ((_a)-((_a)-(_b)&-((_b)>(_a))))

Definition at line 32 of file ecintrin.h.

6.14.1.6 #define EC_MINI(_a, _b) ((_a)+((_b)-(_a)&-((_b)<(_a))))

Definition at line 33 of file ecintrin.h.

Referenced by ec_decode(), and ec_decode_bin().

6.14.1.7 #define EC_SIGNI(_a) (((_a)>0)-((_a)<0))

Definition at line 38 of file ecintrin.h.

6.14.1.8 #define EC_SIGNMASK(_a) (-((_a)<0))

Definition at line 42 of file ecintrin.h.

6.15 libcelt/entcode.c File Reference

```
#include "entcode.h"
```

Functions

- int ec_ilog (ec_uint32 _v)
- int ec_ilog64 (ec_uint64 _v)

6.15.1 Function Documentation

6.15.1.1 int ec_ilog (ec_uint32 _v)

Definition at line 13 of file entcode.c.

6.15.1.2 int ec_ilog64 (ec_uint64 _v)

Definition at line 40 of file entcode.c.

6.16 libcelt/entcode.h File Reference

```
#include "celt_types.h"
#include <limits.h>
#include "ecintrin.h"
```

Data Structures

- struct ec_byte_buffer

Defines

- #define _entcode_H (1)
- #define EC_UNIT_BITS (8)
- #define EC_UNIT_MASK ((1U<<EC_UNIT_BITS)-1)

Typedefs

- typedef celt_int32_t ec_int32
- typedef celt_uint32_t ec_uint32
- typedef celt_uint64_t ec_uint64
- typedef struct ec_byte_buffer ec_byte_buffer

Functions

- void ec_byte_writeinit (ec_byte_buffer *_b)
- void ec_byte_writetrunc (ec_byte_buffer *_b, long _bytes)
- void ec_byte_write1 (ec_byte_buffer *_b, unsigned _value)
- void ec_byte_write4 (ec_byte_buffer *_b, ec_uint32 _value)
- void ec_byte_writecopy (ec_byte_buffer *_b, void *_source, long _bytes)
- void ec_byte_writeclear (ec_byte_buffer *_b)
- void ec_byte_readinit (ec_byte_buffer *_b, unsigned char *_buf, long _bytes)
- int ec_byte_look1 (ec_byte_buffer *_b)
- int ec_byte_look4 (ec_byte_buffer *_b, ec_uint32 *_val)
- void ec_byte_adv1 (ec_byte_buffer *_b)
- void ec_byte_adv4 (ec_byte_buffer *_b)
- int ec_byte_read1 (ec_byte_buffer *_b)
- int ec_byte_read4 (ec_byte_buffer *_b, ec_uint32 *_val)
- int ec_ilog (ec_uint32 _v)
- int ec_ilog64 (ec_uint64 _v)

6.16.1 Define Documentation

6.16.1.1 #define _entcode_H (1)

Definition at line 4 of file entcode.h.

6.16.1.2 #define EC_UNIT_BITS (8)

Definition at line 18 of file entcode.h.

Referenced by ec_dec_bits(), ec_dec_uint(), ec_dec_uint64(), ec_enc_bits(), ec_enc_uint(), and ec_enc_-uint64().

6.16.1.3 #define EC_UNIT_MASK ((1U<<EC_UNIT_BITS)-1)

Definition at line 20 of file entcode.h.

Referenced by ec_dec_bits(), and ec_enc_bits().

6.16.2 Typedef Documentation

6.16.2.1 typedef struct ec_byte_buffer ec_byte_buffer

Definition at line 13 of file entcode.h.

6.16.2.2 typedef celt_int32_t ec_int32

Definition at line 10 of file entcode.h.

6.16.2.3 typedef celt_uint32_t ec_uint32

Definition at line 11 of file entcode.h.

6.16.2.4 typedef celt_uint64_t ec_uint64

Definition at line 12 of file entcode.h.

6.16.3 Function Documentation

6.16.3.1 void ec_byte_adv1 (ec_byte_buffer * *_b*)

Definition at line 46 of file entdec.c.

6.16.3.2 void ec_byte_adv4 (ec_byte_buffer * *_b*)

Definition at line 50 of file entdec.c.

6.16.3.3 int ec_byte_look1 (ec_byte_buffer * *_b*)

Definition at line 15 of file entdec.c.

6.16.3.4 int ec_byte_look4 (ec_byte_buffer * *_b*, ec_uint32 * *_val*)

Definition at line 22 of file entdec.c.

6.16.3.5 int ec_byte_read1 (ec_byte_buffer * _b)

Definition at line 54 of file entdec.c.

6.16.3.6 int ec_byte_read4 (ec_byte_buffer * _b, ec_uint32 * _val)

Definition at line 61 of file entdec.c.

6.16.3.7 void ec_byte_readinit (ec_byte_buffer * _b, unsigned char * _buf, long _bytes)

Definition at line 10 of file entdec.c.
Referenced by celt_decode().

6.16.3.8 void ec_byte_write1 (ec_byte_buffer * _b, unsigned _value)

Definition at line 21 of file entenc.c.

6.16.3.9 void ec_byte_write4 (ec_byte_buffer * _b, ec_uint32 _value)

Definition at line 32 of file entenc.c.

6.16.3.10 void ec_byte_writeclear (ec_byte_buffer * _b)

Definition at line 61 of file entenc.c.
Referenced by celt_encoder_destroy().

6.16.3.11 void ec_byte_writecopy (ec_byte_buffer * _b, void * _source, long _bytes)

Definition at line 49 of file entenc.c.

6.16.3.12 void ec_byte_writeinit (ec_byte_buffer * _b)

Definition at line 12 of file entenc.c.
Referenced by celt_encoder_create().

6.16.3.13 void ec_byte_writetrunc (ec_byte_buffer * _b, long _bytes)

Definition at line 17 of file entenc.c.

6.16.3.14 int ec_ilog (ec_uint32 _v)

Definition at line 13 of file entcode.c.

6.16.3.15 int ec_ilog64 (ec_uint64 _v)

Definition at line 40 of file entcode.c.

References v.

6.17 libcelt/entdec.c File Reference

```
#include <stddef.h>
#include "entdec.h"
#include "os_support.h"
```

Functions

- void ec_byte_readinit (ec_byte_buffer *_b, unsigned char *_buf, long _bytes)
- int ec_byte_look1 (ec_byte_buffer *_b)
- int ec_byte_look4 (ec_byte_buffer *_b, ec_uint32 *_val)
- void ec_byte_adv1 (ec_byte_buffer *_b)
- void ec_byte_adv4 (ec_byte_buffer *_b)
- int ec_byte_read1 (ec_byte_buffer *_b)
- int ec_byte_read4 (ec_byte_buffer *_b, ec_uint32 *_val)
- ec_uint32 ec_dec_bits (ec_dec *_this, int _ftb)
- ec_uint64 ec_dec_bits64 (ec_dec *_this, int _ftb)
- ec_uint32 ec_dec_uint (ec_dec *_this, ec_uint32 _ft)
- ec_uint64 ec_dec_uint64 (ec_dec *_this, ec_uint64 _ft)

6.17.1 Function Documentation

6.17.1.1 void ec_byte_adv1 (ec_byte_buffer * _b)

Definition at line 46 of file entdec.c.

References ec_byte_buffer::ptr.

6.17.1.2 void ec_byte_adv4 (ec_byte_buffer * _b)

Definition at line 50 of file entdec.c.

References ec_byte_buffer::ptr.

6.17.1.3 int ec_byte_look1 (ec_byte_buffer * _b)

Definition at line 15 of file entdec.c.

References ec_byte_buffer::buf, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.17.1.4 int ec_byte_look4 (ec_byte_buffer * _b, ec_uint32 * _val)

Definition at line 22 of file entdec.c.

References ec_byte_buffer::buf, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.17.1.5 int ec_byte_read1 (ec_byte_buffer * _b)

Definition at line 54 of file entdec.c.

References ec_byte_buffer::buf, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.17.1.6 int ec_byte_read4 (ec_byte_buffer * _b, ec_uint32 * _val)

Definition at line 61 of file entdec.c.

References ec_byte_buffer::buf, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.17.1.7 void ec_byte_readinit (ec_byte_buffer * _b, unsigned char * _buf, long _bytes)

Definition at line 10 of file entdec.c.

References ec_byte_buffer::buf, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.17.1.8 ec_uint32 ec_dec_bits (ec_dec * _this, int _ftb)

Definition at line 85 of file entdec.c.

Referenced by ec_dec_bits64(), ec_dec_uint(), intra_unquant(), and unquant_pitch().

6.17.1.9 ec_uint64 ec_dec_bits64 (ec_dec * _this, int _ftb)

Definition at line 103 of file entdec.c.

Referenced by ec_dec_uint64().

6.17.1.10 ec_uint32 ec_dec_uint (ec_dec * _this, ec_uint32 _ft)

Definition at line 113 of file entdec.c.

Referenced by celt_decode().

6.17.1.11 ec_uint64 ec_dec_uint64 (ec_dec * _this, ec_uint64 _ft)

Definition at line 143 of file entdec.c.

6.18 libcelt/entdec.h File Reference

```
#include "entcode.h"
```

Data Structures

- struct ec_dec

Defines

- #define _entdec_H (1)

Typedefs

- typedef struct ec_dec ec_dec

Functions

- void ec_dec_init (ec_dec *_this, ec_byte_buffer *_buf)
- unsigned ec_decode (ec_dec *_this, unsigned _ft)
- unsigned ec_decode_bin (ec_dec *_this, unsigned bits)
- void ec_dec_update (ec_dec *_this, unsigned _fl, unsigned _fh, unsigned _ft)
- ec_uint32 ec_dec_bits (ec_dec *_this, int _ftb)
- ec_uint64 ec_dec_bits64 (ec_dec *_this, int _ftb)
- ec_uint32 ec_dec_uint (ec_dec *_this, ec_uint32 _ft)
- ec_uint64 ec_dec_uint64 (ec_dec *_this, ec_uint64 _ft)
- long ec_dec_tell (ec_dec *_this, int _b)

6.18.1 Define Documentation

6.18.1.1 #define _entdec_H (1)

Definition at line 2 of file entdec.h.

6.18.2 Typedef Documentation

6.18.2.1 typedef struct ec_dec ec_dec

Definition at line 7 of file entdec.h.

6.18.3 Function Documentation

6.18.3.1 ec_uint32 ec_dec_bits (ec_dec * *_this*, int *_ftb*)

Definition at line 85 of file entdec.c.

References ec_dec_update(), ec_decode_bin(), EC_UNIT_BITS, and EC_UNIT_MASK.

6.18.3.2 ec_uint64 ec_dec_bits64 (ec_dec * *_this*, int *_ftb*)

Definition at line 103 of file entdec.c.

References ec_dec_bits().

6.18.3.3 void ec_dec_init (ec_dec * *_this*, ec_byte_buffer * *_buf*)

Definition at line 155 of file rangedec.c.

Referenced by celt_decode().

6.18.3.4 long ec_dec_tell (ec_dec * *_this*, int *_b*)

Definition at line 189 of file rangedec.c.

Referenced by celt_decode(), and unquant_bands().

6.18.3.5 ec_uint32 ec_dec_uint (ec_dec * *_this*, ec_uint32 *_ft*)

Definition at line 113 of file entdec.c.

References ec_dec_bits(), ec_dec_update(), ec_decode(), EC_ILOG, and EC_UNIT_BITS.

6.18.3.6 ec_uint64 ec_dec_uint64 (ec_dec * *_this*, ec_uint64 *_ft*)

Definition at line 143 of file entdec.c.

References ec_dec_bits64(), ec_dec_update(), ec_decode(), EC_ILOG64, and EC_UNIT_BITS.

6.18.3.7 void ec_dec_update (ec_dec * *_this*, unsigned *_fl*, unsigned *_fh*, unsigned *_ft*)

Definition at line 181 of file rangedec.c.

Referenced by ec_dec_bits(), ec_dec_uint(), ec_dec_uint64(), and ec_laplace_decode_start().

6.18.3.8 unsigned ec_decode (ec_dec * *_this*, unsigned *_ft*)

Definition at line 165 of file rangedec.c.

Referenced by ec_dec_uint(), ec_dec_uint64(), and ec_laplace_decode_start().

6.18.3.9 unsigned ec_decode_bin (ec_dec * *_this*, unsigned *bits*)

Definition at line 172 of file rangedec.c.

Referenced by ec_dec_bits().

6.19 libcelt/entenc.c File Reference

```
#include "os_support.h"
#include "entenc.h"
```

Defines

- #define EC_BUFFER_INCREMENT (256)

Functions

- void ec_byte_writeinit (ec_byte_buffer *_b)
- void ec_byte_writetrunc (ec_byte_buffer *_b, long _bytes)
- void ec_byte_write1 (ec_byte_buffer *_b, unsigned _value)
- void ec_byte_write4 (ec_byte_buffer *_b, ec_uint32 _value)
- void ec_byte_writecopy (ec_byte_buffer *_b, void *_source, long _bytes)
- void ec_byte_writeclear (ec_byte_buffer *_b)
- void ec_enc_bits (ec_enc *_this, ec_uint32 _fl, int _ftb)
- void ec_enc_bits64 (ec_enc *_this, ec_uint64 _fl, int _ftb)
- void ec_enc_uint (ec_enc *_this, ec_uint32 _fl, ec_uint32 _ft)
- void ec_enc_uint64 (ec_enc *_this, ec_uint64 _fl, ec_uint64 _ft)

6.19.1 Define Documentation

6.19.1.1 #define EC_BUFFER_INCREMENT (256)

Definition at line 10 of file entenc.c.

Referenced by ec_byte_write1(), ec_byte_write4(), ec_byte_writecopy(), and ec_byte_writeinit().

6.19.2 Function Documentation

6.19.2.1 void ec_byte_write1 (ec_byte_buffer * *_b*, unsigned *_value*)

Definition at line 21 of file entenc.c.

References ec_byte_buffer::buf, EC_BUFFER_INCREMENT, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.19.2.2 void ec_byte_write4 (ec_byte_buffer * *_b*, ec_uint32 *_value*)

Definition at line 32 of file entenc.c.

References ec_byte_buffer::buf, EC_BUFFER_INCREMENT, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.19.2.3 void ec_byte_writeclear (ec_byte_buffer * *_b*)

Definition at line 61 of file entenc.c.

References ec_byte_buffer::buf.

6.19.2.4 void ec_byte_writecopy (ec_byte_buffer * _b, void * _source, long _bytes)

Definition at line 49 of file entenc.c.

References ec_byte_buffer::buf, EC_BUFFER_INCREMENT, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.19.2.5 void ec_byte_writeinit (ec_byte_buffer * _b)

Definition at line 12 of file entenc.c.

References ec_byte_buffer::buf, EC_BUFFER_INCREMENT, ec_byte_buffer::ptr, and ec_byte_buffer::storage.

6.19.2.6 void ec_byte_writetrunc (ec_byte_buffer * _b, long _bytes)

Definition at line 17 of file entenc.c.

References ec_byte_buffer::buf, and ec_byte_buffer::ptr.

6.19.2.7 void ec_enc_bits (ec_enc * _this, ec_uint32 _fl, int _ftb)

Definition at line 67 of file entenc.c.

Referenced by celt_encode(), ec_enc_bits64(), ec_enc_uint(), intra_prediction(), and quant_pitch().

6.19.2.8 void ec_enc_bits64 (ec_enc * _this, ec_uint64 _fl, int _ftb)

Definition at line 80 of file entenc.c.

Referenced by ec_enc_uint64().

6.19.2.9 void ec_enc_uint (ec_enc * _this, ec_uint32 _fl, ec_uint32 _ft)

Definition at line 89 of file entenc.c.

Referenced by celt_encode().

6.19.2.10 void ec_enc_uint64 (ec_enc * _this, ec_uint64 _fl, ec_uint64 _ft)

Definition at line 106 of file entenc.c.

6.20 libcelt/entenc.h File Reference

```
#include <stddef.h>
#include "entcode.h"
```

Data Structures

- struct ec_enc

Defines

- #define _entenc_H (1)

Typedefs

- typedef struct ec_enc ec_enc

Functions

- void ec_enc_init (ec_enc *_this, ec_byte_buffer *_buf)
- void ec_encode (ec_enc *_this, unsigned _fl, unsigned _fh, unsigned _ft)
- void ec_encode_bin (ec_enc *_this, unsigned _fl, unsigned _fh, unsigned bits)
- void ec_enc_bits (ec_enc *_this, ec_uint32 _fl, int _ftb)
- void ec_enc_bits64 (ec_enc *_this, ec_uint64 _fl, int _ftb)
- void ec_enc_uint (ec_enc *_this, ec_uint32 _fl, ec_uint32 _ft)
- void ec_enc_uint64 (ec_enc *_this, ec_uint64 _fl, ec_uint64 _ft)
- long ec_enc_tell (ec_enc *_this, int _b)
- void ec_enc_done (ec_enc *_this)

6.20.1 Define Documentation

6.20.1.1 #define _entenc_H (1)

Definition at line 2 of file entenc.h.

6.20.2 Typedef Documentation

6.20.2.1 typedef struct ec_enc ec_enc

Definition at line 8 of file entenc.h.

6.20.3 Function Documentation

6.20.3.1 void ec_enc_bits (ec_enc * *_this*, ec_uint32 *_fl*, int *_ftb*)

Definition at line 67 of file entenc.c.

References ec_encode_bin(), EC_UNIT_BITS, and EC_UNIT_MASK.

6.20.3.2 void ec_enc_bits64 (ec_enc * *_this*, ec_uint64 *_fl*, int *_ftb*)

Definition at line 80 of file entenc.c.

References ec_enc_bits().

6.20.3.3 void ec_enc_done (ec_enc * *_this*)

Definition at line 133 of file rangeenc.c.

Referenced by celt_encode().

6.20.3.4 void ec_enc_init (ec_enc * *_this*, ec_byte_buffer * *_buf*)

Definition at line 77 of file rangeenc.c.

Referenced by celt_encode(), and celt_encoder_create().

6.20.3.5 long ec_enc_tell (ec_enc * *_this*, int *_b*)

Definition at line 108 of file rangeenc.c.

Referenced by celt_encode(), and quant_bands().

6.20.3.6 void ec_enc_uint (ec_enc * *_this*, ec_uint32 *_fl*, ec_uint32 *_ft*)

Definition at line 89 of file entenc.c.

References ec_enc_bits(), ec_encode(), EC_ILOG, and EC_UNIT_BITS.

6.20.3.7 void ec_enc_uint64 (ec_enc * *_this*, ec_uint64 *_fl*, ec_uint64 *_ft*)

Definition at line 106 of file entenc.c.

References ec_enc_bits64(), ec_encode(), EC_ILOG64, and EC_UNIT_BITS.

6.20.3.8 void ec_encode (ec_enc * *_this*, unsigned *_fl*, unsigned *_fh*, unsigned *_ft*)

Definition at line 85 of file rangeenc.c.

Referenced by ec_enc_uint(), ec_enc_uint64(), and ec_laplace_encode_start().

6.20.3.9 void ec_encode_bin (ec_enc * *_this*, unsigned *_fl*, unsigned *_fh*, unsigned *bits*)

Definition at line 96 of file rangeenc.c.

Referenced by ec_enc_bits().

6.21 libcelt/entropy.m File Reference

Functions

- plot ([0:length(p)-1]/(length(p)-1)∗(ma-mi)+mi, p)

Variables

- function b
- mi = round(min(v))
- ma = round(max(v))
- p = zeros(1,ma-mi+1)
- for k
- end p = p./sum(p)
- b = sum(-p.∗log2(p+.0000000000001))

6.21.1 Function Documentation

6.21.1.1 plot ([0:length(p)-1] /(length(p)-1)∗(ma-mi)+ *mi*, p)

6.21.2 Variable Documentation

6.21.2.1 b = sum(-p.∗log2(p+.0000000000001))

Definition at line 13 of file entropy.m.

6.21.2.2 function b

Initial value:

```
entropy(v)

    r = round(v)
```

Definition at line 1 of file entropy.m.

6.21.2.3 for k

Initial value:

```
1:ma-mi+1
        p(k)=length(find(r==k+mi-1))
```

Definition at line 8 of file entropy.m.

6.21.2.4 ma = round(max(v))

Definition at line 5 of file entropy.m.

6.21.2.5 mi = round(min(v))

Definition at line 4 of file entropy.m.

6.21.2.6 end p = p./sum(p)

Definition at line 11 of file entropy.m.

6.21.2.7 p = zeros(1,ma-mi+1)

Definition at line 7 of file entropy.m.

Referenced by cwrsi32(), cwrsi64(), icwrs32(), and icwrs64().

6.22 libcelt/entropy2.m File Reference

Functions

- plot ([0:length(p)-1]/(length(p)-1)∗(ma-mi)+mi, p)

Variables

- function b
- mi = round(min(v))
- ma = round(max(v))
- p = zeros(1,ma-mi+1)
- for k
- end p = p./sum(p)
- b = sum(-p.∗log2(p+.0000000000001))

6.22.1 Function Documentation

6.22.1.1 plot ([0:length(p)-1] /(length(p)-1)∗(ma-mi)+ *mi*, p)

6.22.2 Variable Documentation

6.22.2.1 b = sum(-p.∗log2(p+.0000000000001))

Definition at line 13 of file entropy2.m.

6.22.2.2 function b

Initial value:

```
entropy2(v)

    r = round(v)
```

Definition at line 1 of file entropy2.m.

6.22.2.3 for k

Initial value:

```
1:ma-mi+1
        p(k)=length(find(r==k+mi-1))
```

Definition at line 8 of file entropy2.m.

6.22.2.4 ma = round(max(v))

Definition at line 5 of file entropy2.m.

6.22.2.5 mi = round(min(v))

Definition at line 4 of file entropy2.m.

6.22.2.6 end p = p./sum(p)

Definition at line 11 of file entropy2.m.

6.22.2.7 p = zeros(1,ma-mi+1)

Definition at line 7 of file entropy2.m.

6.23 libcelt/fixed_c5x.h File Reference

Fixed-point operations for the TI C5x DSP family.

```
#include "dsplib.h"
```

Defines

- #define MAX16(a, b) _max(a,b)
- #define MIN16(a, b) _min(a,b)
- #define MAX32(a, b) _lmax(a,b)
- #define MIN32(a, b) _lmin(a,b)
- #define VSHR32(a, shift) _lshl(a,-(shift))
- #define MULT16_16_Q15(a, b) (_smpy(a,b))
- #define MULT16_16SU(a, b) _lmpysu(a,b)
- #define MULT_16_16(a, b) _lmpy(a,b)
- #define MULT16_32_Q15(a, b) ADD32(SHL(MULT16_16((a),SHR((b),16)),1), SHR(MULT16_16SU((a),(b)),15))
- #define celt_ilog2(x) (30 - _lnorm(x))
- #define OVERRIDE_CELT_ILOG2
- #define celt_maxabs16(x, len) MAX16(maxval((DATA *)x, len),-minval((DATA *)x, len))
- #define OVERRIDE_CELT_MAXABS16
- #define OVERRIDE_FIND_MAX16

6.23.1 Detailed Description

Fixed-point operations for the TI C5x DSP family.

Definition in file fixed_c5x.h.

6.23.2 Define Documentation

6.23.2.1 #define celt_ilog2(x) (30 - _lnorm(x))

Definition at line 78 of file fixed_c5x.h.

Referenced by alg_quant(), decode_pulses(), encode_pulses(), and find_spectral_pitch().

6.23.2.2 #define celt_maxabs16(x, len) MAX16(maxval((DATA *)x, len),-minval((DATA *)x, len))

Definition at line 81 of file fixed_c5x.h.

6.23.2.3 #define MAX16(a, b) _max(a,b)

Definition at line 50 of file fixed_c5x.h.

6.23.2.4 #define MAX32(a, b) _lmax(a,b)

Definition at line 56 of file fixed_c5x.h.

6.23.2.5 #define MIN16(a, b) _min(a,b)

Definition at line 53 of file fixed_c5x.h.

6.23.2.6 #define MIN32(a, b) _lmin(a,b)

Definition at line 59 of file fixed_c5x.h.

6.23.2.7 #define MULT16_16_Q15(a, b) (_smpy(a,b))

Definition at line 65 of file fixed_c5x.h.

6.23.2.8 #define MULT16_16SU(a, b) _lmpysu(a,b)

Definition at line 68 of file fixed_c5x.h.

6.23.2.9 #define MULT16_32_Q15(a, b) ADD32(SHL(MULT16_16((a),SHR((b),16)),1), SHR(MULT16_16SU((a),(b)),15))

Definition at line 75 of file fixed_c5x.h.

6.23.2.10 #define MULT_16_16(a, b) _lmpy(a,b)

Definition at line 71 of file fixed_c5x.h.

6.23.2.11 #define OVERRIDE_CELT_ILOG2

Definition at line 79 of file fixed_c5x.h.

6.23.2.12 #define OVERRIDE_CELT_MAXABS16

Definition at line 82 of file fixed_c5x.h.

6.23.2.13 #define OVERRIDE_FIND_MAX16

Definition at line 84 of file fixed_c5x.h.

6.23.2.14 #define VSHR32(a, shift) _lshl(a,-(shift))

Definition at line 62 of file fixed_c5x.h.

6.24 libcelt/fixed_c6x.h File Reference

Fixed-point operations for the TI C6x DSP family.

Defines

- #define MULT16_16SU(a, b) _mpysu(a,b)
- #define MULT_16_16(a, b) _mpy(a,b)
- #define celt_ilog2(x) (30 - _norm(x))
- #define OVERRIDE_CELT_ILOG2
- #define MULT16_32_Q15(a, b) ADD32(SHL(_mpylh(a,b),1), SHR(_mpsu(a,b),15)

6.24.1 Detailed Description

Fixed-point operations for the TI C6x DSP family.

Definition in file fixed_c6x.h.

6.24.2 Define Documentation

6.24.2.1 #define celt_ilog2(x) (30 - _norm(x))

Definition at line 44 of file fixed_c6x.h.

6.24.2.2 #define MULT16_16SU(a, b) _mpysu(a,b)

Definition at line 39 of file fixed_c6x.h.

6.24.2.3 #define MULT16_32_Q15(a, b) ADD32(SHL(_mpylh(a,b),1), SHR(_mpsu(a,b),15)

Definition at line 48 of file fixed_c6x.h.

6.24.2.4 #define MULT_16_16(a, b) _mpy(a,b)

Definition at line 42 of file fixed_c6x.h.

6.24.2.5 #define OVERRIDE_CELT_ILOG2

Definition at line 45 of file fixed_c6x.h.

6.25 libcelt/fixed_debug.h File Reference

Fixed-point operations with debugging.

```
#include <stdio.h>
```

Defines

- #define MIPS_INC celt_mips++,
- #define MULT16_16SU(a, b) ((celt_word32_t)(celt_word16_t)(a)∗(celt_word32_t)(celt_uint16_t)(b))
- #define MULT32_32_Q31(a, b) ADD32(ADD32(SHL32(MULT16_16(SHR((a),16),SHR((b),16)),1), SHR32(MULT16_16SU(SHR((a),16),((b)&0x0000ffff)),15)), SHR(MULT16_16SU(SHR((b),16),((a)&0x0000ffff)),15))
- #define MULT16_32_Q16(a, b) ADD32(MULT16_16((a),SHR((b),16)), SHR(MULT16_16SU((a),((b)&0x0000ffff)),16))
- #define QCONST16(x, bits) ((celt_word16_t)(.5+(x)∗(((celt_word32_t)1)<<(bits))))
- #define QCONST32(x, bits) ((celt_word32_t)(.5+(x)∗(((celt_word32_t)1)<<(bits))))
- #define VERIFY_SHORT(x) ((x)<=32767&&(x)>=-32768)
- #define VERIFY_INT(x) ((x)<=2147483647LL&&(x)>=-2147483648LL)
- #define SHR(a, b) SHR32(a,b)
- #define PSHR(a, b) PSHR32(a,b)
- #define EXTRACT16(x) _EXTRACT16(x, __FILE__, __LINE__)
- #define EXTEND32(x) _EXTEND32(x, __FILE__, __LINE__)
- #define SHR16(a, shift) _SHR16(a, shift, __FILE__, __LINE__)
- #define SHL16(a, shift) _SHL16(a, shift, __FILE__, __LINE__)
- #define PSHR16(a, shift) (SHR16(ADD16((a),((1<<((shift))>>1))),shift))
- #define PSHR32(a, shift) (SHR32(ADD32((a),((EXTEND32(1)<<((shift))>>1))),shift))
- #define VSHR32(a, shift) (((shift)>0) ? SHR32(a, shift) : SHL32(a, -(shift)))
- #define SATURATE16(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))
- #define SATURATE32(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))
- #define ROUND16(x, a) (EXTRACT16(PSHR32((x),(a))))
- #define HALF32(x) (SHR32(x,1))
- #define ADD16(a, b) _ADD16(a, b, __FILE__, __LINE__)
- #define SUB16(a, b) _SUB16(a, b, __FILE__, __LINE__)
- #define ADD32(a, b) _ADD32(a, b, __FILE__, __LINE__)
- #define ADD64(a, b) (MIPS_INC(a)+(b))
- #define MULT16_16(a, b) _MULT16_16(a, b, __FILE__, __LINE__)
- #define MAC16_16(c, a, b) (celt_mips–,ADD32((c),MULT16_16((a),(b))))
- #define MAC16_16_Q11(c, a, b) (EXTRACT16(ADD16((c),EXTRACT16(SHR32(MULT16_16((a),(b)),11)))))
- #define MAC16_16_Q13(c, a, b) (EXTRACT16(ADD16((c),EXTRACT16(SHR32(MULT16_16((a),(b)),13)))))
- #define MAC16_16_P13(c, a, b) (EXTRACT16(ADD32((c),SHR32(ADD32(4096,MULT16_16((a),(b))),13))))
- #define MULT16_32_QX(a, b, Q) _MULT16_32_QX(a, b, Q, __FILE__, __LINE__)
- #define MULT16_32_Q11(a, b) MULT16_32_QX(a,b,11)
- #define MAC16_32_Q11(c, a, b) ADD32((c),MULT16_32_Q11((a),(b)))
- #define MULT16_32_Q12(a, b) MULT16_32_QX(a,b,12)
- #define MULT16_32_Q13(a, b) MULT16_32_QX(a,b,13)
- #define MULT16_32_Q14(a, b) MULT16_32_QX(a,b,14)

- #define MULT16_32_Q15(a, b) MULT16_32_QX(a,b,15)
- #define MULT16_32_P15(a, b) MULT16_32_PX(a,b,15)
- #define MAC16_32_Q15(c, a, b) ADD32((c),MULT16_32_Q15((a),(b)))
- #define DIV32_16(a, b) _DIV32_16(a, b, __FILE__, __LINE__)
- #define DIV32(a, b) _DIV32(a, b, __FILE__, __LINE__)
- #define PDIV32(a, b) DIV32(ADD32((a),(b)>>1),b)
- #define PDIV32_16(a, b) DIV32_16(ADD32((a),(b)>>1),b)

6.25.1 Detailed Description

Fixed-point operations with debugging.

Definition in file fixed_debug.h.

6.25.2 Define Documentation

6.25.2.1 #define ADD16(a, b) _ADD16(a, b, __FILE__, __LINE__)

Definition at line 186 of file fixed_debug.h.

6.25.2.2 #define ADD32(a, b) _ADD32(a, b, __FILE__, __LINE__)

Definition at line 218 of file fixed_debug.h.

6.25.2.3 #define ADD64(a, b) (MIPS_INC(a)+(b))

Definition at line 249 of file fixed_debug.h.

6.25.2.4 #define DIV32(a, b) _DIV32(a, b, __FILE__, __LINE__)

Definition at line 477 of file fixed_debug.h.

6.25.2.5 #define DIV32_16(a, b) _DIV32_16(a, b, __FILE__, __LINE__)

Definition at line 450 of file fixed_debug.h.

6.25.2.6 #define EXTEND32(x) _EXTEND32(x, __FILE__, __LINE__)

Definition at line 100 of file fixed_debug.h.

6.25.2.7 #define EXTRACT16(x) _EXTRACT16(x, __FILE__, __LINE__)

Definition at line 87 of file fixed_debug.h.

6.25.2.8 #define HALF32(x) (SHR32(x,1))

Definition at line 181 of file fixed_debug.h.

6.25.2.9 #define MAC16_16(c, a, b) (celt_mips–,ADD32((c),MULT16_16((a),(b))))

Definition at line 281 of file fixed_debug.h.

6.25.2.10 #define MAC16_16_P13(c, a, b) (EXTRACT16(ADD32((c),SHR32(ADD32(4096,MULT16_16((a),(b))),13))))

Definition at line 284 of file fixed_debug.h.

6.25.2.11 #define MAC16_16_Q11(c, a, b) (EXTRACT16(ADD16((c),EXTRACT16(SHR32(MULT16_16((a),(b)),11)))))

Definition at line 282 of file fixed_debug.h.

6.25.2.12 #define MAC16_16_Q13(c, a, b) (EXTRACT16(ADD16((c),EXTRACT16(SHR32(MULT16_16((a),(b)),13)))))

Definition at line 283 of file fixed_debug.h.

6.25.2.13 #define MAC16_32_Q11(c, a, b) ADD32((c),MULT16_32_Q11((a),(b)))

Definition at line 322 of file fixed_debug.h.

6.25.2.14 #define MAC16_32_Q15(c, a, b) ADD32((c),MULT16_32_Q15((a),(b)))

Definition at line 328 of file fixed_debug.h.

6.25.2.15 #define MIPS_INC celt_mips++,

Definition at line 42 of file fixed_debug.h.

6.25.2.16 #define MULT16_16(a, b) _MULT16_16(a, b, __FILE__, __LINE__)

Definition at line 266 of file fixed_debug.h.

6.25.2.17 #define MULT16_16SU(a, b) ((celt_word32_t)(celt_word16_t)(a)∗(celt_word32_t)(celt_uint16_t)(b))

Definition at line 44 of file fixed_debug.h.

6.25.2.18 #define MULT16_32_P15(a, b) MULT16_32_PX(a,b,15)

Definition at line 327 of file fixed_debug.h.

6.25.2.19 #define MULT16_32_Q11(a, b) MULT16_32_QX(a,b,11)

Definition at line 321 of file fixed_debug.h.

6.25.2.20 #define MULT16_32_Q12(a, b) MULT16_32_QX(a,b,12)

Definition at line 323 of file fixed_debug.h.

6.25.2.21 #define MULT16_32_Q13(a, b) MULT16_32_QX(a,b,13)

Definition at line 324 of file fixed_debug.h.

6.25.2.22 #define MULT16_32_Q14(a, b) MULT16_32_QX(a,b,14)

Definition at line 325 of file fixed_debug.h.

6.25.2.23 #define MULT16_32_Q15(a, b) MULT16_32_QX(a,b,15)

Definition at line 326 of file fixed_debug.h.

6.25.2.24 #define MULT16_32_Q16(a, b) ADD32(MULT16_16((a),SHR((b),16)), SHR(MULT16_16SU((a),((b)&0x0000ffff)),16))

16x32 multiplication, followed by a 16-bit shift right. Results fits in 32 bits

Definition at line 48 of file fixed_debug.h.

6.25.2.25 #define MULT16_32_QX(a, b, Q) _MULT16_32_QX(a, b, Q, __FILE__, __LINE__)

Definition at line 287 of file fixed_debug.h.

6.25.2.26 #define MULT32_32_Q31(a, b) ADD32(ADD32(SHL32(MULT16_16(SHR((a),16),SHR((b),16)),1), SHR32(MULT16_16SU(SHR((a),16),((b)&0x0000ffff)),15)), SHR(MULT16_16SU(SHR((b),16),((a)&0x0000ffff)),15))

Definition at line 45 of file fixed_debug.h.

6.25.2.27 #define PDIV32(a, b) DIV32(ADD32((a),(b)>>1),b)

Definition at line 497 of file fixed_debug.h.

6.25.2.28 #define PDIV32_16(a, b) DIV32_16(ADD32((a),(b)>>1),b)

Definition at line 498 of file fixed_debug.h.

6.25.2.29 #define PSHR(a, b) PSHR32(a,b)

Definition at line 58 of file fixed_debug.h.

6.25.2.30 #define PSHR16(a, shift) (SHR16(ADD16((a),((1<<((shift))>>1))),shift))

Definition at line 173 of file fixed_debug.h.

6.25.2.31 #define PSHR32(a, shift) (SHR32(ADD32((a),((EXTEND32(1)<<((shift))>>1))),shift))

Definition at line 174 of file fixed_debug.h.

6.25.2.32 #define QCONST16(x, bits) ((celt_word16_t)(.5+(x)∗(((celt_word32_t)1)<<(bits))))

Definition at line 50 of file fixed_debug.h.

6.25.2.33 #define QCONST32(x, bits) ((celt_word32_t)(.5+(x)∗(((celt_word32_t)1)<<(bits))))

Definition at line 51 of file fixed_debug.h.

6.25.2.34 #define ROUND16(x, a) (EXTRACT16(PSHR32((x),(a))))

Definition at line 180 of file fixed_debug.h.

6.25.2.35 #define SATURATE16(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))

Definition at line 177 of file fixed_debug.h.

6.25.2.36 #define SATURATE32(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))

Definition at line 178 of file fixed_debug.h.

6.25.2.37 #define SHL16(a, shift) _SHL16(a, shift, __FILE__, __LINE__)

Definition at line 127 of file fixed_debug.h.

6.25.2.38 #define SHR(a, b) SHR32(a,b)

Definition at line 57 of file fixed_debug.h.

6.25.2.39 #define SHR16(a, shift) _SHR16(a, shift, __FILE__, __LINE__)

Definition at line 113 of file fixed_debug.h.

6.25.2.40 #define SUB16(a, b) _SUB16(a, b, __FILE__, __LINE__)

Definition at line 203 of file fixed_debug.h.

6.25.2.41 #define VERIFY_INT(x) ((x)<=2147483647LL&&(x)>=-2147483648LL)

Definition at line 55 of file fixed_debug.h.

6.25.2.42 #define VERIFY_SHORT(x) ((x)<=32767&&(x)>=-32768)

Definition at line 54 of file fixed_debug.h.

6.25.2.43 #define VSHR32(a, shift) (((shift)>0) ? SHR32(a, shift) : SHL32(a, -(shift)))

Definition at line 175 of file fixed_debug.h.

6.26 libcelt/fixed_generic.h File Reference

Generic fixed-point operations.

Defines

- #define MULT16_16SU(a, b) ((celt_word32_t)(celt_word16_t)(a)*(celt_word32_t)(celt_uint16_t)(b))
- #define MULT16_32_Q16(a, b) ADD32(MULT16_16((a),SHR((b),16)), SHR(MULT16_16SU((a),((b)&0x0000ffff)),16))
- #define MULT16_32_Q15(a, b) ADD32(SHL(MULT16_16((a),SHR((b),16)),1), SHR(MULT16_16SU((a),((b)&0x0000ffff)),15))
- #define MULT32_32_Q31(a, b) ADD32(ADD32(SHL(MULT16_16(SHR((a),16),SHR((b),16)),1), SHR(MULT16_16SU(SHR((a),16),((b)&0x0000ffff)),15)), SHR(MULT16_16SU(SHR((b),16),((a)&0x0000ffff)),15))
- #define MULT32_32_Q32(a, b) ADD32(ADD32(MULT16_16(SHR((a),16),SHR((b),16)), SHR(MULT16_16SU(SHR((a),16),((b)&0x0000ffff)),16)), SHR(MULT16_16SU(SHR((b),16),((a)&0x0000ffff)),16))
- #define QCONST16(x, bits) ((celt_word16_t)(.5+(x)*(((celt_word32_t)1)<<(bits))))
- #define QCONST32(x, bits) ((celt_word32_t)(.5+(x)*(((celt_word32_t)1)<<(bits))))
- #define NEG16(x) (-(x))
- #define NEG32(x) (-(x))
- #define EXTRACT16(x) ((celt_word16_t)(x))
- #define EXTEND32(x) ((celt_word32_t)(x))
- #define SHR16(a, shift) ((a) >> (shift))
- #define SHL16(a, shift) ((a) << (shift))
- #define SHR32(a, shift) ((a) >> (shift))
- #define SHL32(a, shift) ((celt_word32_t)(a) << (shift))
- #define PSHR16(a, shift) (SHR16((a)+((1<<((shift))>>1)),shift))
- #define PSHR32(a, shift) (SHR32((a)+((EXTEND32(1)<<((shift))>>1)),shift))
- #define VSHR32(a, shift) (((shift)>0) ? SHR32(a, shift) : SHL32(a, -(shift)))
- #define SATURATE16(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))
- #define SATURATE32(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))
- #define SHR(a, shift) ((a) >> (shift))
- #define SHL(a, shift) ((celt_word32_t)(a) << (shift))
- #define PSHR(a, shift) (SHR((a)+((EXTEND32(1)<<((shift))>>1)),shift))
- #define SATURATE(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))
- #define ROUND16(x, a) (EXTRACT16(PSHR32((x),(a))))
- #define HALF32(x) (SHR32(x,1))
- #define ADD16(a, b) ((celt_word16_t)((celt_word16_t)(a)+(celt_word16_t)(b)))
- #define SUB16(a, b) ((celt_word16_t)(a)-(celt_word16_t)(b))
- #define ADD32(a, b) ((celt_word32_t)(a)+(celt_word32_t)(b))
- #define SUB32(a, b) ((celt_word32_t)(a)-(celt_word32_t)(b))
- #define MULT16_16_16(a, b) ((((celt_word16_t)(a))*((celt_word16_t)(b))))
- #define MULT16_16(a, b) (((celt_word32_t)(celt_word16_t)(a))*((celt_word32_t)(celt_word16_t)(b)))
- #define MAC16_16(c, a, b) (ADD32((c),MULT16_16((a),(b))))
- #define MULT16_32_Q12(a, b) ADD32(MULT16_16((a),SHR((b),12)), SHR(MULT16_16((a),((b)&0x00000fff)),12))

- #define MULT16_32_Q13(a, b) ADD32(MULT16_16((a),SHR((b),13)), SHR(MULT16_16((a),((b)&0x00001fff)),13))
- #define MULT16_32_Q14(a, b) ADD32(MULT16_16((a),SHR((b),14)), SHR(MULT16_16((a),((b)&0x00003fff)),14))
- #define MULT16_32_Q11(a, b) ADD32(MULT16_16((a),SHR((b),11)), SHR(MULT16_16((a),((b)&0x000007ff)),11))
- #define MAC16_32_Q11(c, a, b) ADD32(c,ADD32(MULT16_16((a),SHR((b),11)), SHR(MULT16_16((a),((b)&0x000007ff)),11)))
- #define MULT16_32_P15(a, b) ADD32(MULT16_16((a),SHR((b),15)), PSHR(MULT16_16((a),((b)&0x00007fff)),15))
- #define MAC16_32_Q15(c, a, b) ADD32(c,ADD32(MULT16_16((a),SHR((b),15)), SHR(MULT16_16((a),((b)&0x00007fff)),15)))
- #define MAC16_16_Q11(c, a, b) (ADD32((c),SHR(MULT16_16((a),(b)),11)))
- #define MAC16_16_Q13(c, a, b) (ADD32((c),SHR(MULT16_16((a),(b)),13)))
- #define MAC16_16_P13(c, a, b) (ADD32((c),SHR(ADD32(4096,MULT16_16((a),(b))),13)))
- #define MULT16_16_Q11_32(a, b) (SHR(MULT16_16((a),(b)),11))
- #define MULT16_16_Q13(a, b) (SHR(MULT16_16((a),(b)),13))
- #define MULT16_16_Q14(a, b) (SHR(MULT16_16((a),(b)),14))
- #define MULT16_16_Q15(a, b) (SHR(MULT16_16((a),(b)),15))
- #define MULT16_16_P13(a, b) (SHR(ADD32(4096,MULT16_16((a),(b))),13))
- #define MULT16_16_P14(a, b) (SHR(ADD32(8192,MULT16_16((a),(b))),14))
- #define MULT16_16_P15(a, b) (SHR(ADD32(16384,MULT16_16((a),(b))),15))
- #define DIV32_16(a, b) ((celt_word16_t)(((celt_word32_t)(a))/((celt_word16_t)(b))))
- #define PDIV32_16(a, b) ((celt_word16_t)(((celt_word32_t)(a)+((celt_word16_t)(b)>>1))/((celt_word16_t)(b))))
- #define DIV32(a, b) (((celt_word32_t)(a))/((celt_word32_t)(b)))
- #define PDIV32(a, b) (((celt_word32_t)(a)+((celt_word16_t)(b)>>1))/((celt_word32_t)(b)))

6.26.1 Detailed Description

Generic fixed-point operations.

Definition in file fixed_generic.h.

6.26.2 Define Documentation

6.26.2.1 #define ADD16(a, b) ((celt_word16_t)((celt_word16_t)(a)+(celt_word16_t)(b)))

Add two 16-bit values

Definition at line 101 of file fixed_generic.h.

6.26.2.2 #define ADD32(a, b) ((celt_word32_t)(a)+(celt_word32_t)(b))

Add two 32-bit values

Definition at line 105 of file fixed_generic.h.

6.26.2.3 #define DIV32(a, b) (((celt_word32_t)(a))/((celt_word32_t)(b)))

Divide a 32-bit value by a 32-bit value. Result fits in 32 bits

Definition at line 155 of file fixed_generic.h.

6.26.2.4 #define DIV32_16(a, b) ((celt_word16_t)(((celt_word32_t)(a))/((celt_word16_t)(b))))

Divide a 32-bit value by a 16-bit value. Result fits in 16 bits

Definition at line 151 of file fixed_generic.h.

6.26.2.5 #define EXTEND32(x) ((celt_word32_t)(x))

Change a 16-bit value into a 32-bit value

Definition at line 66 of file fixed_generic.h.

6.26.2.6 #define EXTRACT16(x) ((celt_word16_t)(x))

Change a 32-bit value into a 16-bit value. The value is assumed to fit in 16-bit, otherwise the result is undefined

Definition at line 64 of file fixed_generic.h.

6.26.2.7 #define HALF32(x) (SHR32(x,1))

Divide by two

Definition at line 98 of file fixed_generic.h.

6.26.2.8 #define MAC16_16(c, a, b) (ADD32((c),MULT16_16((a),(b))))

16x16 multiply-add where the result fits in 32 bits

Definition at line 118 of file fixed_generic.h.

6.26.2.9 #define MAC16_16_P13(c, a, b) (ADD32((c),SHR(ADD32(4096,MULT16_16((a),(b))),13)))

Definition at line 139 of file fixed_generic.h.

6.26.2.10 #define MAC16_16_Q11(c, a, b) (ADD32((c),SHR(MULT16_16((a),(b)),11)))

Definition at line 137 of file fixed_generic.h.

6.26.2.11 #define MAC16_16_Q13(c, a, b) (ADD32((c),SHR(MULT16_16((a),(b)),13)))

Definition at line 138 of file fixed_generic.h.

6.26.2.12 #define MAC16_32_Q11(c, a, b) ADD32(c,ADD32(MULT16_16((a),SHR((b),11)), SHR(MULT16_16((a),((b)&0x000007ff)),11)))

16x32 multiply-add, followed by an 11-bit shift right. Results fits in 32 bits

Definition at line 129 of file fixed_generic.h.

6.26.2.13 #define MAC16_32_Q15(c, a, b) ADD32(c,ADD32(MULT16_16((a),SHR((b),15)), SHR(MULT16_16((a),((b)&0x00007fff)),15)))

16x32 multiply-add, followed by a 15-bit shift right. Results fits in 32 bits

Definition at line 134 of file fixed_generic.h.

6.26.2.14 #define MULT16_16(a, b) (((celt_word32_t)(celt_word16_t)(a))∗((celt_word32_t)(celt_word16_t)(b)))

16x16 multiplication where the result fits in 32 bits

Definition at line 115 of file fixed_generic.h.

6.26.2.15 #define MULT16_16_16(a, b) ((((celt_word16_t)(a))∗((celt_word16_t)(b))))

16x16 multiplication where the result fits in 16 bits

Definition at line 111 of file fixed_generic.h.

6.26.2.16 #define MULT16_16_P13(a, b) (SHR(ADD32(4096,MULT16_16((a),(b))),13))

Definition at line 146 of file fixed_generic.h.

6.26.2.17 #define MULT16_16_P14(a, b) (SHR(ADD32(8192,MULT16_16((a),(b))),14))

Definition at line 147 of file fixed_generic.h.

6.26.2.18 #define MULT16_16_P15(a, b) (SHR(ADD32(16384,MULT16_16((a),(b))),15))

Definition at line 148 of file fixed_generic.h.

6.26.2.19 #define MULT16_16_Q11_32(a, b) (SHR(MULT16_16((a),(b)),11))

Definition at line 141 of file fixed_generic.h.

6.26.2.20 #define MULT16_16_Q13(a, b) (SHR(MULT16_16((a),(b)),13))

Definition at line 142 of file fixed_generic.h.

6.26.2.21 #define MULT16_16_Q14(a, b) (SHR(MULT16_16((a),(b)),14))

Definition at line 143 of file fixed_generic.h.

6.26.2.22 #define MULT16_16_Q15(a, b) (SHR(MULT16_16((a),(b)),15))

Definition at line 144 of file fixed_generic.h.

6.26.2.23 #define MULT16_16SU(a, b) ((celt_word32_t)(celt_word16_t)(a)∗(celt_word32_t)(celt_-uint16_t)(b))

Multiply a 16-bit signed value by a 16-bit unsigned value. The result is a 32-bit signed value

Definition at line 39 of file fixed_generic.h.

6.26.2.24 #define MULT16_32_P15(a, b) ADD32(MULT16_16((a),SHR((b),15)), PSHR(MULT16_16((a),((b)&0x00007fff)),15))

16x32 multiplication, followed by a 15-bit shift right (round-to-nearest). Results fits in 32 bits

Definition at line 132 of file fixed_generic.h.

6.26.2.25 #define MULT16_32_Q11(a, b) ADD32(MULT16_16((a),SHR((b),11)), SHR(MULT16_16((a),((b)&0x000007ff)),11))

16x32 multiplication, followed by an 11-bit shift right. Results fits in 32 bits

Definition at line 127 of file fixed_generic.h.

6.26.2.26 #define MULT16_32_Q12(a, b) ADD32(MULT16_16((a),SHR((b),12)), SHR(MULT16_16((a),((b)&0x00000fff)),12))

16x32 multiplication, followed by a 12-bit shift right. Results fits in 32 bits

Definition at line 120 of file fixed_generic.h.

6.26.2.27 #define MULT16_32_Q13(a, b) ADD32(MULT16_16((a),SHR((b),13)), SHR(MULT16_16((a),((b)&0x00001fff)),13))

16x32 multiplication, followed by a 13-bit shift right. Results fits in 32 bits

Definition at line 122 of file fixed_generic.h.

6.26.2.28 #define MULT16_32_Q14(a, b) ADD32(MULT16_16((a),SHR((b),14)), SHR(MULT16_16((a),((b)&0x00003fff)),14))

16x32 multiplication, followed by a 14-bit shift right. Results fits in 32 bits

Definition at line 124 of file fixed_generic.h.

6.26.2.29 #define MULT16_32_Q15(a, b) ADD32(SHL(MULT16_16((a),SHR((b),16)),1), SHR(MULT16_16SU((a),((b)&0x0000ffff)),15))

16x32 multiplication, followed by a 15-bit shift right. Results fits in 32 bits

Definition at line 45 of file fixed_generic.h.

6.26.2.30 #define MULT16_32_Q16(a, b) ADD32(MULT16_16((a),SHR((b),16)), SHR(MULT16_16SU((a),((b)&0x0000ffff)),16))

16x32 multiplication, followed by a 16-bit shift right. Results fits in 32 bits

Definition at line 42 of file fixed_generic.h.

6.26.2.31 #define MULT32_32_Q31(a, b) ADD32(ADD32(SHL(MULT16_16(SHR((a),16),SHR((b),16)),1), SHR(MULT16_16SU(SHR((a),16),((b)&0x0000ffff)),15)), SHR(MULT16_16SU(SHR((b),16),((a)&0x0000ffff)),15))

32x32 multiplication, followed by a 31-bit shift right. Results fits in 32 bits

Definition at line 48 of file fixed_generic.h.

6.26.2.32 #define MULT32_32_Q32(a, b) ADD32(ADD32(MULT16_16(SHR((a),16),SHR((b),16)), SHR(MULT16_16SU(SHR((a),16),((b)&0x0000ffff)),16)), SHR(MULT16_16SU(SHR((b),16),((a)&0x0000ffff)),16))

32x32 multiplication, followed by a 32-bit shift right. Results fits in 32 bits

Definition at line 51 of file fixed_generic.h.

6.26.2.33 #define NEG16(x) (-(x))

Negate a 16-bit value

Definition at line 59 of file fixed_generic.h.

6.26.2.34 #define NEG32(x) (-(x))

Negate a 32-bit value

Definition at line 61 of file fixed_generic.h.

6.26.2.35 #define PDIV32(a, b) (((celt_word32_t)(a)+((celt_word16_t)(b)>>1))/((celt_word32_t)(b)))

Divide a 32-bit value by a 32-bit value and round to nearest. Result fits in 32 bits

Definition at line 157 of file fixed_generic.h.

6.26.2.36 #define PDIV32_16(a, b) ((celt_word16_t)(((celt_word32_t)(a)+((celt_word16_t)(b)>>1))/((celt_word16_t)(b))))

Divide a 32-bit value by a 16-bit value and round to nearest. Result fits in 16 bits

Definition at line 153 of file fixed_generic.h.

6.26.2.37 #define PSHR(a, shift) (SHR((a)+((EXTEND32(1)<<((shift))>>1)),shift))

Definition at line 92 of file fixed_generic.h.

6.26.2.38 #define PSHR16(a, shift) (SHR16((a)+((1<<((shift))>>1)),shift))

16-bit arithmetic shift right with rounding-to-nearest instead of rounding down

Definition at line 78 of file fixed_generic.h.

6.26.2.39 #define PSHR32(a, shift) (SHR32((a)+((EXTEND32(1)<<((shift))>>1)),shift))

32-bit arithmetic shift right with rounding-to-nearest instead of rounding down

Definition at line 80 of file fixed_generic.h.

6.26.2.40 #define QCONST16(x, bits) ((celt_word16_t)(.5+(x)∗(((celt_word32_t)1)<<(bits))))

Compile-time conversion of float constant to 16-bit value

Definition at line 54 of file fixed_generic.h.

6.26.2.41 #define QCONST32(x, bits) ((celt_word32_t)(.5+(x)∗(((celt_word32_t)1)<<(bits))))

Compile-time conversion of float constant to 32-bit value

Definition at line 56 of file fixed_generic.h.

6.26.2.42 #define ROUND16(x, a) (EXTRACT16(PSHR32((x),(a))))

Shift by a and round-to-neareast 32-bit value. Result is a 16-bit value

Definition at line 96 of file fixed_generic.h.

6.26.2.43 #define SATURATE(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))

Definition at line 93 of file fixed_generic.h.

6.26.2.44 #define SATURATE16(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))

Saturates 16-bit value to +/- a

Definition at line 85 of file fixed_generic.h.

6.26.2.45 #define SATURATE32(x, a) (((x)>(a) ? (a) : (x)<-(a) ? -(a) : (x)))

Saturates 32-bit value to +/- a

Definition at line 87 of file fixed_generic.h.

6.26.2.46 #define SHL(a, shift) ((celt_word32_t)(a) << (shift))

Definition at line 91 of file fixed_generic.h.

6.26.2.47 #define SHL16(a, shift) ((a) << (shift))

Arithmetic shift-left of a 16-bit value

Definition at line 71 of file fixed_generic.h.

6.26.2.48 #define SHL32(a, shift) ((celt_word32_t)(a) << (shift))

Arithmetic shift-left of a 32-bit value

Definition at line 75 of file fixed_generic.h.

6.26.2.49 #define SHR(a, shift) ((a) >> (shift))

"RAW" macros, should not be used outside of this header file

Definition at line 90 of file fixed_generic.h.

6.26.2.50 #define SHR16(a, shift) ((a) >> (shift))

Arithmetic shift-right of a 16-bit value

Definition at line 69 of file fixed_generic.h.

6.26.2.51 #define SHR32(a, shift) ((a) >> (shift))

Arithmetic shift-right of a 32-bit value

Definition at line 73 of file fixed_generic.h.

6.26.2.52 #define SUB16(a, b) ((celt_word16_t)(a)-(celt_word16_t)(b))

Subtract two 16-bit values

Definition at line 103 of file fixed_generic.h.

6.26.2.53 #define SUB32(a, b) ((celt_word32_t)(a)-(celt_word32_t)(b))

Subtract two 32-bit values

Definition at line 107 of file fixed_generic.h.

6.26.2.54 #define VSHR32(a, shift) (((shift)>0) ? SHR32(a, shift) : SHL32(a, -(shift)))

32-bit arithmetic shift right where the argument can be negative

Definition at line 82 of file fixed_generic.h.

6.27 libcelt/header.c File Reference

```
#include "celt_header.h"
#include "os_support.h"
#include "modes.h"
```

Functions

- void EXPORT celt_header_init (CELTHeader *header, const CELTMode *m)
- int EXPORT celt_header_to_packet (const CELTHeader *header, unsigned char *packet, celt_uint32_t size)
- int EXPORT celt_header_from_packet (const unsigned char *packet, celt_uint32_t size, CELTHeader *header)

6.27.1 Function Documentation

6.27.1.1 int EXPORT celt_header_from_packet (const unsigned char * *packet*, celt_uint32_t *size*, CELTHeader * *header*)

Definition at line 106 of file header.c.

References CELT_COPY.

6.27.1.2 void EXPORT celt_header_init (CELTHeader * *header*, const CELTMode * *m*)

Creates a basic header struct

Definition at line 65 of file header.c.

References CELTHeader::bytes_per_packet, CELT_COPY, CELT_GET_BITSTREAM_VERSION, celt_mode_info(), CELTHeader::codec_id, CELTHeader::codec_version, CELTHeader::extra_headers, CELTHeader::frame_size, CELTMode::Fs, CELTHeader::header_size, CELTMode::mdctSize, CELTHeader::nb_channels, CELTMode::nbChannels, CELTMode::overlap, CELTHeader::overlap, CELTHeader::sample_rate, and CELTHeader::version_id.

6.27.1.3 int EXPORT celt_header_to_packet (const CELTHeader * *header*, unsigned char * *packet*, celt_uint32_t *size*)

Definition at line 80 of file header.c.

References CELT_BAD_ARG, CELT_COPY, and CELT_MEMSET.

6.28 libcelt/kfft_double.h File Reference

```
#include "kiss_fft.h"
#include "_kiss_fft_guts.h"
```

Defines

- #define cpx32_fft_alloc(length) kiss_fft_alloc(length, 0, 0);
- #define cpx32_fft_free(state) kiss_fft_free(state)
- #define cpx32_fft(state, X, Y, nx) kiss_fft(state,(const kiss_fft_cpx *)(X), (kiss_fft_cpx *)(Y))
- #define cpx32_ifft(state, X, Y, nx) kiss_ifft(state,(const kiss_fft_cpx *)(X), (kiss_fft_cpx *)(Y))

6.28.1 Define Documentation

6.28.1.1 #define cpx32_fft(state, X, Y, nx) kiss_fft(state,(const kiss_fft_cpx *)(X), (kiss_fft_cpx *)(Y))

Definition at line 63 of file kfft_double.h.

Referenced by mdct_forward().

6.28.1.2 #define cpx32_fft_alloc(length) kiss_fft_alloc(length, 0, 0);

Definition at line 61 of file kfft_double.h.

Referenced by mdct_init().

6.28.1.3 #define cpx32_fft_free(state) kiss_fft_free(state)

Definition at line 62 of file kfft_double.h.

Referenced by mdct_clear().

6.28.1.4 #define cpx32_ifft(state, X, Y, nx) kiss_ifft(state,(const kiss_fft_cpx *)(X), (kiss_fft_cpx *)(Y))

Definition at line 64 of file kfft_double.h.

Referenced by mdct_backward().

6.29 libcelt/kfft_single.c File Reference

6.30 libcelt/kfft_single.h File Reference

```
#include "kiss_fft.h"
#include "kiss_fftr.h"
#include "_kiss_fft_guts.h"
```

Defines

- #define real16_fft_alloc(length) kiss_fftr_alloc_celt_single(length, 0, 0);
- #define real16_fft_free(state) kiss_fft_free(state)
- #define real16_fft_inplace(state, X, nx) kiss_fftr_inplace(state,X)
- #define BITREV(state, i) ((state) → substate → bitrev[i])
- #define real16_ifft(state, X, Y, nx) kiss_fftri(state,X, Y)

6.30.1 Define Documentation

6.30.1.1 #define BITREV(state, i) ((state) → substate → bitrev[i])

Definition at line 79 of file kfft_single.h.

Referenced by find_spectral_pitch().

6.30.1.2 #define real16_fft_alloc(length) kiss_fftr_alloc_celt_single(length, 0, 0);

Definition at line 76 of file kfft_single.h.

Referenced by pitch_state_alloc().

6.30.1.3 #define real16_fft_free(state) kiss_fft_free(state)

Definition at line 77 of file kfft_single.h.

Referenced by pitch_state_free().

6.30.1.4 #define real16_fft_inplace(state, X, nx) kiss_fftr_inplace(state,X)

Definition at line 78 of file kfft_single.h.

Referenced by find_spectral_pitch().

6.30.1.5 #define real16_ifft(state, X, Y, nx) kiss_fftri(state,X, Y)

Definition at line 80 of file kfft_single.h.

Referenced by find_spectral_pitch().

6.31 libcelt/kiss_fft.c File Reference

```
#include "_kiss_fft_guts.h"
#include "arch.h"
#include "os_support.h"
#include "mathops.h"
```

Functions

- void kf_work (kiss_fft_cpx *Fout, const kiss_fft_cpx *f, const size_t fstride, int in_stride, int *factors, const kiss_fft_cfg st, int N, int s2, int m2)
- void ki_work (kiss_fft_cpx *Fout, const kiss_fft_cpx *f, const size_t fstride, int in_stride, int *factors, const kiss_fft_cfg st, int N, int s2, int m2)
- kiss_fft_cfg kiss_fft_alloc (int nfft, void *mem, size_t *lenmem)
- void kiss_fft_stride (kiss_fft_cfg st, const kiss_fft_cpx *fin, kiss_fft_cpx *fout, int in_stride)
- void kiss_fft (kiss_fft_cfg cfg, const kiss_fft_cpx *fin, kiss_fft_cpx *fout)
- void kiss_ifft_stride (kiss_fft_cfg st, const kiss_fft_cpx *fin, kiss_fft_cpx *fout, int in_stride)
- void kiss_ifft (kiss_fft_cfg cfg, const kiss_fft_cpx *fin, kiss_fft_cpx *fout)

6.31.1 Function Documentation

6.31.1.1 void kf_work (kiss_fft_cpx * *Fout*, const kiss_fft_cpx * *f*, const size_t *fstride*, int *in_stride*, int * *factors*, const kiss_fft_cfg *st*, int *N*, int *s2*, int *m2*)

Definition at line 517 of file kiss_fft.c.

6.31.1.2 void ki_work (kiss_fft_cpx * *Fout*, const kiss_fft_cpx * *f*, const size_t *fstride*, int *in_stride*, int * *factors*, const kiss_fft_cfg *st*, int *N*, int *s2*, int *m2*)

Internal function. Can be useful when you want to do the bit-reversing yourself

Definition at line 553 of file kiss_fft.c.

6.31.1.3 void kiss_fft (kiss_fft_cfg *cfg*, const kiss_fft_cpx * *fin*, kiss_fft_cpx * *fout*)

kiss_fft(cfg,in_out_buf)

Perform an FFT on a complex input buffer. for a forward FFT, fin should be f[0] , f[1] , ... ,f[nfft-1] fout will be F[0] , F[1] , ... ,F[nfft-1] Note that each element is complex and can be accessed like f[k].r and f[k].i

Definition at line 683 of file kiss_fft.c.

6.31.1.4 kiss_fft_cfg kiss_fft_alloc (int *nfft*, void * *mem*, size_t * *lenmem*)

kiss_fft_alloc

Initialize a FFT (or IFFT) algorithm's cfg/state buffer.

typical usage: kiss_fft_cfg mycfg=kiss_fft_alloc(1024,0,NULL,NULL);

The return value from fft_alloc is a cfg buffer used internally by the fft routine or NULL.

If lenmem is NULL, then kiss_fft_alloc will allocate a cfg buffer using malloc. The returned value should be free()d when done to avoid memory leaks.

The state can be placed in a user supplied buffer 'mem': If lenmem is not NULL and mem is not NULL and ∗lenmem is large enough, then the function places the cfg in mem and the size used in ∗lenmem and returns mem.

If lenmem is not NULL and (mem is NULL or ∗lenmem is not large enough), then the function returns NULL and places the minimum cfg buffer size in ∗lenmem.

Definition at line 620 of file kiss_fft.c.

6.31.1.5 void kiss_fft_stride (kiss_fft_cfg *cfg*, const kiss_fft_cpx ∗ *fin*, kiss_fft_cpx ∗ *fout*, int *fin_stride*)

A more generic version of the above function. It reads its input from every Nth sample.

Definition at line 663 of file kiss_fft.c.

6.31.1.6 void kiss_ifft (kiss_fft_cfg *cfg*, const kiss_fft_cpx ∗ *fin*, kiss_fft_cpx ∗ *fout*)

Definition at line 702 of file kiss_fft.c.

6.31.1.7 void kiss_ifft_stride (kiss_fft_cfg *st*, const kiss_fft_cpx ∗ *fin*, kiss_fft_cpx ∗ *fout*, int *in_stride*)

Definition at line 688 of file kiss_fft.c.

6.32 libcelt/kiss_fft.h File Reference

```
#include <stdlib.h>
#include <math.h>
#include "arch.h"
```

Data Structures

- struct kiss_fft_cpx
- struct kiss_twiddle_cpx

Defines

- #define KISS_FFT_MALLOC celt_alloc
- #define kiss_fft_scalar float
- #define kiss_twiddle_scalar float
- #define KF_SUFFIX _celt_single
- #define CAT_SUFFIX(a, b) a ## b
- #define SUF(a, b) CAT_SUFFIX(a, b)
- #define kiss_fft_alloc SUF(kiss_fft_alloc,KF_SUFFIX)
- #define kf_work SUF(kf_work,KF_SUFFIX)
- #define ki_work SUF(ki_work,KF_SUFFIX)
- #define kiss_fft SUF(kiss_fft,KF_SUFFIX)
- #define kiss_ifft SUF(kiss_ifft,KF_SUFFIX)
- #define kiss_fft_stride SUF(kiss_fft_stride,KF_SUFFIX)
- #define kiss_ifft_stride SUF(kiss_ifft_stride,KF_SUFFIX)
- #define kiss_fft_free celt_free

Typedefs

- typedef struct kiss_fft_state * kiss_fft_cfg

Functions

- kiss_fft_cfg kiss_fft_alloc (int nfft, void *mem, size_t *lenmem)
- void kf_work (kiss_fft_cpx *Fout, const kiss_fft_cpx *f, const size_t fstride, int in_stride, int *factors, const kiss_fft_cfg st, int N, int s2, int m2)
- void ki_work (kiss_fft_cpx *Fout, const kiss_fft_cpx *f, const size_t fstride, int in_stride, int *factors, const kiss_fft_cfg st, int N, int s2, int m2)
- void kiss_fft (kiss_fft_cfg cfg, const kiss_fft_cpx *fin, kiss_fft_cpx *fout)
- void kiss_ifft (kiss_fft_cfg cfg, const kiss_fft_cpx *fin, kiss_fft_cpx *fout)
- void kiss_fft_stride (kiss_fft_cfg cfg, const kiss_fft_cpx *fin, kiss_fft_cpx *fout, int fin_stride)
- void kiss_ifft_stride (kiss_fft_cfg cfg, const kiss_fft_cpx *fin, kiss_fft_cpx *fout, int fin_stride)

6.32.1 Define Documentation

6.32.1.1 #define CAT_SUFFIX(a, b) a ## b

Definition at line 57 of file kiss_fft.h.

6.32.1.2 #define KF_SUFFIX _celt_single

Definition at line 50 of file kiss_fft.h.

6.32.1.3 #define kf_work SUF(kf_work,KF_SUFFIX)

Definition at line 61 of file kiss_fft.h.

Referenced by kf_work(), kiss_fft_stride(), and kiss_fftr_inplace().

6.32.1.4 #define ki_work SUF(ki_work,KF_SUFFIX)

Definition at line 62 of file kiss_fft.h.

Referenced by ki_work(), kiss_fftri(), and kiss_ifft_stride().

6.32.1.5 #define kiss_fft SUF(kiss_fft,KF_SUFFIX)

Definition at line 63 of file kiss_fft.h.

Referenced by kiss_fftr().

6.32.1.6 #define kiss_fft_alloc SUF(kiss_fft_alloc,KF_SUFFIX)

Definition at line 60 of file kiss_fft.h.

Referenced by kiss_fftr_alloc().

6.32.1.7 #define kiss_fft_free celt_free

If kiss_fft_alloc allocated a buffer, it is one contiguous buffer and can be simply free()d when no longer needed

Definition at line 134 of file kiss_fft.h.

6.32.1.8 #define KISS_FFT_MALLOC celt_alloc

Definition at line 30 of file kiss_fft.h.

Referenced by kiss_fft_alloc(), and kiss_fftr_alloc().

6.32.1.9 #define kiss_fft_scalar float

Definition at line 48 of file kiss_fft.h.

Referenced by mdct_backward(), and mdct_forward().

6.32.1.10 #define kiss_fft_stride SUF(kiss_fft_stride,KF_SUFFIX)

Definition at line 65 of file kiss_fft.h.

Referenced by kiss_fft().

6.32.1.11 #define kiss_ifft SUF(kiss_ifft,KF_SUFFIX)

Definition at line 64 of file kiss_fft.h.

6.32.1.12 #define kiss_ifft_stride SUF(kiss_ifft_stride,KF_SUFFIX)

Definition at line 66 of file kiss_fft.h.

Referenced by kiss_ifft().

6.32.1.13 #define kiss_twiddle_scalar float

Definition at line 49 of file kiss_fft.h.

Referenced by mdct_init().

6.32.1.14 #define SUF(a, b) CAT_SUFFIX(a, b)

Definition at line 58 of file kiss_fft.h.

6.32.2 Typedef Documentation

6.32.2.1 typedef struct kiss_fft_state* kiss_fft_cfg

Definition at line 79 of file kiss_fft.h.

6.32.3 Function Documentation

6.32.3.1 void kf_work (kiss_fft_cpx * *Fout*, const kiss_fft_cpx * *f*, const size_t *fstride*, int *in_stride*, int * *factors*, const kiss_fft_cfg *st*, int *N*, int *s2*, int *m2*)

Definition at line 517 of file kiss_fft.c.

References celt_fatal, and kf_work.

6.32.3.2 void ki_work (kiss_fft_cpx * *Fout*, const kiss_fft_cpx * *f*, const size_t *fstride*, int *in_stride*, int * *factors*, const kiss_fft_cfg *st*, int *N*, int *s2*, int *m2*)

Internal function. Can be useful when you want to do the bit-reversing yourself

Definition at line 553 of file kiss_fft.c.

References celt_fatal, and ki_work.

6.32.3.3 void kiss_fft (kiss_fft_cfg *cfg*, const kiss_fft_cpx * *fin*, kiss_fft_cpx * *fout*)

kiss_fft(cfg,in_out_buf)

Perform an FFT on a complex input buffer. for a forward FFT, fin should be f[0] , f[1] , ... ,f[nfft-1] fout will be F[0] , F[1] , ... ,F[nfft-1] Note that each element is complex and can be accessed like f[k].r and f[k].i

Definition at line 683 of file kiss_fft.c.

References kiss_fft_stride.

6.32.3.4 kiss_fft_cfg kiss_fft_alloc (int *nfft*, void * *mem*, size_t * *lenmem*)

kiss_fft_alloc

Initialize a FFT (or IFFT) algorithm's cfg/state buffer.

typical usage: kiss_fft_cfg mycfg=kiss_fft_alloc(1024,0,NULL,NULL);

The return value from fft_alloc is a cfg buffer used internally by the fft routine or NULL.

If lenmem is NULL, then kiss_fft_alloc will allocate a cfg buffer using malloc. The returned value should be free()d when done to avoid memory leaks.

The state can be placed in a user supplied buffer 'mem': If lenmem is not NULL and mem is not NULL and *lenmem is large enough, then the function places the cfg in mem and the size used in *lenmem and returns mem.

If lenmem is not NULL and (mem is NULL or *lenmem is not large enough), then the function returns NULL and places the minimum cfg buffer size in *lenmem.

Definition at line 620 of file kiss_fft.c.

References kiss_fft_state::bitrev, DIV32, kiss_fft_state::factors, kf_cexp, kf_cexp2, KISS_FFT_MALLOC, kiss_fft_state::nfft, kiss_fft_state::scale, SHL32, and kiss_fft_state::twiddles.

6.32.3.5 void kiss_fft_stride (kiss_fft_cfg *cfg*, const kiss_fft_cpx * *fin*, kiss_fft_cpx * *fout*, int *fin_stride*)

A more generic version of the above function. It reads its input from every Nth sample.

Definition at line 663 of file kiss_fft.c.

References kiss_fft_state::bitrev, celt_fatal, kiss_fft_state::factors, kiss_fft_cpx::i, kf_work, kiss_fft_state::nfft, kiss_fft_cpx::r, and kiss_fft_state::scale.

6.32.3.6 void kiss_ifft (kiss_fft_cfg *cfg*, const kiss_fft_cpx * *fin*, kiss_fft_cpx * *fout*)

Definition at line 702 of file kiss_fft.c.

References kiss_ifft_stride.

6.32.3.7 void kiss_ifft_stride (kiss_fft_cfg *cfg*, const kiss_fft_cpx * *fin*, kiss_fft_cpx * *fout*, int *fin_stride*)

Definition at line 688 of file kiss_fft.c.

References kiss_fft_state::bitrev, celt_fatal, kiss_fft_state::factors, ki_work, and kiss_fft_state::nfft.

6.33 libcelt/kiss_fftr.c File Reference

```
#include "os_support.h"
#include "mathops.h"
#include "kiss_fftr.h"
#include "_kiss_fft_guts.h"
```

Functions

- kiss_fftr_cfg kiss_fftr_alloc (int nfft, void *mem, size_t *lenmem)
- void kiss_fftr_twiddles (kiss_fftr_cfg st, kiss_fft_scalar *freqdata)
- void kiss_fftr (kiss_fftr_cfg st, const kiss_fft_scalar *timedata, kiss_fft_scalar *freqdata)
- void kiss_fftr_inplace (kiss_fftr_cfg st, kiss_fft_scalar *X)
- void kiss_fftri (kiss_fftr_cfg st, const kiss_fft_scalar *freqdata, kiss_fft_scalar *timedata)

6.33.1 Function Documentation

6.33.1.1 void kiss_fftr (kiss_fftr_cfg *st*, const kiss_fft_scalar * *timedata*, kiss_fft_scalar * *freqdata*)

Definition at line 123 of file kiss_fftr.c.

6.33.1.2 kiss_fftr_cfg kiss_fftr_alloc (int *nfft*, void * *mem*, size_t * *lenmem*)

Definition at line 31 of file kiss_fftr.c.

6.33.1.3 void kiss_fftr_inplace (kiss_fftr_cfg *st*, kiss_fft_scalar * *X*)

Definition at line 131 of file kiss_fftr.c.

6.33.1.4 void kiss_fftr_twiddles (kiss_fftr_cfg *st*, kiss_fft_scalar * *freqdata*)

Definition at line 79 of file kiss_fftr.c.

6.33.1.5 void kiss_fftri (kiss_fftr_cfg *st*, const kiss_fft_scalar * *freqdata*, kiss_fft_scalar * *timedata*)

Definition at line 137 of file kiss_fftr.c.

6.34 libcelt/kiss_fftr.h File Reference

```
#include "kiss_fft.h"
```

Data Structures

- struct kiss_fftr_state

Defines

- #define kiss_fftr_alloc SUF(kiss_fftr_alloc,KF_SUFFIX)
- #define kiss_fftr_alloc SUF(kiss_fftr_alloc,KF_SUFFIX)
- #define kiss_fftr_twiddles SUF(kiss_fftr_twiddles,KF_SUFFIX)
- #define kiss_fftr SUF(kiss_fftr,KF_SUFFIX)
- #define kiss_fftri SUF(kiss_fftri,KF_SUFFIX)
- #define kiss_fftr_free speex_free

Typedefs

- typedef struct kiss_fftr_state * kiss_fftr_cfg

Functions

- kiss_fftr_cfg kiss_fftr_alloc (int nfft, void *mem, size_t *lenmem)
- void kiss_fftr_twiddles (kiss_fftr_cfg st, kiss_fft_scalar *freqdata)
- void kiss_fftr (kiss_fftr_cfg st, const kiss_fft_scalar *timedata, kiss_fft_scalar *freqdata)
- void kiss_fftr_inplace (kiss_fftr_cfg st, kiss_fft_scalar *X)
- void kiss_fftri (kiss_fftr_cfg st, const kiss_fft_scalar *freqdata, kiss_fft_scalar *timedata)

6.34.1 Define Documentation

6.34.1.1 #define kiss_fftr SUF(kiss_fftr,KF_SUFFIX)

Definition at line 13 of file kiss_fftr.h.

6.34.1.2 #define kiss_fftr_alloc SUF(kiss_fftr_alloc,KF_SUFFIX)

Definition at line 11 of file kiss_fftr.h.

6.34.1.3 #define kiss_fftr_alloc SUF(kiss_fftr_alloc,KF_SUFFIX)

Definition at line 11 of file kiss_fftr.h.

6.34.1.4 #define kiss_fftr_free speex_free

Definition at line 59 of file kiss_fftr.h.

6.34.1.5 #define kiss_fftr_twiddles SUF(kiss_fftr_twiddles,KF_SUFFIX)

Definition at line 12 of file kiss_fftr.h.

Referenced by kiss_fftr(), and kiss_fftr_inplace().

6.34.1.6 #define kiss_fftri SUF(kiss_fftri,KF_SUFFIX)

Definition at line 14 of file kiss_fftr.h.

6.34.2 Typedef Documentation

6.34.2.1 typedef struct kiss_fftr_state* kiss_fftr_cfg

Definition at line 32 of file kiss_fftr.h.

6.34.3 Function Documentation

6.34.3.1 void kiss_fftr (kiss_fftr_cfg *st*, const kiss_fft_scalar * *timedata*, kiss_fft_scalar * *freqdata*)

Definition at line 123 of file kiss_fftr.c.

References kiss_fft, kiss_fftr_twiddles, and kiss_fftr_state::substate.

6.34.3.2 kiss_fftr_cfg kiss_fftr_alloc (int *nfft*, void * *mem*, size_t * *lenmem*)

Definition at line 31 of file kiss_fftr.c.

References DIV32, kf_cexp, kf_cexp2, kiss_fft_alloc, KISS_FFT_MALLOC, kiss_fft_state::scale, SHL32, kiss_fftr_state::substate, and kiss_fftr_state::super_twiddles.

6.34.3.3 void kiss_fftr_inplace (kiss_fftr_cfg *st*, kiss_fft_scalar * *X*)

Definition at line 131 of file kiss_fftr.c.

References kiss_fft_state::factors, kf_work, kiss_fftr_twiddles, and kiss_fftr_state::substate.

6.34.3.4 void kiss_fftr_twiddles (kiss_fftr_cfg *st*, kiss_fft_scalar * *freqdata*)

Definition at line 79 of file kiss_fftr.c.

References ADD32, C_FIXDIV, C_MULC, CHECK_OVERFLOW_OP, EXT32, HALF_OF, kiss_fft_cpx::i, k, kiss_fft_state::nfft, PSHR32, kiss_fft_cpx::r, SHR32, SUB32, kiss_fftr_state::substate, and kiss_fftr_state::super_twiddles.

6.34.3.5 void kiss_fftri (kiss_fftr_cfg *st*, const kiss_fft_scalar * *freqdata*, kiss_fft_scalar * *timedata*)

Definition at line 137 of file kiss_fftr.c.

References kiss_fft_state::bitrev, C_ADD, C_MUL, C_SUB, kiss_fft_state::factors, kiss_fft_cpx::i, k, kiss_fft_state::nfft, kiss_fft_cpx::r, kiss_fftr_state::substate, and kiss_fftr_state::super_twiddles.

6.35 libcelt/laplace.c File Reference

```
#include "laplace.h"
```

Functions

- int ec_laplace_get_start_freq (int decay)
- void ec_laplace_encode_start (ec_enc *enc, int value, int decay, int fs)
- void ec_laplace_encode (ec_enc *enc, int value, int decay)
- int ec_laplace_decode_start (ec_dec *dec, int decay, int fs)
- int ec_laplace_decode (ec_dec *dec, int decay)

6.35.1 Function Documentation

6.35.1.1 int ec_laplace_decode (ec_dec * *dec*, int *decay*)

Decode a value that is assumed to be the realisation of a Laplace-distributed random process

Parameters:

dec Entropy decoder state

decay Probability of the value +/- 1, multiplied by 16384

Returns:

Value decoded

Definition at line 121 of file laplace.c.

6.35.1.2 int ec_laplace_decode_start (ec_dec * *dec*, int *decay*, int *fs*)

Definition at line 85 of file laplace.c.

Referenced by ec_laplace_decode().

6.35.1.3 void ec_laplace_encode (ec_enc * *enc*, int *value*, int *decay*)

Encode a value that is assumed to be the realisation of a Laplace-distributed random process

Parameters:

enc Entropy encoder state

value Value to encode

decay Probability of the value +/- 1, multiplied by 16384

Definition at line 78 of file laplace.c.

6.35.1.4 void ec_laplace_encode_start (ec_enc * *enc*, int *value*, int *decay*, int *fs*)

Definition at line 43 of file laplace.c.

Referenced by ec_laplace_encode().

6.35.1.5 int ec_laplace_get_start_freq (int *decay*)

Definition at line 38 of file laplace.c.

Referenced by ec_laplace_decode(), ec_laplace_encode(), and quant_prob_alloc().

6.36 libcelt/laplace.h File Reference

```
#include "entenc.h"
#include "entdec.h"
```

Functions

- int ec_laplace_get_start_freq (int decay)
- void ec_laplace_encode (ec_enc *enc, int value, int decay)
- void ec_laplace_encode_start (ec_enc *enc, int value, int decay, int fs)
- int ec_laplace_decode (ec_dec *dec, int decay)
- int ec_laplace_decode_start (ec_dec *dec, int decay, int fs)

6.36.1 Function Documentation

6.36.1.1 int ec_laplace_decode (ec_dec * *dec*, int *decay*)

Decode a value that is assumed to be the realisation of a Laplace-distributed random process

Parameters:

dec Entropy decoder state

decay Probability of the value +/- 1, multiplied by 16384

Returns:

Value decoded

Definition at line 121 of file laplace.c.

References ec_laplace_decode_start(), and ec_laplace_get_start_freq().

6.36.1.2 int ec_laplace_decode_start (ec_dec * *dec*, int *decay*, int *fs*)

Definition at line 85 of file laplace.c.

References ec_dec_update(), and ec_decode().

6.36.1.3 void ec_laplace_encode (ec_enc * *enc*, int *value*, int *decay*)

Encode a value that is assumed to be the realisation of a Laplace-distributed random process

Parameters:

enc Entropy encoder state

value Value to encode

decay Probability of the value +/- 1, multiplied by 16384

Definition at line 78 of file laplace.c.

References ec_laplace_encode_start(), and ec_laplace_get_start_freq().

6.36.1.4 void ec_laplace_encode_start (ec_enc * *enc*, int *value*, int *decay*, int *fs*)

Definition at line 43 of file laplace.c.

References ec_encode().

6.36.1.5 int ec_laplace_get_start_freq (int *decay*)

Definition at line 38 of file laplace.c.

6.37 libcelt/mathops.h File Reference

Various math functions.

```
#include "arch.h"
#include "entcode.h"
#include "os_support.h"
```

Defines

- #define celt_sqrt(x) ((float)sqrt(x))
- #define celt_psqrt(x) ((float)sqrt(x))
- #define celt_rsqrt(x) (1.f/celt_sqrt(x))
- #define celt_acos acos
- #define celt_exp exp
- #define celt_cos_norm(x) (cos((.5f∗M_PI)∗(x)))
- #define celt_atan atan
- #define celt_rcp(x) (1.f/(x))
- #define celt_div(a, b) ((a)/(b))

6.37.1 Detailed Description

Various math functions.

Definition in file mathops.h.

6.37.2 Define Documentation

6.37.2.1 #define celt_acos acos

Definition at line 91 of file mathops.h.

6.37.2.2 #define celt_atan atan

Definition at line 94 of file mathops.h.

6.37.2.3 #define celt_cos_norm(x) (cos((.5f∗M_PI)∗(x)))

Definition at line 93 of file mathops.h.

Referenced by mdct_init().

6.37.2.4 #define celt_div(a, b) ((a)/(b))

Definition at line 96 of file mathops.h.

Referenced by compute_pitch_gain(), intra_prediction(), and intra_unquant().

6.37.2.5 #define celt_exp exp

Definition at line 92 of file mathops.h.

6.37.2.6 #define celt_psqrt(x) ((float)sqrt(x))

Definition at line 89 of file mathops.h.

Referenced by alg_quant().

6.37.2.7 #define celt_rcp(x) (1.f/(x))

Definition at line 95 of file mathops.h.

Referenced by intra_fold(), intra_prediction(), and intra_unquant().

6.37.2.8 #define celt_rsqrt(x) (1.f/celt_sqrt(x))

Definition at line 90 of file mathops.h.

Referenced by find_spectral_pitch().

6.37.2.9 #define celt_sqrt(x) ((float)sqrt(x))

Definition at line 88 of file mathops.h.

Referenced by intra_fold(), intra_prediction(), intra_unquant(), quant_bands(), quant_pitch(), stereo_mix(), and unquant_bands().

6.38 libcelt/mdct.c File Reference

```
#include "mdct.h"
#include "kfft_double.h"
#include <math.h>
#include "os_support.h"
#include "mathops.h"
#include "stack_alloc.h"
```

Defines

- #define M_PI 3.141592653

Functions

- void mdct_init (mdct_lookup *l, int N)
- void mdct_clear (mdct_lookup *l)
- void mdct_forward (const mdct_lookup *l, kiss_fft_scalar *in, kiss_fft_scalar *restrict out, const celt_word16_t *window, int overlap)
- void mdct_backward (const mdct_lookup *l, kiss_fft_scalar *in, kiss_fft_scalar *restrict out, const celt_word16_t *restrict window, int overlap)

6.38.1 Define Documentation

6.38.1.1 #define M_PI 3.141592653

Definition at line 57 of file mdct.c.

Referenced by celt_mode_create(), and mdct_init().

6.38.2 Function Documentation

6.38.2.1 void mdct_backward (const mdct_lookup * *l*, kiss_fft_scalar * *in*, kiss_fft_scalar *restrict *out*, const celt_word16_t *restrict *window*, int *overlap*)

Definition at line 180 of file mdct.c.

References ALLOC, cpx32_ifft, mdct_lookup::kfft, kiss_fft_scalar, MULT16_32_Q15, mdct_lookup::n, N, RESTORE_STACK, S_MUL, SAVE_STACK, mdct_lookup::trig, and VARDECL.

6.38.2.2 void mdct_clear (mdct_lookup * *l*)

Definition at line 83 of file mdct.c.

Referenced by celt_mode_destroy().

6.38.2.3 void mdct_forward (const mdct_lookup * *l*, kiss_fft_scalar * *in*, kiss_fft_scalar *restrict *out*, const celt_word16_t * *window*, int *overlap*)

Definition at line 89 of file mdct.c.

References ALLOC, cpx32_fft, mdct_lookup::kfft, kiss_fft_scalar, MULT16_32_Q15, mdct_lookup::n, N, RESTORE_STACK, S_MUL, SAVE_STACK, mdct_lookup::trig, and VARDECL.

6.38.2.4 void mdct_init (mdct_lookup * *l*, int *N*)

Definition at line 60 of file mdct.c.

Referenced by celt_mode_create().

6.39 libcelt/mdct.h File Reference

```
#include "kiss_fft.h"
#include "arch.h"
```

Data Structures

- struct mdct_lookup

Functions

- void mdct_init (mdct_lookup *l, int N)
- void mdct_clear (mdct_lookup *l)
- void mdct_forward (const mdct_lookup *l, kiss_fft_scalar *in, kiss_fft_scalar *out, const celt_word16_t *window, int overlap)
- void mdct_backward (const mdct_lookup *l, kiss_fft_scalar *in, kiss_fft_scalar *out, const celt_word16_t *restrict window, int overlap)

6.39.1 Function Documentation

6.39.1.1 void mdct_backward (const mdct_lookup * *l*, kiss_fft_scalar * *in*, kiss_fft_scalar * *out*, const celt_word16_t *restrict *window*, int *overlap*)

Compute a backward MDCT (no scaling) and performs weighted overlap-add (scales implicitly by 1/2)

6.39.1.2 void mdct_clear (mdct_lookup * *l*)

Definition at line 83 of file mdct.c.

References cpx32_fft_free, mdct_lookup::kfft, and mdct_lookup::trig.

6.39.1.3 void mdct_forward (const mdct_lookup * *l*, kiss_fft_scalar * *in*, kiss_fft_scalar * *out*, const celt_word16_t * *window*, int *overlap*)

Compute a forward MDCT and scale by 4/N

6.39.1.4 void mdct_init (mdct_lookup * *l*, int *N*)

Definition at line 60 of file mdct.c.

References ADD32, celt_cos_norm, cpx32_fft_alloc, DIV32, EXTEND32, mdct_lookup::kfft, kiss_twiddle_scalar, M_PI, mdct_lookup::n, SHL32, and mdct_lookup::trig.

6.40 libcelt/mfrngcod.h File Reference

```
#include "entcode.h"
```

Defines

- #define _mfrngcode_H (1)
- #define EC_SYM_BITS (8)
- #define EC_CODE_BITS (32)
- #define EC_SYM_MAX ((1U<<EC_SYM_BITS)-1)
- #define EC_CODE_SHIFT (EC_CODE_BITS-EC_SYM_BITS-1)
- #define EC_CODE_TOP (((ec_uint32)1U)<<EC_CODE_BITS-1)
- #define EC_CODE_BOT (EC_CODE_TOP>>EC_SYM_BITS)
- #define EC_CODE_CARRY (((ec_uint32)EC_SYM_MAX)<<EC_CODE_SHIFT)
- #define EC_CODE_EXTRA ((EC_CODE_BITS-2)%EC_SYM_BITS+1)
- #define EC_CODE_MASK ((((ec_uint32)1U)<<EC_CODE_BITS-1)-1<<1|1)
- #define EC_FOF_RSV1 (1<<EC_SYM_BITS-1)
- #define EC_FOF_RSV1_MASK (EC_FOF_RSV1-1)

6.40.1 Define Documentation

6.40.1.1 #define _mfrngcode_H (1)

Definition at line 2 of file mfrngcod.h.

6.40.1.2 #define EC_CODE_BITS (32)

Definition at line 10 of file mfrngcod.h.

Referenced by ec_dec_tell(), and ec_enc_tell().

6.40.1.3 #define EC_CODE_BOT (EC_CODE_TOP>>EC_SYM_BITS)

Definition at line 18 of file mfrngcod.h.

6.40.1.4 #define EC_CODE_CARRY (((ec_uint32)EC_SYM_MAX)<<EC_CODE_SHIFT)

Definition at line 20 of file mfrngcod.h.

6.40.1.5 #define EC_CODE_EXTRA ((EC_CODE_BITS-2)%EC_SYM_BITS+1)

Definition at line 22 of file mfrngcod.h.

Referenced by ec_dec_init().

6.40.1.6 #define EC_CODE_MASK ((((ec_uint32)1U)<<EC_CODE_BITS-1)-1<<1|1)

Definition at line 27 of file mfrngcod.h.

6.40.1.7 #define EC_CODE_SHIFT (EC_CODE_BITS-EC_SYM_BITS-1)

Definition at line 14 of file mfrngcod.h.

Referenced by ec_enc_done().

6.40.1.8 #define EC_CODE_TOP (((ec_uint32)1U)<<EC_CODE_BITS-1)

Definition at line 16 of file mfrngcod.h.

Referenced by ec_enc_done(), and ec_enc_init().

6.40.1.9 #define EC_FOF_RSV1 (1<<EC_SYM_BITS-1)

Definition at line 32 of file mfrngcod.h.

6.40.1.10 #define EC_FOF_RSV1_MASK (EC_FOF_RSV1-1)

Definition at line 34 of file mfrngcod.h.

6.40.1.11 #define EC_SYM_BITS (8)

Definition at line 8 of file mfrngcod.h.

Referenced by ec_dec_init(), and ec_dec_tell().

6.40.1.12 #define EC_SYM_MAX ((1U<<EC_SYM_BITS)-1)

Definition at line 12 of file mfrngcod.h.

6.41 libcelt/modes.c File Reference

```
#include "celt.h"
#include "modes.h"
#include "rate.h"
#include "os_support.h"
#include "stack_alloc.h"
#include "quant_bands.h"
```

Defines

- #define MODEVALID 0xa110ca7e
- #define MODEFREED 0xb10cf8ee
- #define M_PI 3.141592653
- #define PBANDS 8
- #define MIN_BINS 3
- #define BARK_BANDS 25
- #define BITALLOC_SIZE 10

Functions

- int EXPORT celt_mode_info (const CELTMode *mode, int request, celt_int32_t *value)
- CELTMode EXPORT * celt_mode_create (celt_int32_t Fs, int channels, int frame_size, int lookahead, int *error)
- void EXPORT celt_mode_destroy (CELTMode *mode)
- int check_mode (const CELTMode *mode)

6.41.1 Define Documentation

6.41.1.1 #define BARK_BANDS 25

Definition at line 89 of file modes.c.

6.41.1.2 #define BITALLOC_SIZE 10

Definition at line 107 of file modes.c.

6.41.1.3 #define M_PI 3.141592653

Definition at line 51 of file modes.c.

6.41.1.4 #define MIN_BINS 3

Definition at line 84 of file modes.c.

Referenced by celt_mode_create().

6.41.1.5 #define MODEFREED 0xb10cf8ee

Definition at line 48 of file modes.c.

Referenced by celt_mode_destroy(), and check_mode().

6.41.1.6 #define MODEVALID 0xa110ca7e

Definition at line 47 of file modes.c.

Referenced by celt_mode_create(), and check_mode().

6.41.1.7 #define PBANDS 8

Definition at line 79 of file modes.c.

6.41.2 Function Documentation

6.41.2.1 int check_mode (const CELTMode * *mode*)

Definition at line 422 of file modes.c.

Referenced by celt_decode(), celt_decoder_create(), celt_decoder_destroy(), celt_encode(), celt_encoder_create(), celt_encoder_destroy(), and celt_mode_destroy().

6.42 libcelt/modes.h File Reference

```
#include "celt_types.h"
#include "celt.h"
#include "arch.h"
#include "mdct.h"
#include "psy.h"
#include "pitch.h"
```

Data Structures

- struct CELTMode
  Mode definition.

Defines

- #define CELT_BITSTREAM_VERSION 0x80000002
- #define MAX_PERIOD 1024
- #define CHANNELS(mode) ((mode) → nbChannels)
- #define MDCT(mode) (&(mode) → mdct)
- #define OVERLAP(mode) ((mode) → overlap)
- #define FRAMESIZE(mode) ((mode) → mdctSize)

Functions

- int check_mode (const CELTMode *mode)

6.42.1 Define Documentation

6.42.1.1 #define CELT_BITSTREAM_VERSION 0x80000002

Definition at line 42 of file modes.h.

Referenced by celt_mode_info().

6.42.1.2 #define CHANNELS(mode) ((mode) → nbChannels)

Definition at line 54 of file modes.h.

Referenced by celt_decode(), celt_decoder_create(), celt_encode(), compute_band_energies(), compute_pitch_gain(), denormalise_bands(), find_spectral_pitch(), intra_fold(), intra_prediction(), intra_unquant(), normalise_bands(), pitch_quant_bands(), quant_bands(), stereo_mix(), and unquant_bands().

6.42.1.3 #define FRAMESIZE(mode) ((mode) → mdctSize)

Definition at line 65 of file modes.h.

6.42.1.4 #define MAX_PERIOD 1024

Definition at line 48 of file modes.h.

Referenced by celt_decode(), celt_decoder_create(), celt_encode(), celt_encoder_create(), celt_mode_create(), do_plc(), dump_modes(), and find_spectral_pitch().

6.42.1.5 #define MDCT(mode) (&(mode) → mdct)

Definition at line 58 of file modes.h.

6.42.1.6 #define OVERLAP(mode) ((mode) → overlap)

Definition at line 61 of file modes.h.

Referenced by find_spectral_pitch().

6.42.2 Function Documentation

6.42.2.1 int check_mode (const CELTMode ∗ *mode*)

Definition at line 422 of file modes.c.

References CELT_INVALID_MODE, CELT_OK, CELTMode::marker_end, CELTMode::marker_start, MODEFREED, and MODEVALID.

6.43 libcelt/os_support.h File Reference

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
```

Defines

- #define EXPORT
- #define CELT_COPY(dst, src, n) (memcpy((dst), (src), (n)∗sizeof(∗(dst)) + 0∗((dst)-(src))))
- #define CELT_MOVE(dst, src, n) (memmove((dst), (src), (n)∗sizeof(∗(dst)) + 0∗((dst)-(src))))
- #define CELT_MEMSET(dst, c, n) (memset((dst), (c), (n)∗sizeof(∗(dst))))

6.43.1 Define Documentation

6.43.1.1 #define CELT_COPY(dst, src, n) (memcpy((dst), (src), (n)∗sizeof(∗(dst)) + 0∗((dst)-(src))))

Copy n bytes of memory from src to dst. The 0∗ term provides compile-time type checking

Definition at line 100 of file os_support.h.

Referenced by celt_encode(), celt_header_from_packet(), celt_header_init(), celt_header_to_packet(), and celt_mode_create().

6.43.1.2 #define CELT_MEMSET(dst, c, n) (memset((dst), (c), (n)∗sizeof(∗(dst))))

Set n bytes of memory to value of c, starting at address s

Definition at line 111 of file os_support.h.

Referenced by celt_header_to_packet(), comb2pulse(), find_spectral_pitch(), ncwrs_u32(), and ncwrs_u64().

6.43.1.3 #define CELT_MOVE(dst, src, n) (memmove((dst), (src), (n)∗sizeof(∗(dst)) + 0∗((dst)-(src))))

Copy n bytes of memory from src to dst, allowing overlapping regions. The 0∗ term provides compile-time type checking

Definition at line 106 of file os_support.h.

Referenced by celt_decode(), and celt_encode().

6.43.1.4 #define EXPORT

Definition at line 50 of file os_support.h.

6.44 libcelt/pgain_table.h File Reference

Variables

- celt_uint16_t pgain_table [512]

6.44.1 Variable Documentation

6.44.1.1 celt_uint16_t pgain_table[512]

Definition at line 3 of file pgain_table.h.

Referenced by quant_pitch().

6.45 libcelt/pitch.c File Reference

Pitch analysis.

```
#include "kfft_single.h"
#include "pitch.h"
#include "psy.h"
#include "os_support.h"
#include "mathops.h"
#include "modes.h"
#include "stack_alloc.h"
```

Defines

- #define normalise16(x, len, val)
- #define INPUT_SHIFT 15

Functions

- kiss_fftr_cfg pitch_state_alloc (int max_lag)
- void pitch_state_free (kiss_fftr_cfg st)
- void find_spectral_pitch (const CELTMode *m, kiss_fftr_cfg fft, const struct PsyDecay *decay, const celt_sig_t *restrict x, const celt_sig_t *restrict y, const celt_word16_t *restrict window, int len, int max_pitch, int *pitch)

6.45.1 Detailed Description

Pitch analysis.

Definition in file pitch.c.

6.45.2 Define Documentation

6.45.2.1 #define INPUT_SHIFT 15

Definition at line 105 of file pitch.c.

Referenced by find_spectral_pitch().

6.45.2.2 #define normalise16(x, len, val)

Definition at line 102 of file pitch.c.

Referenced by find_spectral_pitch().

6.45.3 Function Documentation

6.45.3.1 void find_spectral_pitch (const CELTMode * *m*, kiss_fftr_cfg *fft*, const struct PsyDecay * *decay*, const celt_sig_t *restrict *x*, const celt_sig_t *restrict *y*, const celt_word16_t *restrict *window*, int *len*, int *max_pitch*, int * *pitch*)

Definition at line 107 of file pitch.c.

References ADD16, ALLOC, BITREV, celt_ilog2, CELT_MEMSET, celt_rsqrt, CHANNELS, compute_masking(), EPSILON, EXTRACT16, INPUT_SHIFT, MAX_PERIOD, MULT16_16, MULT16_16_16, MULT16_16_Q15, normalise16, OVERLAP, real16_fft_inplace, real16_ifft, RESTORE_STACK, SAVE_STACK, SHR32, SUB16, VARDECL, and X.

Referenced by celt_encode(), and do_plc().

6.45.3.2 kiss_fftr_cfg pitch_state_alloc (int *max_lag*)

Definition at line 53 of file pitch.c.

Referenced by celt_mode_create().

6.45.3.3 void pitch_state_free (kiss_fftr_cfg *st*)

Definition at line 58 of file pitch.c.

Referenced by celt_mode_destroy().

6.46 libcelt/pitch.h File Reference

Pitch analysis.

```
#include "kiss_fftr.h"
#include "psy.h"
#include "modes.h"
```

Functions

- kiss_fftr_cfg pitch_state_alloc (int max_lag)
- void pitch_state_free (kiss_fftr_cfg st)
- void find_spectral_pitch (const CELTMode *m, kiss_fftr_cfg fft, const struct PsyDecay *decay, const celt_sig_t *x, const celt_sig_t *y, const celt_word16_t *window, int len, int max_pitch, int *pitch)

6.46.1 Detailed Description

Pitch analysis.

Definition in file pitch.h.

6.46.2 Function Documentation

6.46.2.1 void find_spectral_pitch (const CELTMode * *m*, kiss_fftr_cfg *fft*, const struct PsyDecay * *decay*, const celt_sig_t * *x*, const celt_sig_t * *y*, const celt_word16_t * *window*, int *len*, int *max_pitch*, int * *pitch*)

Find the optimal delay for the pitch prediction. Computation is done in the frequency domain, both to save time and to make it easier to apply psychoacoustic weighting

6.46.2.2 kiss_fftr_cfg pitch_state_alloc (int *max_lag*)

Definition at line 53 of file pitch.c.

References real16_fft_alloc.

6.46.2.3 void pitch_state_free (kiss_fftr_cfg *st*)

Definition at line 58 of file pitch.c.

References real16_fft_free.

6.47 libcelt/plc.c File Reference

Defines

- #define LPC_ORDER 12

Functions

- celt_word32_t _celt_lpc (celt_word16_t *lpc, const celt_word16_t *ac, int p)
- void fir_mem16 (const celt_word16_t *x, const celt_word16_t *num, celt_word16_t *y, int N, int ord, celt_word32_t *mem)
- void iir_mem16 (const celt_word16_t *x, const celt_word16_t *den, celt_word16_t *y, int N, int ord, celt_word32_t *mem)
- void _celt_autocorr (const celt_word16_t *x, float *ac, const float *window, int overlap, int lag, int n)
- void do_plc (const CELTMode *m, kiss_fftr_cfg fft, const struct PsyDecay *decay, float *out_mem, float *out, float *time, int len)

6.47.1 Define Documentation

6.47.1.1 #define LPC_ORDER 12

Definition at line 115 of file plc.c.

Referenced by do_plc().

6.47.2 Function Documentation

6.47.2.1 void _celt_autocorr (const celt_word16_t * *x*, float * *ac*, const float * *window*, int *overlap*, int *lag*, int *n*)

Definition at line 85 of file plc.c.

Referenced by do_plc().

6.47.2.2 celt_word32_t _celt_lpc (celt_word16_t * *lpc*, const celt_word16_t * *ac*, int *p*)

Definition at line 5 of file plc.c.

References ADD16, DIV32_16, EXTEND32, MAC16_16_P13, MULT16_16, MULT16_16_Q13, NEG32, PSHR32, r, SHL32, SUB16, and SUB32.

Referenced by do_plc().

6.47.2.3 void do_plc (const CELTMode * *m*, kiss_fftr_cfg *fft*, const struct PsyDecay * *decay*, float * *out_mem*, float * *out*, float * *time*, int *len*)

Definition at line 116 of file plc.c.

References _celt_autocorr(), _celt_lpc(), find_spectral_pitch(), LPC_ORDER, MAX_PERIOD, offset, CELTMode::overlap, and CELTMode::window.

6.47.2.4 void fir_mem16 (const celt_word16_t * *x*, const celt_word16_t * *num*, celt_word16_t * *y*, int *N*, int *ord*, celt_word32_t * *mem*)

Definition at line 49 of file plc.c.

References ADD32, EXTEND32, EXTRACT16, MAC16_16, MULT16_16, PSHR32, and SATURATE.

6.47.2.5 void iir_mem16 (const celt_word16_t * *x*, const celt_word16_t * *den*, celt_word16_t * *y*, int *N*, int *ord*, celt_word32_t * *mem*)

Definition at line 67 of file plc.c.

References ADD32, EXTEND32, EXTRACT16, MAC16_16, MULT16_16, NEG16, PSHR32, and SATURATE.

6.48 libcelt/plot_alloc.m File Reference

Functions

- x (1:2:end)
- x (2:2:end)
- y (1:2:end)
- y (2:2:end)
- axis ([0, 20000,-.02, 7])
- plot (x, y,'o-') xlabel('Allocation[bits/sample]')
- ylabel ('Frequency[Hz]')

Variables

- bps = alloc./(diff(band_def)-1)
- N = length(alloc)
- x = zeros(2∗N,1)
- y = zeros(2∗N,1)

6.48.1 Function Documentation

6.48.1.1 axis ()

6.48.1.2 plot (x, y, 'o-')

6.48.1.3 x (2:2:end)

6.48.1.4 x (1:2:end)

6.48.1.5 y (2:2:end)

6.48.1.6 y (1:2:end)

6.48.1.7 ylabel ('Frequency'[Hz])

6.48.2 Variable Documentation

6.48.2.1 bps = alloc./(diff(band_def)-1)

Definition at line 1 of file plot_alloc.m.

6.48.2.2 N = length(alloc)

Definition at line 2 of file plot_alloc.m.

Referenced by celt_encoder_create(), compute_alloc_cache(), compute_masking(), mdct_backward(), and mdct_forward().

6.48.2.3 x = zeros(2∗N,1)

Definition at line 3 of file plot_alloc.m.

6.48.2.4 y = zeros(2∗N,1)

Definition at line 4 of file plot_alloc.m.

Referenced by alg_quant().

6.49 libcelt/plot_BER.m File Reference

Functions

- axis([1e-6, 0.01, 1, 4.5])%gset xrange[1e-6 ylabel ('PESQ MOS')
- legend ('SECDED', 'No FEC')

6.49.1 Function Documentation

6.49.1.1 legend ('SECDED', 'No FEC')

6.49.1.2 axis ([1e-6,0.01,1,4.5]) %gset xrange [1e-6 ylabel ('PESQ MOS')

6.50 libcelt/plot_distribution.m File Reference

Functions

- clear FF FF (36)=0
- FF (1:2:35)
- FF (2:2:36)
- MM (1:2:35)
- MM (2:2:36)
- plot (FF, MM) xlabel('Frequency[Hz]')
- ylabel ('Bit depth[bit/sample]')

Variables

- gset xrange[0:20000] load distribution B = [0, 4, 8, 12, 16, 20, 24, 28, 32, 37, 43, 51, 62, 74, 90, 110, 140, 180, 233]
- M = mean(distribution)./diff(B)/16
- F = B(1:end)/512∗44100
- legend

6.50.1 Function Documentation

6.50.1.1 FF (2:2:36)

6.50.1.2 FF (1:2:35)

6.50.1.3 clear FF FF (36) [pure virtual]

6.50.1.4 MM (2:2:36)

6.50.1.5 MM (1:2:35)

6.50.1.6 plot (FF, MM)

6.50.1.7 ylabel ('Bit depth'[bit/sample])

6.50.2 Variable Documentation

6.50.2.1 gset xrange [0:20000] load distribution B = [0, 4, 8, 12, 16, 20, 24, 28, 32, 37, 43, 51, 62, 74, 90, 110, 140, 180, 233]

Definition at line 3 of file plot_distribution.m.

Referenced by celt_encode().

6.50.2.2 F = B(1:end)/512∗44100

Definition at line 4 of file plot_distribution.m.

6.50.2.3 legend

Definition at line 14 of file plot_distribution.m.

6.50.2.4 M = mean(distribution)./diff(B)/16

Definition at line 3 of file plot_distribution.m.

6.51 libcelt/plot_entropy.m File Reference

Functions

- plot([0:.05:1], min(MM')-offset, 'x-',.7, 56.2, '*b') plot([0 legend ('Lower bound', 'No prediction', 'Achieved(Laplace)', 1)
- legend ('Lower bound', 'No prediction')

Variables

- r = [8, 6, 5, 4, 3, 3, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
- offset = log2(6)*18-sum(log2(r))

6.51.1 Function Documentation

6.51.1.1 legend ('Lower bound', 'No prediction')

6.51.1.2 plot ([0:.05:1],min(MM')-offset, 'x-', .7, 56.2, '*b') plot ([0 legend ('Lower bound', 'No prediction', 'Achieved(Laplace)', 1)

6.51.2 Variable Documentation

6.51.2.1 offset = log2(6)*18-sum(log2(r))

Definition at line 2 of file plot_entropy.m.

Referenced by do_plc().

6.51.2.2 r = [8, 6, 5, 4, 3, 3, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]

Definition at line 1 of file plot_entropy.m.

Referenced by _celt_lpc(), ec_dec_tell(), ec_enc_tell(), ec_encode(), ec_encode_bin(), and stereo_mix().

6.52 libcelt/plot_latency.m File Reference

Functions

- load latency plot (latency(:, 2)/44.1, 8 ∗latency(:, 3)./latency(:, 1)∗44.1,'x-') xlabel('Algorithmic delay[ms]') ylabel('Bit-rate[kbit/s]') axis([0

6.52.1 Function Documentation

6.52.1.1 load latency plot (latency(:, 2)/44. *1*, 8 ∗latency(:, 3)./latency(:, 1)∗44. *1*, 'x-')

6.53 libcelt/plot_loss.m File Reference

Functions

- plot (X, 8000+(x)(256001:264000))
- plot (X,(y)(256001:264000))
- plot (X,-8000+(x-y)(256001:264000))
- xlabel ('Time[ms]')
- ylabel ('Amplitude')
- legend ('Error-free', 'Reconstructed', 'Error')

Variables

- x = read_audio('tata_errorfree.sw')
- y = read_audio('tata_loss.sw')
- hold off
- gset xrange [0:180]
- X = [0:7999]/44.1
- hold

6.53.1 Function Documentation

6.53.1.1 legend ('Error-free', 'Reconstructed', 'Error')

6.53.1.2 plot (X, -8000+(x-y)(256001:264000))

6.53.1.3 plot (X, (y)(256001:264000))

6.53.1.4 plot (X, 8000+(x)(256001:264000))

6.53.1.5 xlabel ('Time'[ms])

6.53.1.6 ylabel ('Amplitude')

6.53.2 Variable Documentation

6.53.2.1 hold

Definition at line 7 of file plot_loss.m.

6.53.2.2 hold off

Definition at line 3 of file plot_loss.m.

6.53.2.3 X = [0:7999]/44.1

Definition at line 5 of file plot_loss.m.

Referenced by find_spectral_pitch().

6.53.2.4 x = read_audio('tata_errorfree.sw')

Definition at line 1 of file plot_loss.m.

6.53.2.5 gset xrange[0:180]

Definition at line 4 of file plot_loss.m.

6.53.2.6 y = read_audio('tata_loss.sw')

Definition at line 2 of file plot_loss.m.

6.54 libcelt/plot_loss2.m File Reference

Functions

- plot (X, 15000+(x)(508501:512500))
- plot (X,(y)(508501:512500))
- plot (X,-15000+(x-y)(508501:512500))
- xlabel ('Time[ms]')
- ylabel ('')
- axis ('tickx')
- legend ('Error-free', 'Reconstructed', 'Error')

Variables

- x = read_audio('errorfree.sw')
- y = read_audio('plc2.sw')
- hold off
- gset xrange [0:90]
- gset yrange [-23000:26000]
- X = [0:3999]/44.1
- hold

6.54.1 Function Documentation

6.54.1.1 axis ('tickx')

6.54.1.2 legend ('Error-free', 'Reconstructed', 'Error')

6.54.1.3 plot (X, -15000+(x-y)(508501:512500))

6.54.1.4 plot (X, (y)(508501:512500))

6.54.1.5 plot (X, 15000+(x)(508501:512500))

6.54.1.6 xlabel ('Time'[ms])

6.54.1.7 ylabel ('')

6.54.2 Variable Documentation

6.54.2.1 hold

Definition at line 8 of file plot_loss2.m.

6.54.2.2 hold off

Definition at line 3 of file plot_loss2.m.

6.54.2.3 X = [0:3999]/44.1

Definition at line 6 of file plot_loss2.m.

6.54.2.4 x = read_audio('errorfree.sw')

Definition at line 1 of file plot_loss2.m.

6.54.2.5 gset xrange[0:90]

Definition at line 4 of file plot_loss2.m.

6.54.2.6 y = read_audio('plc2.sw')

Definition at line 2 of file plot_loss2.m.

6.54.2.7 gset yrange[-23000:26000]

Definition at line 5 of file plot_loss2.m.

6.55 libcelt/plot_PLC.m File Reference

Functions

- axis([0, 15, 1, 4.5]) load PLC plot(PLC(xlabel ('Packet loss(%)')
- ylabel ('PESQ MOS')

Variables

- legend

6.55.1 Function Documentation

6.55.1.1 axis ([0,15,1,4.5]) load PLC plot (PLC(xlabel ('Packet loss(%)')

6.55.1.2 ylabel ('PESQ MOS')

6.55.2 Variable Documentation

6.55.2.1 legend

Definition at line 6 of file plot_PLC.m.

6.56 libcelt/psy.c File Reference

```
#include "psy.h"
#include <math.h>
#include "os_support.h"
#include "arch.h"
```

Defines

- #define toBARK(n) (13.1f∗atan(.00074f∗(n))+2.24f∗atan((n)∗(n)∗1.85e-8f)+1e-4f∗(n))
- #define fromBARK(z) (102.f∗(z)-2.f∗pow(z,2.f)+.4f∗pow(z,3.f)+pow(1.46f,z)-1.f)

Functions

- void psydecay_init (struct PsyDecay ∗decay, int len, celt_int32_t Fs)
- void psydecay_clear (struct PsyDecay ∗decay)
- void compute_masking (const struct PsyDecay ∗decay, celt_word16_t ∗X, celt_mask_t ∗restrict mask, int len)

6.56.1 Define Documentation

6.56.1.1 #define fromBARK(z) (102.f∗(z)-2.f∗pow(z,2.f)+.4f∗pow(z,3.f)+pow(1.46f,z)-1.f)

Definition at line 43 of file psy.c.

6.56.1.2 #define toBARK(n) (13.1f∗atan(.00074f∗(n))+2.24f∗atan((n)∗(n)∗1.85e-8f)+1e-4f∗(n))

Definition at line 42 of file psy.c.

6.56.2 Function Documentation

6.56.2.1 void compute_masking (const struct PsyDecay ∗ *decay*, celt_word16_t ∗ *X*, celt_mask_t ∗restrict *mask*, int *len*)

Definition at line 132 of file psy.c.

References ADD32, MULT16_16, and N.

Referenced by find_spectral_pitch().

6.56.2.2 void psydecay_clear (struct PsyDecay ∗ *decay*)

Free the memory allocated for the spreading function

Definition at line 74 of file psy.c.

Referenced by celt_mode_destroy().

6.56.2.3 void psydecay_init (struct PsyDecay * *decay*, int *len*, celt_int32_t *Fs*)

Pre-compute the decay of the psycho-acoustic spreading function

Definition at line 50 of file psy.c.

Referenced by celt_mode_create().

6.57 libcelt/psy.h File Reference

```
#include "arch.h"
```

Data Structures

- struct PsyDecay

Functions

- void psydecay_init (struct PsyDecay *decay, int len, celt_int32_t Fs)
- void psydecay_clear (struct PsyDecay *decay)
- void compute_masking (const struct PsyDecay *decay, celt_word16_t *X, celt_mask_t *mask, int len)
- void compute_mdct_masking (const struct PsyDecay *decay, celt_word32_t *X, celt_mask_t *mask, int len)

6.57.1 Function Documentation

6.57.1.1 void compute_masking (const struct PsyDecay * *decay*, celt_word16_t * *X*, celt_mask_t * *mask*, int *len*)

Compute the masking curve for an input (DFT) spectrum X

6.57.1.2 void compute_mdct_masking (const struct PsyDecay * *decay*, celt_word32_t * *X*, celt_mask_t * *mask*, int *len*)

Compute the masking curve for an input (MDCT) spectrum X

Referenced by celt_encode().

6.57.1.3 void psydecay_clear (struct PsyDecay * *decay*)

Free the memory allocated for the spreading function

Definition at line 74 of file psy.c.

References PsyDecay::decayR.

6.57.1.4 void psydecay_init (struct PsyDecay * *decay*, int *len*, celt_int32_t *Fs*)

Pre-compute the decay of the psycho-acoustic spreading function

Definition at line 50 of file psy.c.

References PsyDecay::decayR, and Q15ONE.

6.58 libcelt/quant_bands.c File Reference

```
#include "quant_bands.h"
#include "laplace.h"
#include <math.h>
#include "os_support.h"
#include "arch.h"
#include "mathops.h"
#include "stack_alloc.h"
```

Functions

- int * quant_prob_alloc (const CELTMode *m)
- void quant_prob_free (int *freq)
- void quant_energy (const CELTMode *m, celt_ener_t *eBands, celt_word16_t *oldEBands, int budget, int *prob, ec_enc *enc)
- void unquant_energy (const CELTMode *m, celt_ener_t *eBands, celt_word16_t *oldEBands, int budget, int *prob, ec_dec *dec)

Variables

- const celt_word16_t eMeans [24] = {45.f, -8.f, -12.f, -2.5f, 1.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f}

6.58.1 Function Documentation

6.58.1.1 void quant_energy (const CELTMode * *m*, celt_ener_t * *eBands*, celt_word16_t * *oldEBands*, int *budget*, int * *prob*, ec_enc * *enc*)

Definition at line 322 of file quant_bands.c.

Referenced by celt_encode().

6.58.1.2 int* quant_prob_alloc (const CELTMode * *m*)

Definition at line 172 of file quant_bands.c.

Referenced by celt_mode_create().

6.58.1.3 void quant_prob_free (int * *freq*)

Definition at line 185 of file quant_bands.c.

Referenced by celt_mode_destroy().

6.58.1.4 void unquant_energy (const CELTMode * *m*, celt_ener_t * *eBands*, celt_word16_t * *oldEBands*, int *budget*, int * *prob*, ec_dec * *dec*)

Definition at line 384 of file quant_bands.c.

Referenced by celt_decode().

6.58.2 Variable Documentation

6.58.2.1 const celt_word16_t eMeans[24] = {45.f, -8.f, -12.f, -2.5f, 1.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f, 0.f}

Definition at line 47 of file quant_bands.c.

6.59 libcelt/quant_bands.h File Reference

```
#include "arch.h"
#include "modes.h"
#include "entenc.h"
#include "entdec.h"
```

Functions

- int * quant_prob_alloc (const CELTMode *m)
- void quant_prob_free (int *freq)
- void quant_energy (const CELTMode *m, celt_ener_t *eBands, celt_word16_t *oldEBands, int budget, int *prob, ec_enc *enc)
- void unquant_energy (const CELTMode *m, celt_ener_t *eBands, celt_word16_t *oldEBands, int budget, int *prob, ec_dec *dec)

6.59.1 Function Documentation

6.59.1.1 void quant_energy (const CELTMode * *m*, celt_ener_t * *eBands*, celt_word16_t * *oldEBands*, int *budget*, int * *prob*, ec_enc * *enc*)

Definition at line 322 of file quant_bands.c.

References ALLOC, celt_fatal, ENER_SCALING, ENER_SCALING_1, CELTMode::nbChannels, CELTMode::nbEBands, RESTORE_STACK, SAVE_STACK, and VARDECL.

6.59.1.2 int* quant_prob_alloc (const CELTMode * *m*)

Definition at line 172 of file quant_bands.c.

References ec_laplace_get_start_freq(), and CELTMode::nbEBands.

6.59.1.3 void quant_prob_free (int * *freq*)

Definition at line 185 of file quant_bands.c.

6.59.1.4 void unquant_energy (const CELTMode * *m*, celt_ener_t * *eBands*, celt_word16_t * *oldEBands*, int *budget*, int * *prob*, ec_dec * *dec*)

Definition at line 384 of file quant_bands.c.

References ALLOC, CELTMode::nbChannels, CELTMode::nbEBands, RESTORE_STACK, SAVE_STACK, and VARDECL.

6.60 libcelt/quant_pitch.c File Reference

```
#include "quant_pitch.h"
#include <math.h>
#include "pgain_table.h"
#include "arch.h"
#include "mathops.h"
```

Defines

- #define PGAIN_ODD(codebook, i) ((1.f/32768.f)*(celt_word16_-t)(((codebook)[(i)]&0x00ffU)<<7))
- #define PGAIN_EVEN(codebook, i) ((1.f/32768.f)*(celt_word16_-t)(((codebook)[(i)]&0xff00U)>>1))
- #define PGAIN_ODD14(codebook, i) PGAIN_ODD(codebook, i)
- #define PGAIN_EVEN14(codebook, i) PGAIN_EVEN(codebook, i)
- #define PGAIN(codebook, i) ((i)&1 ? PGAIN_ODD(codebook, (i)>>1) : PGAIN_-EVEN(codebook, (i)>>1))
- #define Q1515ONE MULT16_16(Q15ONE,Q15ONE)

Functions

- int vq_index (const celt_pgain_t *in, const celt_uint16_t *codebook, int len, int entries)
- int quant_pitch (celt_pgain_t *gains, int len, ec_enc *enc)
- int unquant_pitch (celt_pgain_t *gains, int len, ec_dec *dec)

6.60.1 Define Documentation

6.60.1.1 #define PGAIN(codebook, i) ((i)&1 ? PGAIN_ODD(codebook, (i)>>1) : PGAIN_EVEN(codebook, (i)>>1))

Definition at line 55 of file quant_pitch.c.

6.60.1.2 #define PGAIN_EVEN(codebook, i) ((1.f/32768.f)*(celt_word16_-t)(((codebook)[(i)]&0xff00U)>>1))

Definition at line 50 of file quant_pitch.c.

6.60.1.3 #define PGAIN_EVEN14(codebook, i) PGAIN_EVEN(codebook, i)

Definition at line 52 of file quant_pitch.c.

Referenced by vq_index().

6.60.1.4 #define PGAIN_ODD(codebook, i) ((1.f/32768.f)*(celt_word16_-t)(((codebook)[(i)]&0x00ffU)<<7))

Definition at line 49 of file quant_pitch.c.

6.60.1.5 #define PGAIN_ODD14(codebook, i) PGAIN_ODD(codebook, i)

Definition at line 51 of file quant_pitch.c.

Referenced by vq_index().

6.60.1.6 #define Q1515ONE MULT16_16(Q15ONE,Q15ONE)

Definition at line 58 of file quant_pitch.c.

Referenced by quant_pitch().

6.60.2 Function Documentation

6.60.2.1 int quant_pitch (celt_pgain_t * *gains*, int *len*, ec_enc * *enc*)

If this returns 0, the gain is zero (don't encode the pitch index)

Definition at line 95 of file quant_pitch.c.

Referenced by celt_encode().

6.60.2.2 int unquant_pitch (celt_pgain_t * *gains*, int *len*, ec_dec * *dec*)

If this returns 0, the gain is zero (don't decode the pitch index)

Definition at line 119 of file quant_pitch.c.

Referenced by celt_decode().

6.60.2.3 int vq_index (const celt_pgain_t * *in*, const celt_uint16_t * *codebook*, int *len*, int *entries*)

Taken from Speex.Finds the index of the entry in a codebook that best matches the input

Definition at line 61 of file quant_pitch.c.

References MAC16_16, PGAIN_EVEN14, PGAIN_ODD14, SUB16, and VERY_LARGE32.

Referenced by quant_pitch().

6.61 libcelt/quant_pitch.h File Reference

```
#include "arch.h"
#include "entenc.h"
#include "entdec.h"
```

Functions

- int quant_pitch (celt_pgain_t *gains, int len, ec_enc *enc)
- int unquant_pitch (celt_pgain_t *gains, int len, ec_dec *dec)

6.61.1 Function Documentation

6.61.1.1 int quant_pitch (celt_pgain_t * *gains*, int *len*, ec_enc * *enc*)

If this returns 0, the gain is zero (don't encode the pitch index)

Definition at line 95 of file quant_pitch.c.

References ADD32, celt_sqrt, ec_enc_bits(), EXTEND32, MULT16_16, pgain_table, Q1515ONE, Q15ONE, QCONST32, SHR16, and vq_index().

6.61.1.2 int unquant_pitch (celt_pgain_t * *gains*, int *len*, ec_dec * *dec*)

If this returns 0, the gain is zero (don't decode the pitch index)

Definition at line 119 of file quant_pitch.c.

References ec_dec_bits().

6.62 libcelt/rangedec.c File Reference

```
#include "arch.h"
#include "entdec.h"
#include "mfrngcod.h"
```

Functions

- void ec_dec_init (ec_dec *_this, ec_byte_buffer *_buf)
- unsigned ec_decode (ec_dec *_this, unsigned _ft)
- unsigned ec_decode_bin (ec_dec *_this, unsigned bits)
- void ec_dec_update (ec_dec *_this, unsigned _fl, unsigned _fh, unsigned _ft)
- long ec_dec_tell (ec_dec *_this, int _b)

6.62.1 Function Documentation

6.62.1.1 void ec_dec_init (ec_dec * *_this*, ec_byte_buffer * *_buf*)

Definition at line 155 of file rangedec.c.

References ec_dec::buf, ec_dec::dif, EC_CODE_EXTRA, EC_SYM_BITS, ec_dec::rem, and ec_dec::rng.

6.62.1.2 long ec_dec_tell (ec_dec * *_this*, int *_b*)

Definition at line 189 of file rangedec.c.

References b, ec_dec::buf, EC_CODE_BITS, EC_ILOG, EC_SYM_BITS, r, and ec_dec::rng.

6.62.1.3 void ec_dec_update (ec_dec * *_this*, unsigned *_fl*, unsigned *_fh*, unsigned *_ft*)

Definition at line 181 of file rangedec.c.

References ec_dec::dif, IMUL32, ec_dec::nrm, and ec_dec::rng.

6.62.1.4 unsigned ec_decode (ec_dec * *_this*, unsigned *_ft*)

Definition at line 165 of file rangedec.c.

References ec_dec::dif, EC_MINI, ec_dec::nrm, and ec_dec::rng.

6.62.1.5 unsigned ec_decode_bin (ec_dec * *_this*, unsigned *bits*)

Definition at line 172 of file rangedec.c.

References ec_dec::dif, EC_MINI, ec_dec::nrm, and ec_dec::rng.

6.63 libcelt/rangeenc.c File Reference

```
#include "arch.h"
#include "entenc.h"
#include "mfrngcod.h"
```

Functions

- void ec_enc_init (ec_enc *_this, ec_byte_buffer *_buf)
- void ec_encode (ec_enc *_this, unsigned _fl, unsigned _fh, unsigned _ft)
- void ec_encode_bin (ec_enc *_this, unsigned _fl, unsigned _fh, unsigned bits)
- long ec_enc_tell (ec_enc *_this, int _b)
- void ec_enc_done (ec_enc *_this)

6.63.1 Function Documentation

6.63.1.1 void ec_enc_done (ec_enc * *_this*)

Definition at line 133 of file rangeenc.c.

References EC_CODE_SHIFT, EC_CODE_TOP, ec_enc::ext, ec_enc::low, ec_enc::rem, and ec_enc::rng.

6.63.1.2 void ec_enc_init (ec_enc * *_this*, ec_byte_buffer * *_buf*)

Definition at line 77 of file rangeenc.c.

References ec_enc::buf, EC_CODE_TOP, ec_enc::ext, ec_enc::low, ec_enc::rem, and ec_enc::rng.

6.63.1.3 long ec_enc_tell (ec_enc * *_this*, int *_b*)

Definition at line 108 of file rangeenc.c.

References b, ec_enc::buf, EC_CODE_BITS, EC_ILOG, ec_enc::ext, r, ec_enc::rem, and ec_enc::rng.

6.63.1.4 void ec_encode (ec_enc * *_this*, unsigned *_fl*, unsigned *_fh*, unsigned *_ft*)

Definition at line 85 of file rangeenc.c.

References IMUL32, ec_enc::low, r, and ec_enc::rng.

6.63.1.5 void ec_encode_bin (ec_enc * *_this*, unsigned *_fl*, unsigned *_fh*, unsigned *bits*)

Definition at line 96 of file rangeenc.c.

References IMUL32, ec_enc::low, r, and ec_enc::rng.

6.64 libcelt/rate.c File Reference

```
#include <math.h>
#include "modes.h"
#include "cwrs.h"
#include "arch.h"
#include "os_support.h"
#include "entcode.h"
#include "rate.h"
```

Defines

- #define BITRES 4
- #define BITROUND 8
- #define BITOVERFLOW 10000

Functions

- void compute_alloc_cache (CELTMode *m)
- int compute_allocation (const CELTMode *m, int *offsets, int total, int *pulses)

6.64.1 Define Documentation

6.64.1.1 #define BITOVERFLOW 10000

Definition at line 47 of file rate.c.

Referenced by compute_alloc_cache().

6.64.1.2 #define BITRES 4

Definition at line 45 of file rate.c.

Referenced by compute_alloc_cache(), and compute_allocation().

6.64.1.3 #define BITROUND 8

Definition at line 46 of file rate.c.

6.64.2 Function Documentation

6.64.2.1 void compute_alloc_cache (CELTMode * *m*)

Computes a cache of the pulses->bits mapping in each band

Definition at line 100 of file rate.c.

Referenced by celt_mode_create().

6.64.2.2 int compute_allocation (const CELTMode * *m*, int * *offsets*, int *total*, int * *pulses*)

Compute the pulse allocation, i.e. how many pulses will go in each band.

Parameters:

m mode

offsets Requested increase or decrease in the number of bits for each band

total Number of bands

pulses Number of pulses per band (returned)

Returns:

Total number of bits allocated

Definition at line 255 of file rate.c.

Referenced by quant_bands(), and unquant_bands().

6.65 libcelt/rate.h File Reference

Defines

- #define MAX_PULSES 64
- #define LOG_MAX_PULSES 6

Functions

- void compute_alloc_cache (CELTMode *m)
- int compute_allocation (const CELTMode *m, int *offsets, int total, int *pulses)

6.65.1 Define Documentation

6.65.1.1 #define LOG_MAX_PULSES 6

Definition at line 36 of file rate.h.

6.65.1.2 #define MAX_PULSES 64

Definition at line 35 of file rate.h.

Referenced by compute_alloc_cache(), and dump_modes().

6.65.2 Function Documentation

6.65.2.1 void compute_alloc_cache (CELTMode * *m*)

Computes a cache of the pulses->bits mapping in each band

Definition at line 100 of file rate.c.

References ALLOC, BITOVERFLOW, BITRES, CELTMode::bits, CELTMode::eBands, MAX_PULSES, N, CELTMode::nbChannels, CELTMode::nbEBands, ncwrs_u64(), ncwrs_unext64(), CELTMode::pitchEnd, RESTORE_STACK, SAVE_STACK, and VARDECL.

6.65.2.2 int compute_allocation (const CELTMode * *m*, int * *offsets*, int *total*, int * *pulses*)

Compute the pulse allocation, i.e. how many pulses will go in each band.

Parameters:

m mode

offsets Requested increase or decrease in the number of bits for each band

total Number of bands

pulses Number of pulses per band (returned)

Returns:

Total number of bits allocated

Definition at line 255 of file rate.c.

References ALLOC, CELTMode::allocVectors, BITRES, CELTMode::nbAllocVectors, CELTMode::nbEBands, RESTORE_STACK, SAVE_STACK, and VARDECL.

6.66 libcelt/read_audio.m File Reference

Functions

- fclose (F)

Variables

- function x
- x = fread(F,Inf,'short')

6.66.1 Function Documentation

6.66.1.1 fclose (F)

6.66.2 Variable Documentation

6.66.2.1 x = fread(F,Inf,'short')

Definition at line 3 of file read_audio.m.

6.66.2.2 function x

Initial value:

```
read_audio(file)
F=fopen(file,'rb')
```

Definition at line 1 of file read_audio.m.

6.67 libcelt/stack_alloc.h File Reference

Temporary memory allocation on stack.

`#include "os_support.h"`

Defines

- #define ALIGN(stack, size) ((stack) += ((size) - (long)(stack)) & ((size) - 1))
- #define PUSH(stack, size, type) (ALIGN((stack),sizeof(type)/sizeof(char)),(stack)+=(size)∗(sizeof(type)/sizeof(char)),(ty (size)∗(sizeof(type)/sizeof(char))))
- #define RESTORE_STACK (global_stack = _saved_stack)
- #define VARDECL(type, var) type ∗var
- #define ALLOC(var, size, type) var = PUSH(global_stack, size, type)
- #define SAVE_STACK char ∗_saved_stack = global_stack;
- #define ALLOC_STACK (global_stack = (global_stack==0) ? celt_alloc_scratch(25000) : global_- stack)

Variables

- char ∗ global_stack

6.67.1 Detailed Description

Temporary memory allocation on stack.

Definition in file stack_alloc.h.

6.67.2 Define Documentation

6.67.2.1 #define ALIGN(stack, size) ((stack) += ((size) - (long)(stack)) & ((size) - 1))

Aligns the stack to a 'size' boundary

Parameters:

stack Stack

size New size boundary

Definition at line 118 of file stack_alloc.h.

6.67.2.2 #define ALLOC(var, size, type) var = PUSH(global_stack, size, type)

Allocate 'size' elements of 'type' on stack

Parameters:

var Name of variable to allocate

size Number of elements

type Type of element

Definition at line 132 of file stack_alloc.h.

Referenced by alg_quant(), alg_unquant(), celt_decode(), celt_encode(), compute_alloc_cache(), compute_allocation(), decode_pulses(), encode_pulses(), find_spectral_pitch(), mdct_backward(), mdct_forward(), quant_bands(), quant_energy(), renormalise_bands(), unquant_bands(), and unquant_energy().

6.67.2.3 #define ALLOC_STACK (global_stack = (global_stack==0) ? celt_alloc_scratch(25000) : global_stack)

Definition at line 134 of file stack_alloc.h.

Referenced by celt_mode_create().

6.67.2.4 #define PUSH(stack, size, type) (ALIGN((stack),sizeof(type)/sizeof(char)),(stack)+=(size)∗(sizeof(type)/sizeof(char)),(type∗)((stack)-(size)∗(sizeof(type)/sizeof(char))))

Allocates 'size' elements of type 'type' on the stack

Parameters:

stack Stack

size Number of elements

type Type of element

Definition at line 119 of file stack_alloc.h.

6.67.2.5 #define RESTORE_STACK (global_stack = _saved_stack)

Definition at line 120 of file stack_alloc.h.

Referenced by alg_quant(), alg_unquant(), celt_decode(), celt_encode(), compute_alloc_cache(), compute_allocation(), decode_pulses(), encode_pulses(), find_spectral_pitch(), mdct_backward(), mdct_forward(), quant_bands(), quant_energy(), renormalise_bands(), unquant_bands(), and unquant_energy().

6.67.2.6 #define SAVE_STACK char ∗_saved_stack = global_stack;

Definition at line 133 of file stack_alloc.h.

Referenced by alg_quant(), alg_unquant(), celt_decode(), celt_encode(), compute_alloc_cache(), compute_allocation(), decode_pulses(), encode_pulses(), find_spectral_pitch(), mdct_backward(), mdct_forward(), quant_bands(), quant_energy(), renormalise_bands(), unquant_bands(), and unquant_energy().

6.67.2.7 #define VARDECL(type, var) type ∗var

Declare variable on stack

Parameters:

var Variable to declare

Definition at line 131 of file stack_alloc.h.

Referenced by alg_quant(), alg_unquant(), celt_decode(), celt_encode(), compute_alloc_cache(), compute_allocation(), decode_pulses(), encode_pulses(), find_spectral_pitch(), mdct_backward(), mdct_forward(), quant_bands(), quant_energy(), renormalise_bands(), unquant_bands(), and unquant_energy().

6.67.3 Variable Documentation

6.67.3.1 char* global_stack

6.68 libcelt/static_modes.c File Reference

```
#include "modes.h"
#include "rate.h"
```

Defines

- #define allocCache_band1_51200_256_1 allocCache_band0_51200_256_1
- #define allocCache_band2_51200_256_1 allocCache_band1_51200_256_1
- #define allocCache_band3_51200_256_1 allocCache_band2_51200_256_1
- #define allocCache_band4_51200_256_1 allocCache_band3_51200_256_1
- #define allocCache_band5_51200_256_1 allocCache_band4_51200_256_1
- #define allocCache_band6_51200_256_1 allocCache_band5_51200_256_1
- #define allocCache_band7_51200_256_1 allocCache_band6_51200_256_1
- #define TOTAL_MODES 1

6.68.1 Define Documentation

6.68.1.1 #define allocCache_band1_51200_256_1 allocCache_band0_51200_256_1

Definition at line 50 of file static_modes.c.

6.68.1.2 #define allocCache_band2_51200_256_1 allocCache_band1_51200_256_1

Definition at line 51 of file static_modes.c.

6.68.1.3 #define allocCache_band3_51200_256_1 allocCache_band2_51200_256_1

Definition at line 52 of file static_modes.c.

6.68.1.4 #define allocCache_band4_51200_256_1 allocCache_band3_51200_256_1

Definition at line 53 of file static_modes.c.

6.68.1.5 #define allocCache_band5_51200_256_1 allocCache_band4_51200_256_1

Definition at line 54 of file static_modes.c.

6.68.1.6 #define allocCache_band6_51200_256_1 allocCache_band5_51200_256_1

Definition at line 55 of file static_modes.c.

6.68.1.7 #define allocCache_band7_51200_256_1 allocCache_band6_51200_256_1

Definition at line 56 of file static_modes.c.

6.68.1.8 #define TOTAL_MODES 1

Definition at line 103 of file static_modes.c.

Referenced by celt_mode_create().

6.69 libcelt/static_modes.h File Reference

Defines

- #define CHANNELS(mode) 1
- #define DISABLE_STEREO
- #define FRAMESIZE(mode) 256
- #define OVERLAP(mode) 128

6.69.1 Define Documentation

6.69.1.1 #define CHANNELS(mode) 1

Definition at line 2 of file static_modes.h.

6.69.1.2 #define DISABLE_STEREO

Definition at line 3 of file static_modes.h.

6.69.1.3 #define FRAMESIZE(mode) 256

Definition at line 4 of file static_modes.h.

6.69.1.4 #define OVERLAP(mode) 128

Definition at line 5 of file static_modes.h.

6.70 libcelt/static_modes_rev.h File Reference

```
#include "modes.h"
#include "rate.h"
```

Defines

- #define allocCache_band1_51200_256_1 allocCache_band0_51200_256_1
- #define allocCache_band2_51200_256_1 allocCache_band1_51200_256_1
- #define allocCache_band3_51200_256_1 allocCache_band2_51200_256_1
- #define allocCache_band4_51200_256_1 allocCache_band3_51200_256_1
- #define allocCache_band5_51200_256_1 allocCache_band4_51200_256_1
- #define allocCache_band6_51200_256_1 allocCache_band5_51200_256_1
- #define allocCache_band7_51200_256_1 allocCache_band6_51200_256_1
- #define TOTAL_MODES 1

6.70.1 Define Documentation

6.70.1.1 #define allocCache_band1_51200_256_1 allocCache_band0_51200_256_1

Definition at line 50 of file static_modes_rev.h.

6.70.1.2 #define allocCache_band2_51200_256_1 allocCache_band1_51200_256_1

Definition at line 51 of file static_modes_rev.h.

6.70.1.3 #define allocCache_band3_51200_256_1 allocCache_band2_51200_256_1

Definition at line 52 of file static_modes_rev.h.

6.70.1.4 #define allocCache_band4_51200_256_1 allocCache_band3_51200_256_1

Definition at line 53 of file static_modes_rev.h.

6.70.1.5 #define allocCache_band5_51200_256_1 allocCache_band4_51200_256_1

Definition at line 54 of file static_modes_rev.h.

6.70.1.6 #define allocCache_band6_51200_256_1 allocCache_band5_51200_256_1

Definition at line 55 of file static_modes_rev.h.

6.70.1.7 #define allocCache_band7_51200_256_1 allocCache_band6_51200_256_1

Definition at line 56 of file static_modes_rev.h.

6.70.1.8 #define TOTAL_MODES 1

Definition at line 102 of file static_modes_rev.h.

6.71 libcelt/testcelt.c File Reference

```
#include "celt.h"
#include "arch.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
```

Functions

- int main (int argc, char *argv[ ])

6.71.1 Function Documentation

6.71.1.1 int main (int *argc*, char * *argv*[ ])

Definition at line 43 of file testcelt.c.

References celt_decode(), celt_decoder_create(), celt_decoder_destroy(), celt_encode(), celt_encoder_create(), celt_encoder_destroy(), CELT_GET_FRAME_SIZE, CELT_GET_NB_CHANNELS, celt_mode_create(), celt_mode_destroy(), celt_mode_info(), fclose(), fprintf(), and fwrite().

6.72 libcelt/useful.m File Reference

6.73 libcelt/ventropy.m File Reference

Variables

- function v

6.73.1 Variable Documentation

6.73.1.1 function v

Initial value:

```
 ventropy(M)

for k=1:size(M,2)
   v(k) = entropy2(M(:,k))
```

Definition at line 1 of file ventropy.m.

Referenced by ec_ilog64().

6.74 libcelt/vq.c File Reference

```
#include "mathops.h"
#include "cwrs.h"
#include "vq.h"
#include "arch.h"
#include "os_support.h"
```

Defines

- #define KGAIN 6

Functions

- void alg_quant (celt_norm_t *X, celt_mask_t *W, int N, int K, const celt_norm_t *P, ec_enc *enc)
- void alg_unquant (celt_norm_t *X, int N, int K, celt_norm_t *P, ec_dec *dec)
- void intra_prediction (const CELTMode *m, celt_norm_t *restrict x, celt_mask_t *W, int N, int K, celt_norm_t *Y, celt_norm_t *restrict P, int N0, int Nmax, ec_enc *enc)
- void intra_unquant (const CELTMode *m, celt_norm_t *x, int N, int K, celt_norm_t *Y, celt_norm_t *restrict P, int N0, int Nmax, ec_dec *dec)
- void intra_fold (const CELTMode *m, celt_norm_t *x, int N, celt_norm_t *Y, celt_norm_t *restrict P, int N0, int Nmax)

6.74.1 Define Documentation

6.74.1.1 #define KGAIN 6

Definition at line 262 of file vq.c.

Referenced by intra_prediction(), and intra_unquant().

6.74.2 Function Documentation

6.74.2.1 void alg_quant (celt_norm_t * *X*, celt_mask_t * *W*, int *N*, int *K*, const celt_norm_t * *P*, ec_enc * *enc*)

Algebraic pulse-vector quantiser. The signal x is replaced by the sum of the pitch and a combination of pulses such that its norm is still equal to 1. This is the function that will typically require the most CPU.

Parameters:

x Residual signal to quantise/encode (returns quantised version)

W Perceptual weight to use when optimising (currently unused)

N Number of samples to encode

K Number of pulses to use

p Pitch vector (it is assumed that p+x is a unit vector)

enc Entropy encoder state

Definition at line 93 of file vq.c.

Referenced by quant_bands().

6.74.2.2 void alg_unquant (celt_norm_t * *X*, int *N*, int *K*, celt_norm_t * *P*, ec_dec * *dec*)

Decode pulse vector and combine the result with the pitch vector to produce the final normalised signal in the current band.

Definition at line 251 of file vq.c.

Referenced by unquant_bands().

6.74.2.3 void intra_fold (const CELTMode * *m*, celt_norm_t * *x*, int *N*, celt_norm_t * *Y*, celt_norm_t *restrict *P*, int *N0*, int *Nmax*)

Definition at line 363 of file vq.c.

References celt_rcp, celt_sqrt, CHANNELS, EPSILON, MAC16_16, MULT16_16, PSHR32, and SHL32.

Referenced by quant_bands(), and unquant_bands().

6.74.2.4 void intra_prediction (const CELTMode * *m*, celt_norm_t *restrict *x*, celt_mask_t * *W*, int *N*, int *K*, celt_norm_t * *Y*, celt_norm_t *restrict *P*, int *N0*, int *Nmax*, ec_enc * *enc*)

Intra-frame predictor that matches a section of the current frame (at lower frequencies) to encode the current band.

Parameters:

- ***x*** Residual signal to quantise/encode (returns quantised version)
- ***W*** Perceptual weight
- ***N*** Number of samples to encode
- ***K*** Number of pulses to use
- ***Y*** Lower frequency spectrum to use, normalised to the same standard deviation
- ***P*** Pitch vector (it is assumed that p+x is a unit vector)
- ***B*** Stride (number of channels multiplied by the number of MDCTs per frame)
- ***N0*** Number of valid offsets
- ***enc*** Entropy encoder state

Definition at line 264 of file vq.c.

Referenced by quant_bands().

6.74.2.5 void intra_unquant (const CELTMode * *m*, celt_norm_t * *x*, int *N*, int *K*, celt_norm_t * *Y*, celt_norm_t *restrict *P*, int *N0*, int *Nmax*, ec_dec * *dec*)

Definition at line 320 of file vq.c.

References celt_div, celt_rcp, celt_sqrt, CHANNELS, ec_dec_bits(), EPSILON, KGAIN, MAC16_16, MULT16_16, MULT16_16_Q15, PSHR32, Q15_ONE, and SHL32.

Referenced by unquant_bands().

6.75 libcelt/vq.h File Reference

Vector quantisation of the residual.

```
#include "entenc.h"
#include "entdec.h"
#include "modes.h"
```

Functions

- void alg_quant (celt_norm_t *X, celt_mask_t *W, int N, int K, const celt_norm_t *P, ec_enc *enc)
- void alg_unquant (celt_norm_t *X, int N, int K, celt_norm_t *P, ec_dec *dec)
- void intra_prediction (const CELTMode *m, celt_norm_t *restrict x, celt_mask_t *W, int N, int K, celt_norm_t *Y, celt_norm_t *restrict P, int N0, int Nmax, ec_enc *enc)
- void intra_unquant (const CELTMode *m, celt_norm_t *x, int N, int K, celt_norm_t *Y, celt_norm_t *P, int N0, int Nmax, ec_dec *dec)
- void intra_fold (const CELTMode *m, celt_norm_t *x, int N, celt_norm_t *Y, celt_norm_t *P, int N0, int Nmax)

6.75.1 Detailed Description

Vector quantisation of the residual.

Definition in file vq.h.

6.75.2 Function Documentation

6.75.2.1 void alg_quant (celt_norm_t * *X*, celt_mask_t * *W*, int *N*, int *K*, const celt_norm_t * *P*, ec_enc * *enc*)

Algebraic pulse-vector quantiser. The signal x is replaced by the sum of the pitch and a combination of pulses such that its norm is still equal to 1. This is the function that will typically require the most CPU.

Parameters:

x Residual signal to quantise/encode (returns quantised version)

W Perceptual weight to use when optimising (currently unused)

N Number of samples to encode

K Number of pulses to use

p Pitch vector (it is assumed that p+x is a unit vector)

enc Entropy encoder state

Definition at line 93 of file vq.c.

References ADD16, ADD32, ALLOC, celt_assert2, celt_ilog2, celt_psqrt, encode_pulses(), EXTRACT16, MAC16_16, MULT16_16, MULT16_16_Q15, NORM_SCALING, QCONST16, RESTORE_STACK, ROUND16, SAVE_STACK, SHL16, SHR32, SUB16, VARDECL, VERY_LARGE16, and y.

6.75.2.2 void alg_unquant (celt_norm_t * *X*, int *N*, int *K*, celt_norm_t * *P*, ec_dec * *dec*)

Algebraic pulse decoder

Parameters:

x Decoded normalised spectrum (returned)

N Number of samples to decode

K Number of pulses to use

p Pitch vector (automatically added to x)

dec Entropy decoder state

Decode pulse vector and combine the result with the pitch vector to produce the final normalised signal in the current band.

Definition at line 251 of file vq.c.

References ALLOC, decode_pulses(), RESTORE_STACK, SAVE_STACK, and VARDECL.

6.75.2.3 void intra_fold (const CELTMode * *m*, celt_norm_t * *x*, int *N*, celt_norm_t * *Y*, celt_norm_t * *P*, int *N0*, int *Nmax*)

Encode the entire band as a "fold" from other parts of the spectrum. No bits required (only use is case of an emergency!)

6.75.2.4 void intra_prediction (const CELTMode * *m*, celt_norm_t *restrict *x*, celt_mask_t * *W*, int *N*, int *K*, celt_norm_t * *Y*, celt_norm_t *restrict *P*, int *N0*, int *Nmax*, ec_enc * *enc*)

Intra-frame predictor that matches a section of the current frame (at lower frequencies) to encode the current band.

Parameters:

x Residual signal to quantise/encode (returns quantised version)

W Perceptual weight

N Number of samples to encode

K Number of pulses to use

Y Lower frequency spectrum to use, normalised to the same standard deviation

P Pitch vector (it is assumed that p+x is a unit vector)

B Stride (number of channels multiplied by the number of MDCTs per frame)

N0 Number of valid offsets

enc Entropy encoder state

Definition at line 264 of file vq.c.

References celt_div, celt_rcp, celt_sqrt, CHANNELS, ec_enc_bits(), EPSILON, KGAIN, MAC16_16, MULT16_16, MULT16_16_Q15, PSHR32, Q15_ONE, and SHL32.

6.75.2.5 void intra_unquant (const CELTMode * *m*, celt_norm_t * *x*, int *N*, int *K*, celt_norm_t * *Y*, celt_norm_t * *P*, int *N0*, int *Nmax*, ec_dec * *dec*)

6.76 libcelt/write_audio.m File Reference

Functions

- function write_audio (x, file) F
- fwrite (F, x,'short')
- fclose (F)

6.76.1 Function Documentation

6.76.1.1 fclose (F)

6.76.1.2 fwrite (F, x, 'short')

Referenced by main().

6.76.1.3 function write_audio (x, file)